

# การพัฒนาเกมโดยประยุกต์ใช้เทคนิค Finite State Machine ในการควบคุมตัวละครและปัญญาประดิษฐ์

# Development of a Game using Finite State Machine Techniques for Character Control and Artificial Intelligence


**ชื่อผู้จัดทำ**

นายจิรายุ สงบวาจา

นางสาวอรไพลิน สอาดเอี่ยม

นางสาววนิดา คล้ายสกุล



**โครงงานวิจัยระดับปริญญาตรีนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิทยาศาสตรบัณฑิต**
**สาขาวิชาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์และเทคโนโลยี**
**มหาวิทยาลัยเทคโนโลยีราชมงคลสุวรรณภูมิ พ.ศ. 2569**

# การพัฒนาเกมโดยประยุกต์ใช้เทคนิค Finite State Machine ในการควบคุมตัวละครและปัญญาประดิษฐ์

# Development of a Game using Finite State Machine Techniques for Character Control and Artificial Intelligence


**ชื่อผู้จัดทำ**

นายจิรายุ สงบวาจา

นางสาวอรไพลิน สอาดเอี่ยม

นางสาววนิดา คล้ายสกุล



**โครงงานวิจัยระดับปริญญาตรีนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิทยาศาสตรบัณฑิต**
**สาขาวิชาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์และเทคโนโลยี**
**มหาวิทยาลัยเทคโนโลยีราชมงคลสุวรรณภูมิ พ.ศ. 2569**

| | |
|---|---|
| **หัวข้อโครงงานวิจัย** | การพัฒนาเกมโดยประยุกต์ใช้เทคนิค Finite State Machine ในการควบคุมตัวละครและปัญญาประดิษฐ์<br>Development of a Game using Finite State Machine Techniques for Character Control and Artificial Intelligence |
| **ชื่อนักศึกษา** | นายจิรายุ สงบวาจา<br>นางสาวอรไพลิน สอาดเอี่ยม<br>นางสาววนิดา คล้ายสกุล |
| **อาจารย์ที่ปรึกษา** | อาจารย์ธนวัฒน์ ถาวรกูล<br>ผศ.วราภรณ์ มั่นทุ่ง |
| **ระดับการศึกษา** | ปริญญาตรี |
| **สาขาวิชา** | วิทยาการคอมพิวเตอร์ |
| **ปีการศึกษา** | 2569 |

## บทคัดย่อ


โครงงานวิจัยนี้มีวัตถุประสงค์เพื่อศึกษาและพัฒนาเกมคอมพิวเตอร์สองมิติแนวเอาชีวิตรอดและลอบเร้น (Survival Stealth Game) โดยมุ่งเน้นการประยุกต์ใช้เทคนิคเครื่องสถานะจำกัด (Finite State Machine: FSM) ในการควบคุมสถานะของตัวละครผู้เล่นและระบบปัญญาประดิษฐ์ (Artificial Intelligence: AI) ภายในเกม เพื่อให้การทำงานของระบบมีโครงสร้างที่ชัดเจน ลดความซับซ้อนของโปรแกรม และเพิ่มประสิทธิภาพในการควบคุมพฤติกรรมของตัวละคร การพัฒนาเกมใช้โปรแกรม Unity Game Engine และเขียนโปรแกรมด้วยภาษา C# โดยนำแนวคิด FSM มาใช้เป็นโครงสร้างหลักในการจัดการสถานะของตัวละครผู้เล่น ได้แก่ การยืน การเดิน การนั่งย่อ และการหมอบคลาน ซึ่งแต่ละสถานะมีผลต่อพารามิเตอร์ภายในเกม เช่น ความเร็วในการเคลื่อนที่ ขนาดพื้นที่การชน (Collider) และระดับความดังของเสียงแบบเรียลไทม์ นอกจากนี้ได้พัฒนาระบบปัญญาประดิษฐ์ของตัวละครศัตรูให้สามารถตรวจจับผู้เล่นผ่านระบบการมองเห็นด้วยเทคนิค Raycasting และตอบสนองต่อระดับเสียงที่ผู้เล่นสร้างขึ้น.โดยทำงานร่วมกับอัลกอริทึมค้นหาเส้นทางแบบ A* (A* Pathfinding) เพื่อให้ศัตรูสามารถเคลื่อนที่หลบหลีกสิ่งกีดขวางและไล่ล่าผู้เล่นได้อย่างสมจริง ผลการดำเนินงานพบว่า การประยุกต์ใช้เทคนิค Finite State Machine ช่วยให้สามารถจัดการสถานะของตัวละครและเงื่อนไขการทำงานของระบบ AI ได้อย่างเป็นระบบ ลดความซับซ้อนของโค้ด และเพิ่มความยืดหยุ่นในการพัฒนา ซอฟต์แวร์ต้นแบบที่พัฒนาขึ้นสามารถตอบสนองต่อคำสั่งของผู้เล่นได้อย่างถูกต้อง และตัวละครศัตรูมีพฤติกรรมการไล่ล่าที่สอดคล้อง

กับตรรกะที่กำหนดไว้ บรรลุตามวัตถุประสงค์ของโครงงานในการประยุกต์ใช้ความรู้ด้านวิทยาการคอมพิวเตอร์เพื่อพัฒนาสื่อบันเทิงเชิงโต้ตอบ

# กิตติกรรมประกาศ

โครงงานวิจัยฉบับนี้สำเร็จลุล่วงไปได้ด้วยดีเนื่องด้วยความกรุณาจาก อาจารย์ธนวัฒน์ ถาวรกูล และ ผศ.วราภรณ์ มั่นทุ่ง อาจารย์ที่ปรึกษาโครงงาน ที่ได้ให้คำปรึกษา แนะนำแนวทาง และแก้ไขข้อบกพร่องต่าง ๆ ในการพัฒนาเกมสยองขวัญ 2 มิติเชิงจริยธรรมด้วยภาษา C# ตลอดจนถ่ายทอดความรู้และเทคนิคต่าง ๆ ในการใช้โปรแกรม Unity จนโครงงานเล่มนี้เสร็จสมบูรณ์

ขอกราบขอบพระคุณบิดา มารดา และครอบครัว ที่ให้การสนับสนุนทั้งในด้านกำลังใจและทุนทรัพย์ในการศึกษาเล่าเรียนเสมอมา รวมถึงขอขอบคุณเพื่อนนักศึกษาทุกท่านที่ให้ความช่วยเหลือในการทดสอบระบบเกมและให้คำแนะนำที่เป็นประโยชน์ต่อการพัฒนาโครงงานในครั้งนี้

คุณค่าและประโยชน์ของโครงงานวิจัยฉบับนี้ ผู้จัดทำขอมอบเป็นเครื่องบูชาพระคุณบิดา มารดา ครูบาอาจารย์ และผู้มีพระคุณทุกท่านที่มีส่วนช่วยให้ผู้จัดทำประสบความสำเร็จในการศึกษา

ชื่อผู้จัดทำ

จิรายุ สงบวาจา

อรไพลิน สอาดเอี่ยม

วนิดา คล้ายสกุล

# บทที่ 1

# บทนำ

## 1.1 ความเป็นมาและความสำคัญของปัญหา

การพัฒนาเกมคอมพิวเตอร์ในปัจจุบันมีความซับซ้อนทางด้านวิศวกรรมซอฟต์แวร์มากขึ้น โดยเฉพาะในเกมแนวลอบเร้น (Stealth Survival) หัวใจสำคัญไม่ได้อยู่ที่ความสวยงามเพียงอย่างเดียว แต่อยู่ที่ "การออกแบบโครงสร้างโค้ด (Software Architecture)" เพื่อควบคุมสถานะของตัวละคร (Player State Management) ที่มีความละเอียดซับซ้อน เช่น การเปลี่ยนอิริยาบถจากการยืน เป็นการนั่งย่อ หรือการหมอบคลาน ซึ่งการกระทำเหล่านี้ต้องส่งผลต่อค่าพารามิเตอร์ต่างๆ ทางฟิสิกส์และการตรวจจับภายในเกมแบบเรียลไทม์

นอกจากนี้ การสร้างศัตรูคอมพิวเตอร์ (AI Agent) ให้มีความฉลาดและตอบสนองต่อพฤติกรรมของผู้เล่นอย่างสมจริง จำเป็นต้องประยุกต์ใช้อัลกอริทึมขั้นสูง เช่น Finite State Machine (FSM) ในการจำแนกสถานะ และการคำนวณทางคณิตศาสตร์เวกเตอร์ (Vector Mathematics) เพื่อตรวจสอบการมองเห็น (Line of Sight) และการได้ยินเสียงที่แปรผันตามท่าทางของผู้เล่น

ผู้จัดทำจึงมีแนวคิดพัฒนาโครงงาน "การพัฒนาเกมโดยประยุกต์ใช้เทคนิค Finite State Machine ในการควบคุมตัวละครและปัญญาประดิษฐ์" โดยใช้ Unity Game Engine และภาษา C# โครงงานนี้จะมุ่งเน้นการเขียนโปรแกรมเพื่อจัดการกลไกการเคลื่อนไหวที่ซับซ้อน (Advanced Movement Mechanics) และระบบ AI ที่ไล่ล่าผู้เล่นอย่างเป็นระบบ เพื่อเป็นกรณีศึกษาการนำความรู้วิทยาการคอมพิวเตอร์มาประยุกต์ใช้ในการสร้างซอฟต์แวร์ที่มีการโต้ตอบสูง

## 1.2 วัตถุประสงค์ของโครงงาน

1.2.1 เพื่อพัฒนาเกมคอมพิวเตอร์ 2 มิติ ด้วยภาษา C# โดยเน้นการวางโครงสร้างโค้ดแบบเชิงวัตถุ (Object-Oriented Programming)

1.2.2 เพื่อประยุกต์ใช้โครงสร้างข้อมูลแบบ Finite State Machine (FSM) ในการจัดการสถานะการเคลื่อนไหวของตัวละครผู้เล่น (Movement States

1.2.3 เพื่อพัฒนาอัลกอริทึมปัญญาประดิษฐ์ (AI) สำหรับศัตรูที่สามารถตอบสนองต่อระดับเสียงและขนาดพื้นที่การมองเห็น (Hitbox) ที่เปลี่ยนแปลงไปตามสถานะของผู้เล่นได้

1.2.4 เพื่อสร้างระบบการเล่น (Gameplay Loop) ที่สมบูรณ์ ทั้งการเก็บไอเทม การแก้ไขปริศนาและการเปลี่ยนฉากเมื่อผ่านเงื่อนไข

1.2.5 เพื่อประเมินความพึงพอใจของผู้เล่นต่อเกมที่พัฒนาขึ้น เพื่อนำไปปรับปรุงและพัฒนาระบบให้มีประสิทธิภาพยิ่งขึ้น

**1.3 ขอบเขตของโครงงาน**

1.3.1 ส่วนพัฒนาระบบควบคุมตัวละคร (Player Controller System)

พัฒนาระบบควบคุมตัวละครผู้เล่นให้สามารถเคลื่อนไหวได้หลากหลายรูปแบบ และส่งผลต่อการเล่นในเชิงกลยุทธ์ เช่น การลดเสียงเพื่อหลบศัตรู

1.3.1.1 State Management

พัฒนาระบบสถานะของตัวละครโดยใช้ Finite State Machine (FSM) เพื่อควบคุมการเปลี่ยนสถานะระหว่าง Idle, Walk, Crouch และ Prone อย่างเป็นลำดับ

1.3.1.2 Dynamic Collider Adjustment

พัฒนาระบบปรับขนาด Hitbox ตามท่าทางของตัวละครแบบเรียลไทม์ เพื่อรองรับการเคลื่อนที่ในพื้นที่จำกัด เช่น การหมอบลอดช่องแคบ

1.3.1.3 Variable Parameters

กำหนดค่าความเร็วและระดับเสียงในแต่ละสถานะ เพื่อใช้เป็นข้อมูลในการตรวจจับของศัตรู

1.3.2 ส่วนพัฒนาระบบปัญญาประดิษฐ์ของศัตรู (Enemy AI System)

พัฒนาระบบ AI ที่สามารถตอบสนองต่อพฤติกรรมของผู้เล่น โดยเน้นการตรวจจับและการตัดสินใจแบบเรียลไทม์

1.3.2.1 Behavior System (FSM)

กำหนดสถานะของศัตรู เช่น Patrol, Investigate, Chaseและ Attack เพื่อควบคุมพฤติกรรม

1.3.2.2 Sensory System

พัฒนาระบบการมองเห็นด้วย Raycasting และระบบการได้ยินโดยใช้การคำนวณระยะทางร่วมกับระดับเสียงของผู้เล่น

1.3.2.3 Pathfinding

ใช้ Navigation Mesh และ A* Algorithm เพื่อให้ศัตรูสามารถเคลื่อนที่หลบสิ่งกีดขวางและติดตามผู้เล่นได้

1.3.3 ส่วนพัฒนาระบบเกม (Game Logic System)

พัฒนากลไกหลักของเกมเพื่อกำหนดเป้าหมายและลำดับการเล่น

1.3.3.1 Inventory และ Mission Logic

ผู้เล่นต้องเก็บไอเทมให้ครบจำนวนที่กำหนด (เช่น 5 ชิ้น) เพื่อปลดล็อกทางออก

1.3.3.2 Scene Management

รองรับการเปลี่ยนฉากและการเก็บข้อมูลความคืบหน้าของเกม

1.3.4 ส่วนออกแบบส่วนติดต่อผู้ใช้งาน (UI/UX)

เป็นการออกแบบส่วนติดต่อผู้ใช้เพื่อแสดงข้อมูลสำคัญ เช่น สถานะตัวละคร ระดับเสียง และจำนวนไอเทม เพื่อช่วยให้ผู้เล่นสามารถวางแผนการเล่นและตัดสินใจได้อย่างมีประสิทธิภาพ

1.3.5 ขอบเขตด้านเนื้อหาเกม (Game Content Scope)

1.3.5.1 เกมที่พัฒนาเป็นเกมคอมพิวเตอร์แบบ 2 มิติ แนวลอบเร้น (Stealth Gameโดยผู้เล่นต้องหลบหลีกศัตรูและวางแผนการเคลื่อนที่

1.3.5.2 ภายในเกมมีฉากการเล่นอย่างน้อย 1–2 ฉาก ซึ่งออกแบบให้มีสิ่งกีดขวางและพื้นที่สำหรับการหลบซ่อน

1.3.5.3 มีตัวละครศัตรูอย่างน้อย 1 ประเภท ที่สามารถลาดตระเวน ตรวจจับเสียง และไล่ล่าผู้เล่นได้

1.3.5.4 มีระบบการเก็บไอเทม โดยผู้เล่นต้องรวบรวมไอเทมให้ครบตามจำนวนที่กำหนดเพื่อผ่านด่าน

1.3.5.5 มีเงื่อนไขการชนะ (Win Condition) เมื่อผู้เล่นเก็บไอเทมครบและไปถึงทางออกของฉาก

1.3.5.6 มีเงื่อนไขการแพ้ (Lose Condition) เมื่อผู้เล่นถูกศัตรูตรวจจับและพลังชีวิหมด

1.3.6 ขอบเขตด้านเทคโนโลยี (Technology Scope)

1.3.6.1 ใช้ Unity Game Engine ในการพัฒนาเกม

1.3.6.2 ใช้ภาษา C# ในการเขียนโปรแกรมควบคุมระบบต่าง ๆ ภายในเกม

1.3.6.3 ใช้แนวคิดการเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Programming OOP) ในการออกแบบโครงสร้างระบบ

1.3.6.4 ใช้เทคนิค Finite State Machine (FSM) ในการควบคุมพฤติกรรมของตัละคผู้เล่นและศัตรู

1.3.6.5 ใช้ Raycasting ในการพัฒนาระบบการมองเห็นของศัตรู และใช้การคำนวณระยะทางสำหรับระบบตรวจจับเสียง

1.3.6.6 ใช้ Navigation Mesh ร่วมกับอัลกอริทึม A* (A-Star Algorithm) ในการค้นหาเส้นทางของศัตรู

**1.4 เครื่องมือที่เกี่ยวข้องกับโครงงาน**

1.4.1 Unity Game Engine เครื่องมือหลักในการพัฒนา

1.4.2 ภาษา C# สำหรับเขียนสคริปต์ควบคุมการทำงาน

1.4.3 Visual Studio: IDE สำหรับเขียนโค้ดและ Debug

1.4.4 โปรแกรมสร้างกราฟิก 2D: สำหรับสร้าง Sprite ตัวละครและฉาก

## 1.5 ตารางการดำเนินโครงงาน

| การดำเนินโครงงานวิจัย | ระยะเวลาการดำเนินโครงงานวิจัย | | | | | | | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | พ.ย. | | ธ.ค. | | ม.ค. | | ก.พ. | | มี.ค. | | เม.ย. | | พ.ค. | | มิ.ย. | | ก.ค. | | ส.ค. | | ก.ย. | |
| 1. ศึกษาปัญหาและความต้องการ | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | |
| 2. กำหนดขอบเขต | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | |
| 3. ศึกษาการใช้งาน Unity | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | |
| 4. ออกแบบตัวละคร | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | |
| 5. ออกแบบระบบ | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | |
| 6. พัฒนาระบบ | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | |
| 7. ติดตั้งโปรแกรมและทดสอบ | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | |
| 8. ปรับปรุงและแก้ไขระบบ | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | |
| 10. ทำเอกสารงานวิจัย | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | |
| 11. พบอาจารย์ที่ปรึกษา | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | |

ระยะเวลาคาดหวัง

ระยะเวลาทำงานจริง

## 1.6 ประโยชน์ที่คาดว่าจะได้รับ

1.6.1 ได้รับความรู้ความเข้าใจเชิงลึกในการเขียนโปรแกรมจัดการสถานะ (State Management) ซึ่งเป็นพื้นฐานสำคัญของการพัฒนาซอฟต์แวร์

1.6.2 สามารถประยุกต์ใช้คณิตศาสตร์และอัลกอริทึม (Physics & Algorithms) ในการสร้างระบบการตรวจจับ (Detection System) ภายในเกมได้

1.6.3 ได้ต้นแบบซอฟต์แวร์เกมที่มีระบบการควบคุมตัวละคร (Character Controller) และ AI ที่ทำงานสัมพันธ์กันอย่างเป็นระบบ

1.6.4 มีทักษะในการออกแบบ UX/UI ที่สื่อสารข้อมูลเชิงเทคนิค (เช่น ระดับเสียง) ให้ผู้ใช้งานเข้าใจได้ง่าย

1.6.5 ทำให้ทราบระดับความพึงพอใจของผู้เล่น และสามารถนำผลที่ได้ไปใช้ในการปรับปรุงและพัฒนาระบบเกมให้มีประสิทธิภาพและตอบสนองต่อผู้ใช้งานได้ดียิ่งขึ้น

# บทที่ 2

# ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

ในการพัฒนาเกมคอมพิวเตอร์ 2 มิติ แนว Side-scrolling Hide-and-Seek Horror Game ด้วย C# ผู้จัดทำได้ศึกษาหลักการทฤษฎีและเทคโนโลยีที่ เกี่ยวข้อง เพื่อใช้เป็นแนวทางในการดำเนินงานวิจัย ประกอบไปด้วยหัวข้อหลัก ๆ ดังต่อไปนี้

2.1 แนวคิดเกี่ยวกับการพัฒนาเกม (Game Development)

2.2 การพัฒนาเกม 2 มิติ

2.3 เกม Side-scrolling game หรือ Side-view game

2.4 เกมประเภท Hide-and-Seek Horror Game

2.5 แนวคิดเกี่ยวกับ Finite State Machine (FSM)

2.6 การพัฒนาปัญญาประดิษฐ์ในเกม

2.7 ภาษาและเครื่องมือที่ใช้พัฒนา

2.8 อัลกอริทึมการค้นหาเส้นทาง (Pathfinding Algorithm)

2.9 การออกแบบ ux ui ให้เหมาะสม

2.10 งานวิจัยที่เกี่ยวข้อง

## 2.1 แนวคิดเกี่ยวกับการพัฒนาเกม (Game Development)

การพัฒนาเกม (Game Development) เป็นกระบวนการสร้างซอฟต์แวร์ เชิงโต้ตอบที่ผสานองค์ประกอบด้านการออกแบบ ศิลปะ และวิศวกรรมซอฟต์แวร์เข้าด้วย กัน โดยทั่วไปกระบวนการพัฒนาเกมประกอบด้วยขั้นตอนการวิเคราะห์ความต้องการ (Requirement Analysis) การออกแบบระบบ (System Design) การพัฒนา (Implementation) การทดสอบ (Testing) และการประเมินผล (Evaluation) กระบวนการสร้างเกม (Game Development) คือ กระบวนการในมุมมอง ของการสร้าง การออกแบบ โดยใช้ประกอบในมุมมองสองมิติ โดยการออกประกอบจาก เกม X (แนวนอน) และ แนวนอน Y (แนวตั้ง) ซึ่งแตกต่างจากเกม 3 มิติที่สามารถแสดง ตัวละครจากเกม ในการรวมกันสร้างสรรค์เกมในการออกแบบ เกม 2 มิติจะดำเนินการใน พื้นที่ หรือสิ่งต่างๆ ในการออกแบบ ทั้งนี้ เกม 2 มิติที่มีสามารถแสดงสถานการณ์สำหรับผู้ เล่นในเนื้อหาหรืออาจจะใช้ในการออกแบบแนวเกมสนุก ที่ตัวละครจะสร้างสรรค์กลยุทธ์ และสามารถสร้างความประทับใจให้ผู้เล่นได้ องค์ประกอบสำคัญของเกม ได้แก่ ระบบกลไกการเล่น (Game Mechanics) ระบบกราฟิกและแอนิเมชัน ระบบเสียง ระบบปัญญาประดิษฐ์ (AI System) และส่วน ติดต่อผู้ใช้ (User Interface) ในเกมที่มีศัตรูหรือ NPC ระบบ AI มีบทบาทสำคัญใน

การสร้างความท้าทายและความสมจริง โดยเฉพาะเกมแนวสยองขวัญ (Horror Game) ที่พฤติกรรมของศัตรูต้องตอบสนองต่อการกระทำของผู้เล่นอย่างมีเหตุผล

**2.2 การพัฒนาเกม 2 มิติ**

การพัฒนาเกม 2 มิติ (2D Game Development) เป็นการสร้างเกมที่ใช้ พิกัดสองมิติในการแสดงผล ประกอบด้วยแกน X (แนวนอน) และแกน Y (แนวตั้ง) ซึ่ง แตกต่างจากเกม 3 มิติที่มีความลึก (Depth) เพิ่มเติม เกม 2 มิติมีความได้เปรียบใน ด้านความง่ายต่อการพัฒนา สุนทรียศาสตร์ที่มีเอกลักษณ์ได้ ใช้ทรัพยากรการประมวลผลต่ำกว่า และสามารถสร้าง นักพัฒนาจะต้องออกแบบและควบคุมสภาพแวดล้อม โดยระบบดำเนินการเส้นทาง (Platformer) ร่วมลักษณะการทำงานในสามารถทำงานได้ดีในเกม 2 มิติ โดยแตก ต่างจากเกม 3 มิติในด้านความซับซ้อน ในการพัฒนาเกม 2 มิติ เครื่องมือที่นิยมใช้คือUnity Game Engine ซึ่งรองรับการพัฒนาเกม 2 มิติได้อย่างครบถ้วน ทั้งระบบ Sprite, Tilemaps, Physics 2D และ Animator สำหรับการสร้างแอนิเมชั่นตัว ละคร ส่วนสำคัญที่สำคัญในการพัฒนาเกม 2 มิติ คือการออกแบบระดับ (Level Design) ซึ่งเป็นการเลือกรูปแบบและทรัพยากรในการพัฒนาเกม รวมถึงการออกแบบ Tilemap เพื่อสร้างฉากที่มีความสวยงามและน่าสนใจ นอกจากนี้ระบบ Collision Detection ก็มีความสำคัญในการตรวจจับการชนระหว่างตัวละครกับสิ่งแวดล้อม ภายในเกม

**2.3 เกม Side-scrolling game หรือ Side-view game**

เกม Side-scrolling game หรือ Side-view game คือเกมที่มุมกล้อง ถูกกำหนดให้มองจากด้านข้างของตัวละคร ทำให้ผู้เล่นเห็นตัวละครในแนวตั้งและสามารถ เคลื่อนที่ไปทางซ้ายหรือขวาตามแกน X เป็นหลัก มุมมองแบบนี้เป็นที่นิยมอย่างกว้างขวางในเกมยุคแรก ๆ และยังคงได้รับความนิยมอย่างต่อเนื่องในเกมยุคปัจจุบัน เนื่องจาก สามารถสร้างประสบการณ์การเล่นที่ชัดเจนและเข้าใจง่าย คุณลักษณะสำคัญของเกม Side-scrolling ได้แก่ การเคลื่อนที่ของตัวละครใน แนวแกน X เป็นหลัก กล้องจะเลื่อนตาม (Follow Camera) ไปพร้อมกับตัวละคร การออกแบบฉากในแนวนอนที่มีความต่อเนื่อง รวมถึงการใช้ระบบชั้น (Layering) เพื่อสร้างความลึกในภาพแบบ 2 มิติ เช่น ฉากหลังเลื่อนช้ากว่าฉากหน้า (Parallax Scrolling) ซึ่งช่วยเพิ่มความสมจริงให้กับเกม ในการพัฒนาเกม Side-scrolling Horror Game นั้น มุมมองด้านข้างช่วย สร้างบรรยากาศความหวาดกลัวได้อย่างมีประสิทธิภาพ เนื่องจากผู้เล่นจะมองเห็นสิ่งที่อยู่ ข้างหน้าและข้างหลังได้อย่างชัดเจน แต่ไม่สามารถมองเห็นสิ่งที่อยู่ด้านบนหรือด้านล่างได้ ครบถ้วน ทำให้เกิดความตื่นเต้นและความไม่แน่นอนขณะเล่น

**2.4 เกมประเภท Hide-and-Seek Horror Game**

เกมประเภท Hide-and-Seek Horror Game คือเกมสยองขวัญที่มีแกน หลักของการเล่น (Core Gameplay) อยู่ที่การหลบซ่อนตัวจากศัตรูหรือผู้ไล่ล่า (Pursuer) ผู้เล่นจะต้องใช้ไหวพริบและความระมัดระวังในการหลบหลีกศัตรู รวบรวม ไอเทมที่กำหนด และหาทางออกจากด่านให้สำเร็จ โดยไม่ถูกศัตรูจับได้ เกมประเภทนี้มีชื่อเสียงเช่น Slender: The Eight Pages, Outlast, และ Amnesia: The Dark Descent องค์ประกอบสำคัญของ Hide-and-Seek Horror Game ได้แก่ ระบบ ตรวจจับของศัตรู (Enemy Detection System) ซึ่งประกอบด้วยการมองเห็น (Line of Sight) และการได้ยินเสียง (Sound Detection) ผู้เล่นจะต้องควบคุม ระดับเสียงที่เกิดจากการเคลื่อนไหว เช่น การเดินธรรมดาจะมีเสียงดังกว่าการนั่งย่อหรือหมอบคลาน รวมถึงการใช้สภาพแวดล้อม เช่น การซ่อนตัวในตู้หรือใต้โต๊ะ เพื่อหลบเลี่ยง การตรวจจับในเกม 2 มิติแนว Side-scrolling Horror Game การออกแบบระบบ Al ของศัตรูมีความสำคัญอย่างยิ่ง ศัตรูจะต้องมีพฤติกรรมที่สมเหตุสมผลและน่าเชื่อถือ โดย อาศัยเทคนิค Finite State Machine (FSM) ในการจัดการสถานะพฤติกรรมต่าง ๆ เช่น การลาดตระเวน การตรวจสอบเสียง และการไล่ล่า รวมถึงระบบ Raycasting สำหรับตรวจสอบการมองเห็น และอัลกอริทึม A* สำหรับการนำทางในฉาก

**2.5 แนวคิดเกี่ยวกับ Finite State Machine (FSM)**

Finite State Machine (FSM) หรือเครื่องสถานะจำกัด เป็นแบบจำลอง และการ การคำนวณที่ใช้กำหนดพฤติกรรมของระบบผ่านชุดของสถานะ (States) เปลี่ยนผ่านสถานะ (Transitions) ตามเงื่อนไขที่กำหนด โดยในช่วงเวลาใดเวลาหนึ่ง ระบบจะอยู่ได้เพียงสถานะเดียวเท่านั้น FSM เป็นที่นิยมอย่างกว้างขวางในการพัฒนาเกม เนื่องจากเข้าใจง่าย ใช้ทรัพยากรประมวลผลต่ำ และสามารถปรับแก้พฤติกรรมได้โดยไม่ กระทบโครงสร้างทั้งหมด

องค์ประกอบหลักของ FSM ได้แก่ สถานะเริ่มต้น (Initial State) สถานะการ ทำงาน (Operational States) เหตุการณ์หรือเงื่อนไข (Event/Condition) การเปลี่ยนสถานะ (Transition) และการกระทำภายในสถานะ (State Action) ใน โครงงานนี้ FSM ถูกนำมาใช้ควบคุมทั้งตัวละครผู้เล่นและ Al ของศัตรู สำหรับตัวละครผู้เล่นใน Hide-and-Seek Horror Game สถานะที่ใช้ใน FSM ได้แก่ การยืนเดิน (Walk) ความเร็วปกติและรัศมีเสียงกว้าง การนั่งย่อ(Crouch) ความเร็วลดลงและรัศมีเสียงปานกลาง และการหมอบคลาน (Prone) ความเร็วต่ำที่สุดและรัศมีเสียงเงียบที่สุด แต่ละสถานะส่งผลต่อขนาด Collider และระดับ Noise Radius ซึ่งเป็นปัจจัยหลักในการถูกตรวจจับโดย A สำหรับ Al ของศัตรู (Ghost/Killer) สถานะที่ใช้ใน FSM ได้แก่ Patrol State (ลาดตระเวนตาม Waypoints), Investigate State (ตรวจสอบจุดที่ ได้ยินเสียง), Chase State (ไล่ล่าเมื่อเห็นผู้เล่นใน Vision Cone) และ Attack State (โจมตีเมื่อระยะห่าง ≤ 2 pixel) นอกจากนี้ FSM ยังกล่าวถึง Hierarchical Finite State Machine (HFSM) ซึ่งเป็น

การจัดลำดับชั้นของสถานะเพื่อลดความ ซับซ้อนของ FSM แบบดั้งเดิม ทำให้สามารถบริหารจัดการพฤติกรรมที่ซับซ้อนได้ดียิ่งขึ้น

**2.6 การพัฒนาปัญญาประดิษฐ์ในเกม**

Game Al (Game Artificial Intelligence) หมายถึง ระบบที่ควบคุม พฤติกรรมของตัวละครในเกมให้สามารถโต้ตอบกับผู้เล่นได้อย่างสมเหตุสมผล เป้าหมาย หลักของ Game Al ไม่ใช่การเรียนรู้ชั้นสูงแบบ Machine Learning แต่เป็นการ สร้าง "ภาพลวงตาของความฉลาด" เพื่อเพิ่มความสนุกและความท้าทายในการเล่น เทคนิคที่นิยมใช้ใน Game Al ได้แก่ Rule-based System, Finite State Machine (FSM), Behavior Tree และ Pathfinding Algorithm ในเกม 2 มิติแนว Hide-and-Seek Horror Game ระบบ Al ของศัตรูจำเป็นต้อง ผสมผสานเทคนิคหลายอย่างเข้าด้วยกัน ดังนี้

2.6.1 Finite State Machine (FSM) สำหรับศัตรู ใช้ FSM ในการจัดการสถานะพฤติกรรมของศัตรู (Ghost/Killer Al) โดยแบ่ง ออกเป็นสถานะ Patrol, Investigate, Chase และ Attack โดยแต่ละสถานะจะเปลี่ยนไปตามเงื่อนไขที่ตรวจจับได้จากสภาพแวดล้อม เช่น ระยะสายตา และระดับเสียงของผู้เล่น

2.6.2 Sensory System (ระบบการรับรู้) พัฒนาระบบรับรู้ของ Al โดยใช้ Raycasting ในการจำลองสายตา (Line of Sight / Vision Cone) เพื่อตรวจสอบว่าผู้เล่นอยู่ในแนวสายตาของศัตรูหรือไม่ และใช้การคำนวณระยะทาง (Distance Check) เปรียบเทียบกับ Noise Radius ที่ผู้เล่นสร้างขึ้นตามท่าทางการเคลื่อนไหว

2.6.3 การค้นหาเส้นทาง ใช้ระบบ Navigation ร่วมกับ A* Algorithm ให้ Al สามารถเดินไล่ล่าผู้เล่น หลบสิ่งกีดขวาง และเคลื่อนที่ไปยังจุดที่ได้ยินเสียงล่าสุดได้อย่างสมจริงภายในฉาก เกม 2 มิติแนว Side-scrolling

**2.7 ภาษาและเครื่องมือที่ใช้พัฒนา**

2.7.1 Visual Studio Code

เป็นโปรแกรมแก้ไขโค้ด (Code Editor) ที่พัฒนาโดย Microsoft มีความ ยืดหยุ่นสูง รองรับภาษาโปรแกรมต่างๆ ด้วยส่วนขยาย (Extensions) ที่หลาก หลาย สามารถเชื่อมต่อกับภาษา C# และ Unity Game Engine ได้อย่าง สะดวก มีระบบ IntelliSense ช่วยแนะนำโค้ดและแก้ไขข้อผิดพลาดได้อย่างมี ประสิทธิภาพ

2.7.2 เอ็นจิ้นเกม Unity

เป็นแอปพลิเคชันสำหรับการพัฒนาเกม มีความยืดหยุ่นสูง รองรับการพัฒนาเกมทั้ง 2 มิติ และ 3 มิติ โดยสามารถส่งออกเกมไปยัง Windows, Android และ iOS ได้อย่างสะดวก Unity มีเครื่องมือสำคัญดังนี้

2.7.2.1 Scene Editor ใช้ในการออกแบบฉากและจัดวางวัตถุใน เกม รวมถึงการสร้าง Tilemap สำหรับเกม 2 มิติแนว Side-scrolling

2.7.2.2 Animator ใช้ควบคุมและแสดงผลแอนิเมชันของตัวละครใน สถานะต่าง ๆ เช่น Walk, Crouch, Prone

2.7.2.3 Physics Engine 2D จัดการระบบฟิสิกส์ การชน (Collision) และการเคลื่อนที่ของวัตถุในเกม 2 มิติ

2.7.3 ภาษาโปรแกรม C#

เป็นภาษาหลักที่ใช้ในการพัฒนาเกมใน Unity มีโครงสร้างเชิงวัตถุ (OOP) ที่ ชัดเจน รองรับการสร้างระบบที่ซับซ้อน เช่น ระบบ FSM ระบบ AI ระบบ Inventory และการจัดการสถานะตัวละคร โดยในโครงงานนี้ใช้ C# เขียน สคริปต์ควบคุมการทำงานทั้งหมด

2.7.3.1 Game Logic Scripting สำหรับเขียนตรรกะการทำงาน ของระบบต่าง ๆ เช่น การเปลี่ยนสถานะ FSM และการตรวจจับ

2.7.3.2 Event Handling ใช้จัดการเหตุการณ์ต่าง ๆ เช่น Input ของผู้เล่น การชน และการเปลี่ยนฉาก

2.7.3.3 Data Management ใช้จัดการข้อมูลไอเทม สถานะผู้เล่น และการส่งต่อข้อมูลข้ามฉาก (Data Persistence)

2.7.4 JSON (JavaScript Object Notation)

เป็นรูปแบบไฟล์ข้อมูลที่มีโครงสร้างแบบคู่ระหว่างชื่อ (Key) และค่า (Value) ซึ่งอ่านและแก้ไขได้ง่ายในรูปแบบ Human-Readable Format สามารถใช้ ร่วมกับภาษา C# ใน Unity เพื่อบันทึกและโหลดข้อมูลเกม เช่น ข้อมูลตัวละคร และความคืบหน้าในการเล่น

**2.8 อัลกอริทึมการค้นหาเส้นทาง (Pathfinding Algorithm)**

ระบบค้นหาเส้นทาง (Pathfinding) มีบทบาทสำคัญในการทำให้ Al ของศัตรู สามารถเคลื่อนที่ภายในฉากเกม 2 มิติ Side-scrolling ได้อย่างมีประสิทธิภาพและ หลบหลีกสิ่งกีดขวางต่าง ๆ ได้อย่างสมจริง A* Algorithm (A-Star) เป็นอัลกอริทึมที่นิยมใช้ในเกม เนื่องจากสามารถ คำนวณเส้นทางที่เหมาะสมที่สุดโดยพิจารณาทั้งค่าใช้จ่ายจริงจากจุดเริ่มต้น (g-cost) และค่าประมาณระยะทางไปยังเป้าหมาย (h-cost) โดย f(n) = g(n) + h(n) ทำให้ ได้เส้นทางที่สั้นที่สุดในเวลาอันรวดเร็ว การทำงานร่วมกันระหว่าง FSM และ A* ใน Hide-and-Seek Horror Game มีลักษณะดังนี้ เมื่อ FSM ของศัตรูเปลี่ยนเป็นสถานะ Investigate ระบบจะ เรียกใช้ A* เพื่อนำทางไปยังตำแหน่งที่ได้ยินเสียงล่าสุด และเมื่อเปลี่ยนเป็นสถานะ Chase ระบบจะคำนวณเส้นทางไปยังตำแหน่งปัจจุบันของผู้เล่นแบบเรียลไทม์ โครงสร้างนี้ช่วยให้ศัตรูมีพฤติกรรมการไล่ล่าที่สมจริงและน่าตื่นเต้นยิ่งขึ้น

## 2.9 การออกแบบ UX/UI ให้เหมาะสม

การออกแบบส่วนติดต่อผู้ใช้ (User Interface: UI) และประสบการณ์ผู้ใช้ (User Experience: UX) เป็นองค์ประกอบสำคัญในการพัฒนาเกมที่ช่วยให้ผู้เล่น สามารถโต้ตอบกับระบบได้อย่างสะดวกและเข้าใจง่าย ในเกม 2 มิติ Side-scrolling Hide-and-Seek Horror Game การออกแบบ UI ที่ดีจะต้องไม่บดบังทัศนวิสัย ของผู้เล่น แต่ยังคงสื่อสารข้อมูลสำคัญได้อย่างชัดเจน องค์ประกอบ UI ที่สำคัญในเกมประเภทนี้ได้แก่ ตัวบ่งชี้ท่าทาง (Stance Indicator) ที่แสดงสถานะปัจจุบันของผู้เล่นว่ากำลังยืน นั่งย่อ หรือหมอบคลาน มิเตอร์ เสียง (Noise Meter) ที่แสดงระดับความดังของเสียงที่ผู้เล่นสร้างขึ้นแบบเรียลไทม์ พลังชีวิต (HP Bar) และตัวนับไอเทม (Item Counter) เพื่อให้ผู้เล่นทราบความคืบ หน้าของภารกิจ หลักการออกแบบ UX ที่ดีในเกม Horror คือการใช้สีและเสียงเพื่อสร้าง บรรยากาศความกลัว โดยไม่ทำให้ UI ดูไม่สอดคล้องกับโทนของเกม นอกจากนี้ควร ออกแบบ HUD (Head-Up Display) ที่กระชับ สื่อความหมายได้ชัดเจน และตอบ สนองต่อสถานการณ์ในเกมได้อย่างทันท่วงที และน่าตื่นเต้น เพื่อสร้างประสบการณ์การเล่นที่น่าจดจำและน่าตื่นเต้น

## 2.10 งานวิจัยที่เกี่ยวข้อง

### 2.10.1 Finite State Machine in Game Development

งานวิจัยนี้ศึกษาแนวคิดและหลักการทำงานของ Finite State Machine (FSM) ในการพัฒนาระบบปัญญาประดิษฐ์ (AI) สำหรับควบคุมพฤติกรรมของตัวละครที่ไม่ใช่ผู้เล่น (Non-Player Characters: NPC) ภายในเกม โดยอธิบายว่า FSM เป็นแบบจำลองที่ประกอบด้วย “สถานะ (State)” และ “การเปลี่ยนสถานะ (Transition)” ซึ่งจะเปลี่ยนไปตามเงื่อนไขที่กำหนดไว้ล่วงหน้า การประยุกต์ใช้ FSM ในเกมช่วยให้สามารถกำหนดพฤติกรรมของตัวละคร เช่น การเดิน การไล่ล่า การโจมตี การป้องกัน หรือการหลบหนี ได้อย่างเป็นระบบ ลดความซับซ้อนของโค้ด และเพิ่มความชัดเจนในการออกแบบระบบ นอกจากนี้ งานวิจัยยังกล่าวถึง Hierarchical Finite State Machine (HFSM) ซึ่งเป็นการจัดลำดับชั้นของสถานะเพื่อลดความซับซ้อนของ FSM แบบดั้งเดิม ทำให้สามารถบริหารจัดการพฤติกรรมที่ซับซ้อนได้ดียิ่งขึ้น ผลการศึกษาสรุปว่า FSM เป็นเทคนิคที่มีประสิทธิภาพ เข้าใจง่าย และยังคงได้รับความนิยมในการพัฒนาเกมสมัยใหม่

### 2.10.2 Development of Two Dimension (2D) Game Engine with Finite State Machine (FSM) Based Artificial Intelligence (AI) Subsystem

งานวิจัยนี้มุ่งพัฒนาเกมเอนจินแบบ 2 มิติ ที่มีระบบ AI ภายในโดยใช้แนวคิด FSM เป็นแกนหลักของการควบคุมพฤติกรรมตัวละคร พัฒนาด้วยเทคโนโลยีเว็บ เช่น JavaScript และ WebGL 2.0 มีการเปรียบเทียบประสิทธิภาพระหว่างการพัฒนา AI แบบทั่วไป (Naive Approach) กับการใช้ FSM เป็นโครงสร้างหลัก พบว่า การใช้ FSM สามารถรักษาค่าเฟรมเรต (Frames Per Second: FPS) ได้ที่

ประมาณ 60 FPS ขณะที่วิธีทั่วไปทำให้เฟรมเรต ลดลงเหลือประมาณ 50 FPS แสดงให้เห็นว่า FSM ช่วยเพิ่มประสิทธิภาพของระบบ ลดความกำกวมของตรรกะการทำงาน และช่วยให้การจัดการพฤติกรรมของตัวละครเป็นระบบมากขึ้น

**2.10.3 An Analysis of Finite State Machine Based Enemy Artificial Intelligence in Kirby: Nightmare in Dream Land**

บทความนี้ศึกษาการประยุกต์ใช้เทคนิค **Finite State Machine (FSM)** ในการออกแบบปัญญาประดิษฐ์ของศัตรูในเกม Kirby: Nightmare in Dream Land โดยมุ่งวิเคราะห์ว่าโครงสร้างแบบกำหนดสถานะสามารถควบคุมพฤติกรรมศัตรูและส่งผลต่อระดับความยากของเกมได้อย่างไร ผู้วิจัยอธิบายว่า FSM เป็นโมเดลที่แบ่งพฤติกรรมออกเป็นสถานะต่าง ๆ เช่น หยุดนิ่ง (Idle) เดินลาดตระเวน (Patrol) โจมตี (Attack) ถูกโจมตี (Hit) และฟื้นตัว (Recover) และกำหนดเงื่อนไขการเปลี่ยนสถานะอย่างชัดเจนตามเหตุการณ์ที่เกิดขึ้นในเกม จากการทดลองสังเกตพฤติกรรมศัตรูหลายครั้ง พบว่าสถานะการโจมตีมีสัดส่วนเวลามากที่สุด ขณะที่สถานะหยุดนิ่งและเดินช่วยสร้างจังหวะพักให้ผู้เล่น ทำให้เกมมีความสมดุล ไม่ยากหรือไม่ง่ายเกินไป นอกจากนี้ ผู้วิจัยยังทดลองปรับค่าระยะเวลาหน่วงก่อนการโจมตี (Attack Cooldown) และพบว่ายิ่งศัตรูโจมตีถี่ขึ้น ระดับความยากของเกมก็เพิ่มสูงขึ้นอย่างชัดเจน แสดงให้เห็นว่า FSM สามารถปรับความท้าทายของเกมได้โดยการแก้ไขพารามิเตอร์เพียงบางค่า โดยไม่ต้องปรับโครงสร้าง AI ทั้งระบบ โดยสรุป งานวิจัยชี้ให้เห็นว่า FSM เป็นเทคนิคที่มีความเสถียร ใช้ทรัพยากรประมวลผลต่ำ ควบคุมและปรับแต่งได้ง่าย และยังคงมีความเหมาะสมสำหรับการพัฒนา AI ในเกมแพลตฟอร์ม 2 มิติ แม้จะมีเทคนิค AI ที่ซับซ้อนกว่าพัฒนาขึ้นในปัจจุบันก็ตาม

**2.10.4 การออกแบบเกมดิจิทัล**

งานวิจัยนี้นำเสนอแนวคิดและกระบวนการออกแบบเกมดิจิทัล โดยกล่าวถึงหลักการออกแบบจากหลากหลายแนวทาง เช่น การออกแบบจากนวัตกรรม ความคิดสร้างสรรค์ เรื่องราว ความต้องการของผู้เล่น และข้อจำกัดด้านเทคโนโลยี นอกจากนี้ยังกล่าวถึงประเภทของเกม เช่น เกม 2 มิติ และ 3 มิติ รวมถึงการแบ่งตามลักษณะการเล่น เช่น เกมต่อสู้ เกมยิง เกมผจญภัย และเกมการศึกษา พร้อมทั้งเสนอขั้นตอนการพัฒนาเกมอย่างเป็นระบบ ได้แก่ การวางแผน การจัดทำเอกสารออกแบบเกม (Game Design Document: GDD) การพัฒนาโปรแกรม การสร้างวัตถุในเกม การทดสอบ และการเผยแพร่ ซึ่งเป็นแนวทางสำคัญในการพัฒนาเกมให้ประสบความสำเร็จ

**2.10.5 การพัฒนาเกมเพื่อการเรียนรู้การเขียนโปรแกรมภาษา C# ด้วยเกมการผจญภัยของอาเรียกับเวทย์มนต์แห่ง C#**

งานวิจัยนี้มีวัตถุประสงค์เพื่อพัฒนาเกมสำหรับเสริมการเรียนรู้การเขียนโปรแกรมภาษา C# โดยใช้โปรแกรม Unity ในการพัฒนาเกมแนวผจญภัย (Adventure Game) และใช้แนวคิด Game-Based Learning เพื่อให้ผู้เรียนเรียนรู้ผ่านการเล่นเกม ผลการประเมินโดยผู้เชี่ยวชาญจำนวน 5 คน

พบว่าคุณภาพของเกมอยู่ในระดับดีมาก (ค่าเฉลี่ย 4.71) และเมื่อนำไปทดลองกับกลุ่มตัวอย่างจำนวน 20 คน พบว่าคะแนนหลังเรียนเพิ่มขึ้นร้อยละ 40.92 แสดงให้เห็นว่าเกมสามารถช่วยเพิ่มความเข้าใจในการเขียนโปรแกรม และส่งเสริมการคิดอย่างเป็นระบบได้อย่างมีประสิทธิภาพ

**2.10.6 A Systematic Review and Analysis of Intelligence-Based Pathfinding Algorithms in the Field of Video Games**

งานวิจัยนี้นำเสนอการทบทวนและวิเคราะห์อัลกอริทึมการค้นหาเส้นทาง (Pathfinding Algorithm) ที่ใช้ปัญญาประดิษฐ์ในเกมวิดีโอ โดยจำแนกอัลกอริทึมออกเป็น 3 กลุ่มหลัก ได้แก่ กลุ่ม Uninformed Search เช่น BFS และ Dijkstra's กลุ่ม Informed Search เช่น A* และ Greedy Best First และกลุ่ม Metaheuristic เช่น Genetic Algorithm และ Ant Colony Optimization ผู้วิจัยได้ทดสอบอัลกอริทึมบน Grid 2 มิติขนาด 4×4, 8×8, 16×16 และ 32×32 โดยใช้ Unity Game Engine เป็นสภาพแวดล้อมในการทดสอบ วัดผลด้วยเวลาประมวลผล (Execution Time) จำนวนบล็อกที่ผ่าน และความยาวเส้นทางที่สั้นที่สุด ผลการทดลองพบว่า A* Algorithm มีความสมดุลระหว่างความเร็วและความแม่นยำสูงที่สุด โดยใช้เวลาประมวลผลน้อยกว่า Dijkstra's และ BFS ในทุกขนาด Grid ทั้งที่มีและไม่มีสิ่งกีดขวาง และยังรับประกันเส้นทางที่สั้นที่สุด ซึ่งต่างจาก Greedy Best First ที่แม้จะเร็วที่สุดแต่ไม่ให้เส้นทางที่เหมาะสมที่สุดเสมอไป งานวิจัยนี้ยืนยันว่า A* เป็นอัลกอริทึมที่เหมาะสมที่สุดสำหรับการนำทาง NPC ในเกม 2 มิติ ซึ่งสอดคล้องกับการออกแบบระบบ AI ของศัตรูในโครงงานนี้

**2.10.7 Improving Mobile Game Performance with Basic Optimization Techniques in Unity**

งานวิจัยนี้ศึกษาผลกระทบของเทคนิค Optimization พื้นฐานที่มีต่อประสิทธิภาพของเกมบนอุปกรณ์มือถือ Android โดยพัฒนาเกม Shooter 3 มิติด้วย Unity และ Blender จำนวน 2 เวอร์ชัน ได้แก่ เวอร์ชันที่ใช้เทคนิค Optimization และเวอร์ชันที่ไม่ใช้ โดยเทคนิคที่นำมาทดสอบประกอบด้วย Low Poly Model, Merge Vertices, Textures and Materials, Occlusion Culling และ Object Pooling ตัวชี้วัดประสิทธิภาพที่ใช้คือค่า Frames Per Second (FPS), Batches และจำนวน Triangles/Polygons ซึ่งสะท้อนการใช้งาน CPU, GPU และหน่วยความจำตามลำดับ ผลการทดสอบบนอุปกรณ์มือถือจริงพบว่าเวอร์ชันที่ใช้เทคนิค Optimization รักษาค่า FPS ได้ในช่วง 50–60 FPS ตลอดการเล่น ขณะที่เวอร์ชันที่ไม่ผ่านการ Optimize มีค่า FPS ตกลงเหลือเพียง 20–30 FPS เมื่อมีศัตรูจำนวนมากบนหน้าจอ ทำให้เกมเกิดอาการกระตุก งานวิจัยนี้ยืนยันว่าการเลือกใช้เทคนิค Optimization ที่เหมาะสมมีผลโดยตรงต่อความลื่นไหลของเกม ซึ่งสอดคล้องกับโครงงานนี้ที่ต้องการให้เกม Side-scrolling Horror ทำงานได้อย่างมีประสิทธิภาพบน Unity

# บทที่ 3
# วิธีการดำเนินงาน

การพัฒนาเกมคอมพิวเตอร์ 2 มิติแนว Side-scrolling Hide-and-Seek Horror Game ในโครงงานนี้ดำเนินการด้วย Unity Game Engine และภาษา C# โดยประยุกต์ใช้เทคนิค Finite State Machine หรือ FSM ในการควบคุมสถานะของตัวละครผู้เล่นและพฤติกรรมของศัตรู ระบบถูกแบ่งเป็นโมดูลเพื่อให้ส่วนควบคุมผู้เล่น ระบบศัตรู ระบบไอเท็ม ระบบฉาก ระบบเสียง และส่วนติดต่อผู้ใช้งานทำงานร่วมกันได้อย่างเป็นระบบและสามารถปรับปรุงต่อได้

เกมประกอบด้วย 3 ด่านซึ่งมีระดับความยากและค่าพารามิเตอร์แตกต่างกัน ผู้เล่นต้องสำรวจพื้นที่ เก็บกุญแจให้ครบ หลีกเลี่ยงศัตรู และเดินทางไปยังประตูทางออก โดยสามารถใช้ไฟฉาย จุดซ่อนตัว และก้อนหินสร้างเสียงล่อศัตรูเพื่อช่วยในการหลบหนี ขั้นตอนการดำเนินงานประกอบด้วยการกำหนดปัญหา การรวบรวมข้อมูล การวิเคราะห์ระบบ การออกแบบระบบ การพัฒนา การทดสอบ และการบำรุงรักษา

## 3.1 การกำหนดปัญหา

### 3.1.1 ปัญหาด้านการควบคุมตัวละครผู้เล่น

ตัวละครผู้เล่นต้องรองรับการเดิน การวิ่ง การกระโดด การไต่บันได การซ่อนตัว การใช้ไอเท็ม การได้รับความเสียหาย และการเปลี่ยนฉาก การทำงานเหล่านี้อาจเกิดขึ้นต่อเนื่องหรือขัดแย้งกัน หากไม่มีระบบควบคุมสถานะที่ชัดเจนอาจทำให้ตัวละครรับคำสั่งในช่วงที่ไม่ควรควบคุมได้ โครงงานจึงใช้ FSM กำหนดสถานะ Normal, Hidden, Frozen, Hurt, UsingItem, Transitioning และ Dead เพื่อควบคุมสิทธิ์การรับคำสั่งและพฤติกรรมในแต่ละช่วง

### 3.1.2 ปัญหาด้านพฤติกรรมของศัตรู

ศัตรูต้องตอบสนองต่อผู้เล่นได้หลายลักษณะ ทั้งการลาดตระเวน การหันหาเสียง การตรวจสอบพื้นที่ การค้นหา การไล่ล่า และการโจมตี หากไม่มีลำดับสถานะที่ชัดเจน ศัตรูอาจเปลี่ยนพฤติกรรมอย่างไม่สมเหตุสมผลหรือไม่สามารถกลับไปลาดตระเวนได้หลังจากสูญเสียเป้าหมาย จึงใช้ Enemy FSM ควบคุมลำดับการตัดสินใจจากข้อมูลการมองเห็น การได้ยิน ระดับความสงสัย และตำแหน่งล่าสุดของผู้เล่น

### 3.1.3 ปัญหาด้านระบบตรวจจับ

ระบบตรวจจับต้องพิจารณาระยะทาง มุมมอง สิ่งกีดขวาง การเปิดไฟฉาย และเสียงที่เกิดจากกิจกรรมต่าง ๆ ของผู้เล่น การวิ่งมีความสำคัญมากกว่าการเดิน ขณะที่ก้อนหินและกับดักกระจกสร้างเสียง

จากตำแหน่งอื่นเพื่อเบี่ยงเบนความสนใจของศัตรู ระบบจึงจำแนกประเภทเสียงเป็น Footstep, Sprint, Rock, Glass และ Environmental ก่อนประเมินความดัง ระยะห่าง และอายุของเสียง

### 3.1.4 ปัญหาด้านตรรกะและเป้าหมายของเกม

ผู้เล่นต้องเก็บกุญแจตามจำนวนที่กำหนดก่อนเปิดประตูทางออก ระบบต้องตรวจสอบจำนวนกุญแจ สถานะประตู ระยะการโต้ตอบ และอันตรายรอบตัวผู้เล่น การปลดล็อกต้องกดปุ่มโต้ตอบค้างไว้และยกเลิกได้เมื่อผู้เล่นปล่อยปุ่ม เดินออกจากระยะ ถูกโจมตี หรือมีศัตรูอยู่ใกล้ เพื่อให้เป้าหมายของเกมชัดเจนและมีความท้าทาย

### 3.1.5 ปัญหาด้านส่วนติดต่อผู้ใช้งาน

ผู้เล่นต้องรับรู้พลังชีวิต แบตเตอรี่ไฟฉาย พลังงานสำหรับวิ่ง จำนวนก้อนหิน จำนวนกุญแจ ข้อความโต้ตอบ และสถานการณ์อันตรายแบบเรียลไทม์ โดยข้อมูลต้องไม่บดบังพื้นที่เล่น ระบบจึงแสดง HIDDEN เมื่อซ่อนตัว แสดง DANGER เมื่อถูกไล่ล่า และใช้เสียงหัวใจร่วมกับขอบจอสีแดงเมื่อศัตรูอยู่ใกล้

### 3.1.6 ปัญหาด้านการออกแบบและปรับสมดุลด่าน

แต่ละด่านต้องมีบรรยากาศและระดับความยากต่างกัน ค่าเกี่ยวกับผู้เล่น ศัตรู และไฟฉายจึงต้องปรับได้โดยไม่แก้โค้ดหลัก โครงงานใช้ LevelConfig แยกค่าของแต่ละด่าน และออกแบบ Level 3 เป็นป่ามืดที่มีหมอก ต้นไม้หลายชั้น ทางเดินต่างระดับ กับดักเสียง และตำแหน่งกุญแจที่เปลี่ยนแปลงได้

## 3.2 การรวบรวมข้อมูล

### 3.2.1 ข้อมูลด้านการพัฒนาเกม

ผู้พัฒนาได้ศึกษาการพัฒนาเกม 2 มิติด้วย Unity ระบบ Sprite และ Tilemap ระบบ Physics 2D การรับข้อมูลด้วย Unity Input System การจัดการฉาก การสร้าง UI และการใช้ ScriptableObject

สำหรับจัดเก็บค่าตั้งต้นของด่าน นอกจากนี้ยังศึกษาการเขียนโปรแกรมเชิงวัตถุเพื่อแยกความรับผิดชอบของแต่ละระบบให้สามารถพัฒนาและทดสอบได้ง่าย

### 3.2.2 ข้อมูลด้าน Finite State Machine

ศึกษาหลักการ FSM ซึ่งแบ่งการทำงานออกเป็นสถานะและกำหนดเงื่อนไขการเปลี่ยนสถานะอย่างชัดเจน หลักการดังกล่าวถูกนำมาใช้กับตัวละครผู้เล่น ศัตรู ประตูทางออก คลังไอเท็ม และระบบความตึงเครียด เพื่อป้องกันการทำงานซ้อนกันและช่วยให้ตรวจสอบพฤติกรรมได้เป็นลำดับ

### 3.2.3 ข้อมูลด้านเกมแนวซ่อนตัวและสยองขวัญ

ศึกษากลไกการมองเห็นและการได้ยินของศัตรู การใช้เสียงเป็นความเสี่ยงและเครื่องมือล่อ การออกแบบจุดซ่อน การจำกัดทรัพยากรไฟฉาย และการใช้เสียงดนตรี เสียงหัวใจ หมอก แสง และฉากหลังหลายชั้นเพื่อสร้างความกดดันแก่ผู้เล่น

## 3.3 การวิเคราะห์ระบบ

### 3.3.1 การวิเคราะห์ระบบควบคุมผู้เล่น

PlayerController รับคำสั่งจาก Unity Input System และควบคุมการเคลื่อนที่แนวนอน การวิ่ง การใช้ Stamina การกระโดด การไต่บันได การโต้ตอบ การเปิดไฟฉาย การซ่อนตัว และการหยุดเกม การวิ่งใช้พลังงานซึ่งลดลงระหว่างใช้งานและฟื้นกลับหลังจากหยุดพัก ส่วนระบบกระโดดมีการตรวจสอบพื้นและ Coyote Time เพื่อช่วยให้การควบคุมตอบสนองเป็นธรรมชาติ

| คำสั่ง | แป้นพิมพ์ | ผลการทำงาน |
|---|---|---|
| เคลื่อนที่ | W A S D หรือปุ่มลูกศร | เดินและกำหนดทิศทาง |
| วิ่ง | Left Shift | เพิ่มความเร็ว ใช้ Stamina และสร้างเสียงมากขึ้น |
| กระโดด | Space | กระโดดเมื่ออยู่บนพื้นหรือในช่วง Coyote Time |
| โต้ตอบ | E | เก็บไอเท็ม ใช้วัตถุ หรือกดค้างเพื่อปลดล็อกประตู |
| ไฟฉาย | F | เปิดหรือปิดไฟฉายและใช้พลังงานแบตเตอรี่ |
| ซ่อนตัว | H | เข้าและออกจากจุดซ่อน |
| คลังไอเท็ม | C | เปิดหรือปิดหน้าต่างคลังไอเท็ม |
| ก้อนหิน | Q | ขว้างก้อนหินเพื่อสร้างเสียงล่อศัตรู |
| หยุดเกม | Escape | เปิดหรือปิด Pause Menu |

### 3.3.2 การวิเคราะห์ Player FSM

| สถานะ | หน้าที่ |
|---|---|
| Normal | สถานะปกติที่รับคำสั่งเดิน วิ่ง กระโดด และโต้ตอบ |
| Hidden | ผู้เล่นอยู่ในจุดซ่อนและถูกจำกัดการเคลื่อนไหว |
| Frozen | หยุดการเคลื่อนไหวชั่วคราวและต้องทำตามเงื่อนไขเพื่อหลุดออก |
| Hurt | สถานะตอบสนองเมื่อได้รับความเสียหาย |
| UsingItem | ล็อกคำสั่งบางส่วนระหว่างเปิดคลังหรือใช้ไอเท็ม |
| Transitioning | ล็อกการควบคุมระหว่างการเปลี่ยนด่าน |
| Dead | หยุดการควบคุมเมื่อพลังชีวิตหมด |

### 3.3.3 การวิเคราะห์ระบบปัญญาประดิษฐ์ของศัตรู

KillerAI ประมวลผลแนวสายตาด้วย Raycasting และประมวลผลเสียงจากคิวเหตุการณ์ โดยให้ค่าน้ำหนักตามประเภทเสียง ความดัง ระยะห่าง และเวลาที่เสียงเกิดขึ้น ศัตรูบันทึกตำแหน่งล่าสุดของผู้เล่นหรือเสียงไว้ชั่วคราว เพื่อนำไปใช้ในสถานะตรวจสอบและค้นหา

การเปิดไฟฉายเพิ่มระยะที่ศัตรูสามารถตรวจพบผู้เล่นด้วยสายตา ส่วนผู้เล่นที่อยู่ในจุดซ่อนจะไม่ถูกตรวจจับตามเงื่อนไขปกติ

| สถานะ | พฤติกรรม |
|---|---|
| Patrol | เดินลาดตระเวนตามจุดที่กำหนด |
| Alert | หันไปยังทิศทางของเหตุการณ์ผิดปกติ |
| Listening | หยุดเพื่อพิจารณาทิศทางและความสำคัญของเสียง |
| InvestigateSound | เคลื่อนที่ไปยังต้นกำเนิดเสียง |
| ConfirmSound | ตรวจสอบบริเวณต้นกำเนิดเสียง |
| Confused | ลังเลหลังพบการใช้ก้อนหินล่อซ้ำบริเวณเดิม |
| Investigate | ไปยังตำแหน่งล่าสุดของผู้เล่น |
| Search | ค้นหาบริเวณรอบตำแหน่งล่าสุด |
| Chase | ไล่ล่าผู้เล่นที่ถูกตรวจพบ |
| Attack | โจมตีผู้เล่นเมื่ออยู่ในระยะ |
| AttackRecover | หยุดชั่วคราวหลังการโจมตี |
| ReturnToPatrol | กลับไปยังเส้นทางลาดตระเวน |
| Scream | กรีดร้องเพื่อทำให้ผู้เล่น Stun หรือ Frozen ตามการตั้งค่า |

### 3.3.4 การวิเคราะห์ระบบไอเท็มและภารกิจ

InventoryManager ใช้โครงสร้าง List เก็บไอเท็มที่ผู้เล่นรวบรวมและแยกกุญแจออกจากไอเท็มทั่วไป ระบบส่งเหตุการณ์ไปยัง UI เมื่อจำนวนกุญแจเปลี่ยนแปลง และตรวจสอบว่าผู้เล่นมีกุญแจครบตามค่า RequiredKeyCount หรือไม่ คลังไอเท็มเปิดด้วยปุ่ม C แสดงไอเท็มตามช่องที่กำหนด และอนุญาตให้ใช้ไอเท็มที่รองรับการใช้งาน

### 3.3.5 การวิเคราะห์ข้อมูลระดับความยาก

ค่าของด่านถูกจัดเก็บใน LevelConfig ซึ่งเป็น ScriptableObject ทำให้ปรับสมดุลเกมได้โดยไม่เปลี่ยนตรรกะหลัก ตารางต่อไปแสดงค่าหลักที่ใช้ในเกมเวอร์ชันปัจจุบัน

| พารามิเตอร์ | Level | Level 2 | Level 3 |
|---|---|---|---|
| ระดับความยาก | Easy | Normal | Nightmare |
| กุญแจที่ต้องใช้ | 5 | 5 | 5 |
| พลังชีวิตผู้เล่น | 100 | 100 | 100 |
| ความเร็วเดิน | 4.5 | 4.5 | 4.5 |
| ความเร็ววิ่ง | 5.7 | 5.7 | 5.7 |
| แบตเตอรี่ไฟฉาย | 80 | 60 | 45 |
| ความเร็วลาดตระเวน | 2.0 | 1.3 | 2.8 |
| ความเร็วไล่ล่า | 4.0 | 3.0 | 7.2 |
| ระยะมองเห็น | 6.0 | 4.55 | 0 |
| ระยะได้ยิน | 4.0 | 3.25 | 10.0 |
| พลังโจมตี | 25 | 50 | 35 |

Level 3 ใช้ชื่อ Echoes of the Blind และกำหนดระยะมองเห็นของศัตรูเป็นศูนย์ แต่เพิ่มระยะได้ยินเป็น 10 หน่วย จึงเน้นการหลบเลี่ยงด้วยการควบคุมเสียงมากกว่าการหลบแนวสายตา

## 3.4 การออกแบบระบบ

### 3.4.1 การออกแบบข้อมูลนำเข้า

ข้อมูลนำเข้าประกอบด้วยคำสั่งจากแป้นพิมพ์ ตำแหน่งและทิศทางของผู้เล่น สถานะของ Player FSM ระยะห่างจากศัตรู สถานะไฟฉาย เหตุการณ์เสียง จำนวนกุญแจ ไอเท็มที่เก็บได้ และข้อมูล LevelConfig ของด่านปัจจุบัน ข้อมูลเหล่านี้ถูกส่งไปยังโมดูลที่รับผิดชอบเพื่อประมวลผลตามเงื่อนไข

### 3.4.2 การออกแบบกระบวนการทำงาน

1) GameManager โหลด LevelConfig และฉากของด่านปัจจุบัน
2) PlayerController รับคำสั่งและตรวจสอบ PlayerState ก่อนดำเนินการ
3) ระบบคำนวณการเคลื่อนที่ Stamina การกระโดด การไต่บันได และการโต้ตอบ
4) FootstepSystem หรือวัตถุในฉากส่งเหตุการณ์เสียงไปยัง KillerAI

5) KillerAI ประเมินการมองเห็น เสียง และระดับความสงสัยก่อนเปลี่ยนสถานะ

6) InventoryManager ตรวจสอบไอเท็มและจำนวนกุญแจ

7) ExitDoor ตรวจสอบเงื่อนไขและความคืบหน้าการปลดล็อก

8) UIManager และระบบเสียงอัปเดตผลลัพธ์ให้ผู้เล่นรับรู้

9) GameManager เปลี่ยนด่านหรือแสดงผลการจบเกมเมื่อเงื่อนไขสำเร็จ

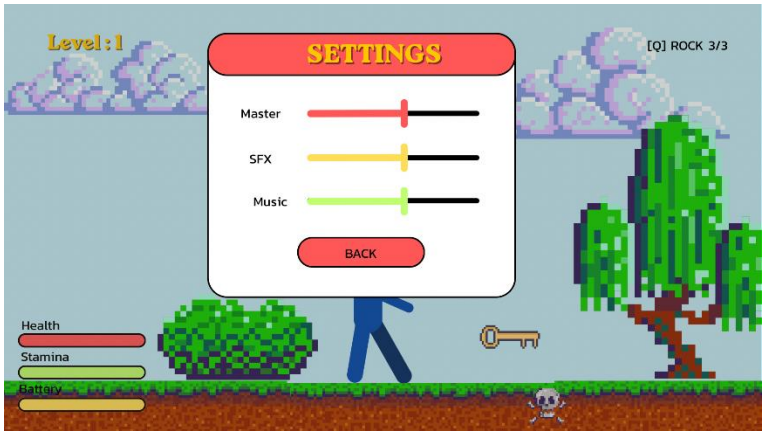
**3.4.3 การออกแบบเอาต์พุต**

หน้าจอเมนูหลักประกอบด้วยปุ่ม Start สำหรับเริ่มเกม Options สำหรับเปิดหน้าตั้งค่า และ Quit สำหรับออกจากเกม โดยใช้โทนสีเข้มให้สอดคล้องกับบรรยากาศสยองขวัญ




ภาพที่ 3-1 หน้าจอเมนูหลัก

หน้าจอตั้งค่าประกอบด้วย Slider สำหรับ Master Volume, SFX Volume และ Music Volume โดยค่าที่ปรับถูกเก็บด้วย PlayerPrefs เพื่อเรียกใช้ในการเปิดเกมครั้งต่อไป




ภาพที่ 3-2 หน้าจอตั้งค่าเสียง

HUD แสดงชื่อด่าน ระดับความยาก จำนวนกุญแจ พลังชีวิต แบตเตอรี่ไฟฉาย Stamina จำนวนก้อนหิน และข้อความโต้ตอบ รวมทั้ง HIDDEN และ DANGER ตามสถานการณ์




ภาพที่ 3-3 หน้าจอระหว่างเล่น (ภาพสาธิตใน Play Mode)

เมื่อกด Left Shift ระหว่างเคลื่อนที่ ผู้เล่นจะวิ่งเร็วขึ้นและ Stamina ลดลง แถบจะเปลี่ยนสีเมื่อเหลือพลังงานต่ำและฟื้นกลับหลังหยุดใช้




ภาพที่ 3-4 การวิ่งและระบบ Stamina (ภาพสาธิตใน Play Mode)

เมื่อเข้าใกล้วัตถุที่โต้ตอบได้ ระบบแสดง Prompt เพื่อแจ้งคำสั่ง ผู้เล่นกด E เพื่อเก็บไอเท็มหรือใช้งานวัตถุ




ภาพที่ 3-5 สาธิตตำแหน่งกุญแจและระบบเก็บไอเท็ม (ภาพสาธิตใน Play Mode)

ผู้เล่นกด *C* เพื่อเปิดและปิดคลังไอเท็มหน้าต่างแสดงช่องไอเท็มที่เก็บได้และรองรับการเลือกใช้ไอเท็มบางประเภท




ภาพที่ 3-6 หน้าจอคลังไอเท็ม (ภาพสาธิตใน Play Mode)

เมื่อผู้เล่นอยู่ใกล้จุดซ่อนและกด H ระบบเปลี่ยนสถานะเป็น Hidden พร้อมแสดง HIDDEN บนหน้าจอ




ภาพที่ 3-7 ระบบซ่อนตัว (ภาพสาธิตใน Play Mode)

ผู้เล่นกด F เพื่อเปิดหรือปิดไฟฉาย แบตเตอรี่ลดลงระหว่างใช้งาน และการเปิดไฟฉายเพิ่มระยะตรวจพบด้วยสายตาของศัตรู




ภาพที่ 3-8 ระบบไฟฉายและแบตเตอรี่ (ภาพสาธิตใน Play Mode)

ผู้เล่นขว้างก้อนหินเพื่อสร้างเสียงจากตำแหน่งอื่น ศัตรูรับเสียงประเภท Rock และเดินไปตรวจสอบ หากใช้ซ้ำบริเวณเดิมหลายครั้งศัตรูจะลดความเชื่อถือของเสียงล่อ




ภาพที่ 3-9 สาธิต Killer รับเหตุการณ์เสียงประเภท Rock (ภาพสาธิตใน Play Mode)

เมื่อศัตรูตรวจพบผู้เล่น ระบบเปลี่ยนเข้าสู่ Chase HUD แสดง DANGER และระบบเสียงปรับเข้าสู่บรรยากาศการไล่ล่า



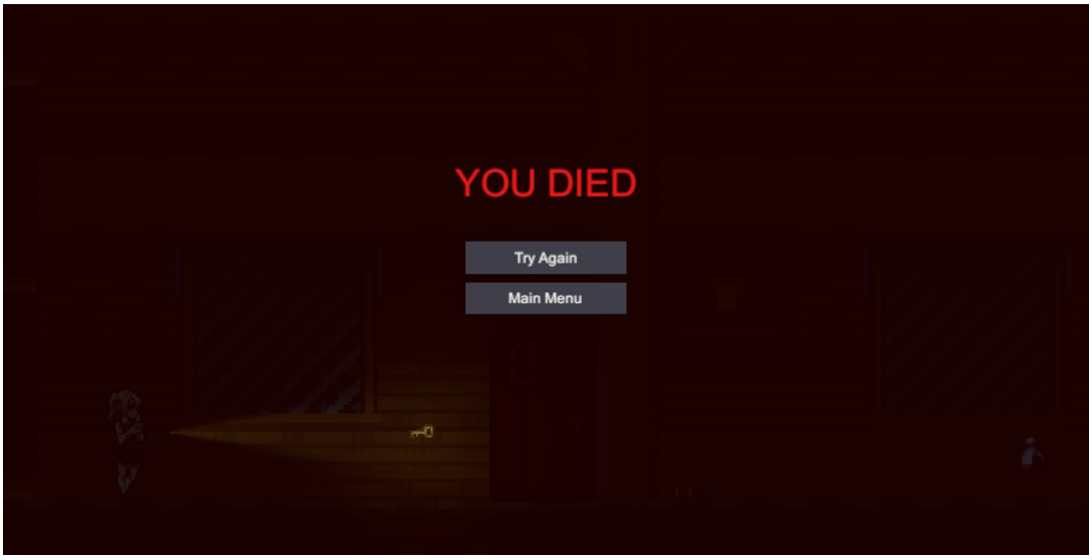

ภาพที่ 3-10 การตรวจพบและไล่ล่าผู้เล่น (ภาพสาธิตใน Play Mode)

เมื่อศัตรูอยู่ในระยะโจมตี ระบบลดพลังชีวิตของผู้เล่น หากพลังชีวิตเป็นศูนย์ Player FSM เปลี่ยนเป็น Dead และแสดงหน้าจอ Game Over

HURT




GAMEOVER




ภาพที่ 3-11 การได้รับความเสียหายและ Game Over (ภาพสาธิตใน Play Mode)

หลังเก็บกุญแจครบ ผู้เล่นต้องกด E ค้างที่ประตูจนความคืบหน้าครบ 100 เปอร์เซ็นต์ การปลดล็อกจะถูกยกเลิกเมื่อปล่อยปุ่ม ออกจากระยะ ถูกโจมตี หรือมีศัตรูอยู่ใกล้



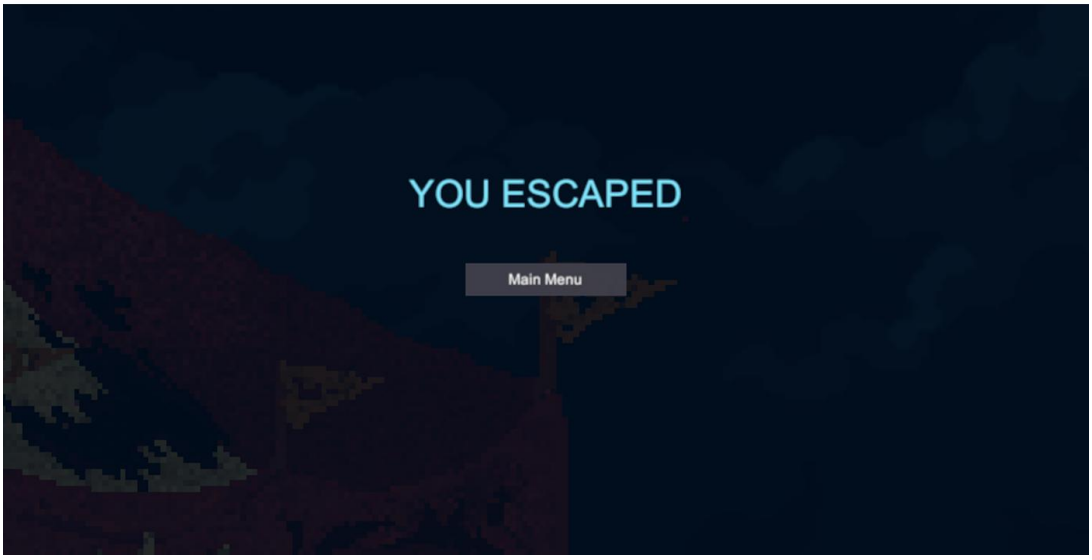

ภาพที่ 3-12 การปลดล็อกประตูทางออก (ภาพสาธิตใน Play Mode)

เมื่อปลดล็อกประตูสำเร็จ ระบบแสดง Level Complete และโหลดด่านถัดไป หากผ่านด่านสุดท้ายระบบแสดง Game Won

COMPLETE




WON



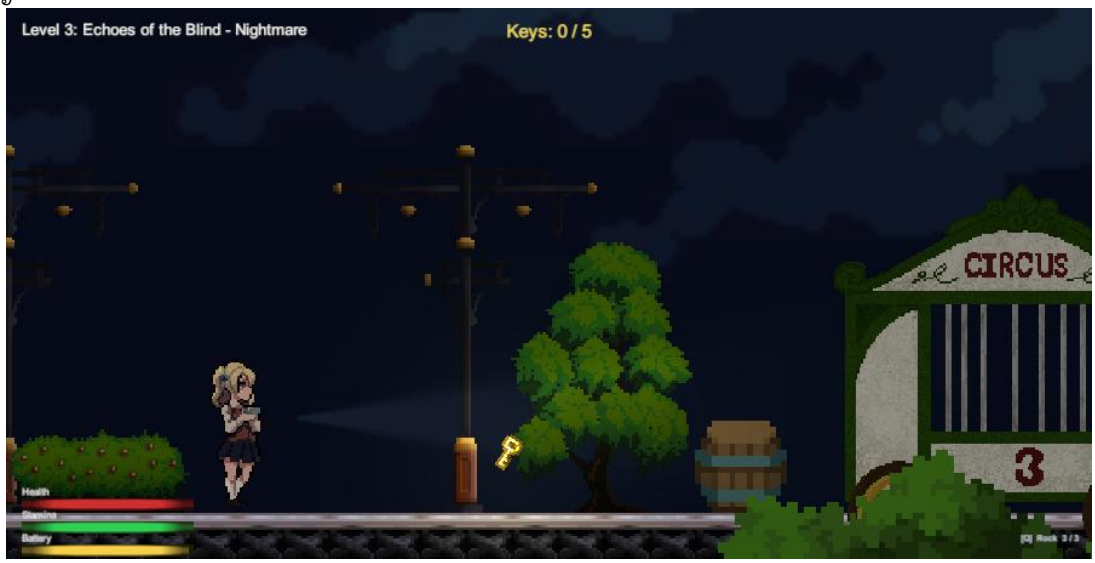

ภาพที่ 3-13 หน้าจอผ่านด่านและจบเกม (ภาพสาธิตใน Play Mode)

Level 2 มีสถานะคำสาป Safe, Exposed, Warning, Critical, Petrified และ Recovering โดย HUD แสดงแถบคำสาป ภาพบดบัง และข้อความเตือนตามระดับอันตราย




ภาพที่ 3-14 ระบบคำสาปใน Level 2 (ภาพสาธิตใน Play Mode)

Level 3 ออกแบบเป็นป่ามืดโดยใช้หมอก ต้นสนระยะไกล ต้นไม้และพุ่มไม้ชั้นกลาง รวมถึงพื้นหญ้าและเส้นทางต่างระดับเพื่อสร้างความลึกของฉาก




ภาพที่ 3-15 สภาพแวดล้อมสวนสนุกและพื้นที่ป่าใน Level 3 (ภาพสาธิตใน Play Mode)

กับดักกระจกทำงานเมื่อผู้เล่นเดินชน โดยเล่นเสียงแตก เปลี่ยนลักษณะวัตถุ และส่งเสียงประเภท Glass ไปยัง KillerAI




ภาพที่ 3-16 สาธิตตำแหน่งกับดักเสียงใน Level 3 (ภาพสาธิตใน Play Mode)

เมื่อศัตรูอยู่ในระยะเตือน เกมเล่นเสียงหัวใจและแสดงขอบจอสีแดง ความถี่และความเข้มเพิ่มขึ้นตามระดับอันตราย




ภาพที่ 3-17 ระบบแจ้งเตือนเมื่อศัตรูอยู่ใกล้ (ภาพสาธิตใน Play Mode)

## 3.5 การพัฒนาระบบ

### 3.5.1 สถาปัตยกรรมของระบบ

ระบบประกอบด้วย PlayerController, PlayerHealth, FlashlightController, HidingSystem, FootstepSystem, KillerAI, InventoryManager, ExitDoor, GameManager, UIManager, AudioManager และ TensionDirector โดย PlayerController รับข้อมูลจากผู้เล่นและส่งเหตุการณ์ให้ระบบที่

เกี่ยวข้อง FootstepSystem ส่งเหตุการณ์เสียงให้ KillerAI ขณะที่ InventoryManager ส่งจำนวนกุญแจให้ UIManager และ ExitDoor ส่วน GameManager ควบคุมสถานะเกมและการเปลี่ยนฉาก

### 3.5.2 การพัฒนา Player FSM

Player FSM ใช้สถานะ Normal, Hidden, Frozen, Hurt, UsingItem, Transitioning และ Dead เพื่อควบคุมการรับคำสั่ง Normal เป็นสถานะที่อนุญาตการควบคุมทั่วไป Hidden ใช้เมื่ออยู่ในจุดซ่อน Frozen และ Hurt ใช้หยุดหรือจำกัดการควบคุมชั่วคราว UsingItem ป้องกันคำสั่งที่ขัดแย้งระหว่างใช้ไอเท็ม Transitioning ใช้ระหว่างเปลี่ยนฉาก และ Dead ใช้เมื่อพลังชีวิตหมด

### 3.5.3 การพัฒนา Enemy FSM

Enemy FSM รับข้อมูลจากระบบการมองเห็นและการได้ยิน จากนั้นประเมินความสงสัยและเลือกสถานะ เสียง Glass และ Sprint มีความสำคัญมากกว่าเสียง Footstep ทั่วไป ศัตรูเก็บตำแหน่งล่าสุดไว้ชั่วคราวเพื่อใช้ตรวจสอบและค้นหา หากไม่พบเป้าหมายจะกลับไปยังเส้นทางลาดตระเวน

### 3.5.4 การพัฒนาระบบความตึงเครียด

TensionDirector รวมสถานะของศัตรูทุกตัวเป็น Calm, Suspicious, Hunted, Escaped และ Cooldown ข้อมูลนี้ใช้ควบคุมระดับความตึงเครียด ดนตรีไล่ล่า เสียงหัวใจ ภาพเตือน และตัวคูณความเร็วของศัตรูระหว่างการไล่ล่า

### 3.5.5 การพัฒนาระบบไอเท็มและประตูทางออก

InventoryManager เก็บไอเท็มใน List และส่งเหตุการณ์เมื่อจำนวนกุญแจเปลี่ยน ExitDoor มีสถานะ Locked, Ready, Unlocking, Open, Transitioning และ Completed การใช้ FSM กับประตูช่วยควบคุมการปลดล็อกแบบกดค้างและรองรับการยกเลิกเมื่อสถานการณ์ไม่ปลอดภัย

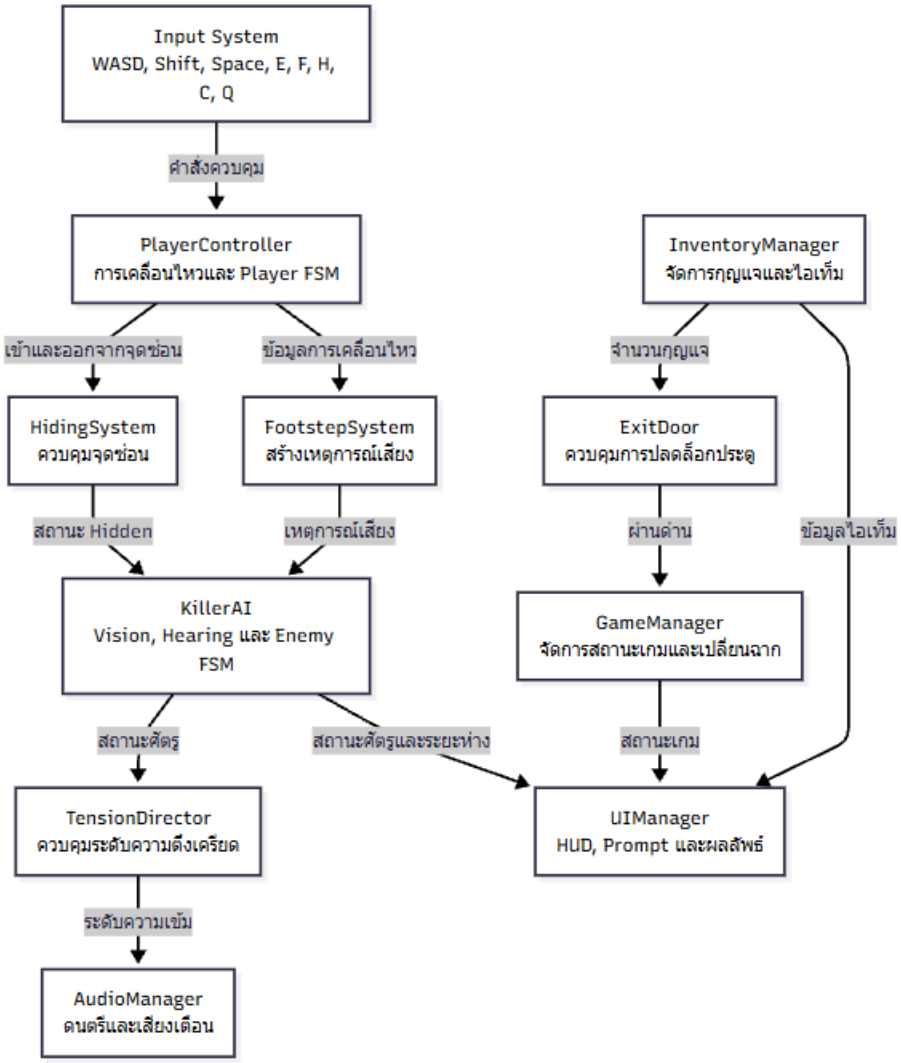
### 3.5.6 การพัฒนาด่าน Level 3

Level 3 ใช้ Tilemap และ Sprite ที่มีอยู่ในโครงการจัดวางเป็นป่า โดยแบ่งเป็นชั้นหมอกและต้นไม้ระยะไกล ชั้นต้นไม้และพุ่มไม้ระยะกลาง และชั้นพื้นสำหรับการเล่น ระบบ Level3RandomEvents ค้นหาพื้นที่เดินได้และกระจายกุญแจกับกับดักระหว่างเส้นทางชั้นล่างและชั้นบน เพื่อลดการรวมตัวของวัตถุในตำแหน่งเดียว

### 3.5.7 การจัดการเงื่อนไขพิเศษ

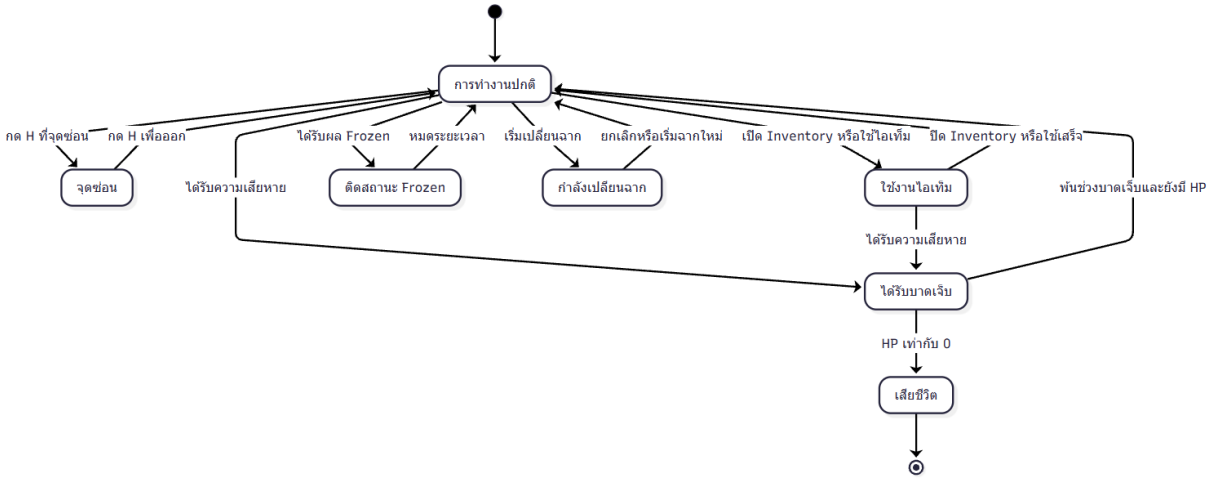
ระบบป้องกันการกระโดดซ้ำกลางอากาศ รองรับการกระโดดออกจากบันได ล็อกการเคลื่อนไหวขณะซ่อน ใช้ไอเท็ม เปลี่ยนฉาก หรือเสียชีวิต ลบเสียงที่หมดอายุจากหน่วยความจำของศัตรู ลดประสิทธิภาพการล่อด้วยก้อนหินซ้ำ และยกเลิกการปลดล็อกประตูเมื่อผู้เล่นอยู่ในอันตราย

แผนภาพแสดงการรับข้อมูลจาก Input System ผ่าน PlayerController การส่งเหตุการณ์ไปยังระบบไอเท็ม ระบบเสียง KillerAI UI และ GameManager



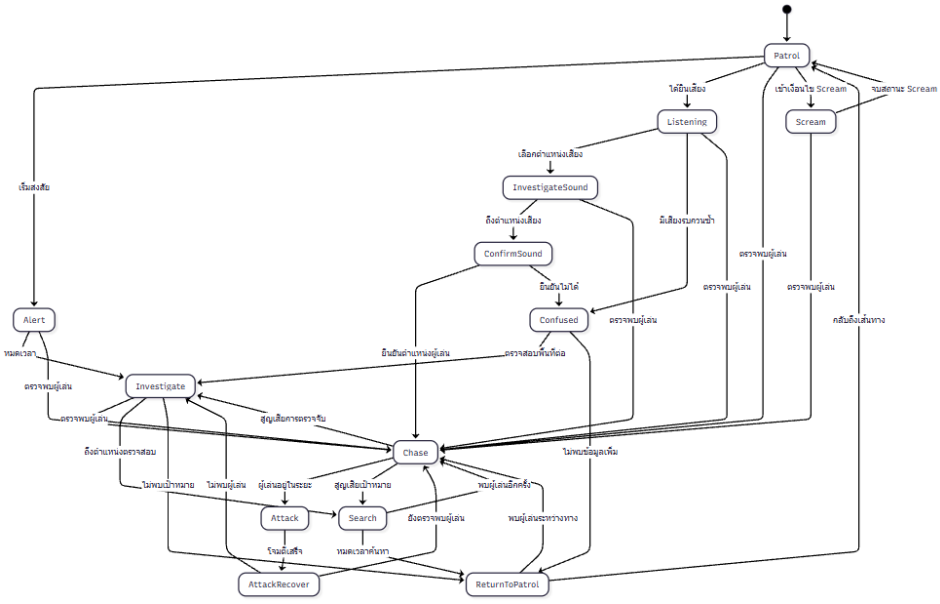

ภาพที่ 3-18 แผนภาพสถาปัตยกรรมของระบบ

แผนภาพแสดงสถานะของผู้เล่นและเหตุการณ์สำคัญที่ทำให้เปลี่ยนระหว่าง Normal, Hidden, Frozen, Hurt, UsingItem, Transitioning และ Dead



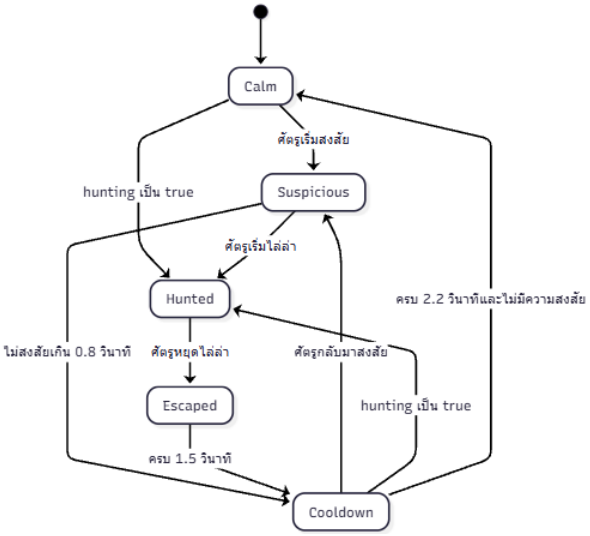

ภาพที่ 3-19 แผนภาพ Player FSM

แผนภาพแสดงกระบวนการตั้งแต่ Patrol การรับรู้เสียงหรือการมองเห็น การตรวจสอบ การค้นหา การไล่ล่า การโจมตี และการกลับไปลาดตระเวน




ภาพที่ 3-20 แผนภาพ Enemy FSM

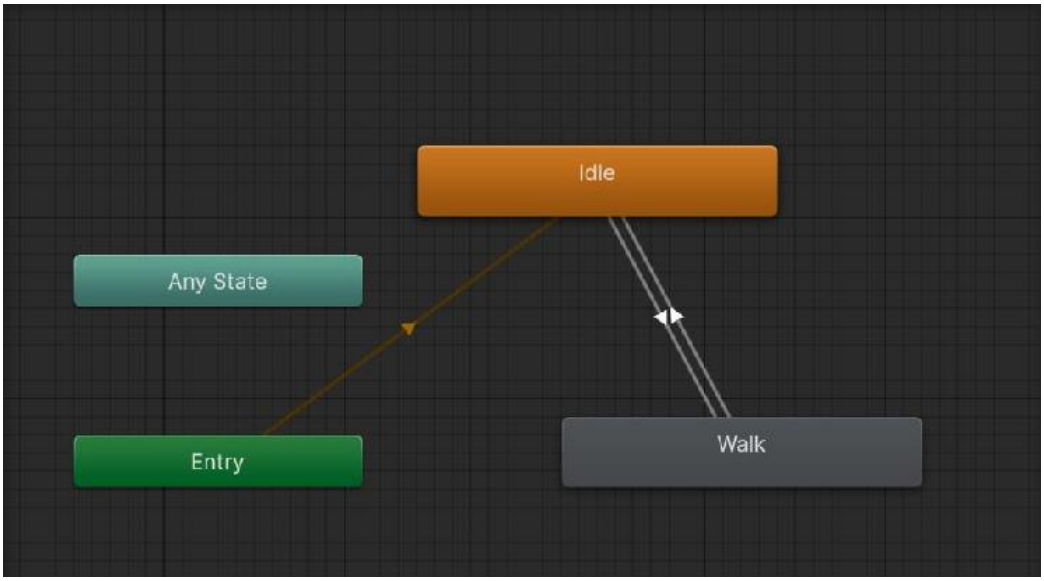
แผนภาพแสดงสถานะ Calm, Suspicious, Hunted, Escaped และ Cooldown ซึ่งควบคุมเสียงและภาพเตือนผู้เล่น




ภาพที่ 3-21 แผนภาพสถานะความตึงเครียด

### 3.5.8 การพัฒนาระบบแอนิเมชันของตัวละครผู้เล่น

ระบบแอนิเมชันใช้ Animator Controller ของ Unity
เพื่อกำหนดสถานะภาพเคลื่อนไหวและเงื่อนไขเปลี่ยนสถานะ โดย Animation Clip
เก็บลำดับภาพของตัวละคร ส่วน Animator เลือกคลิปที่ต้องแสดงตามพารามิเตอร์ซึ่งได้รับจากสคริปต์
วิธีนี้ทำให้การแสดงผลสัมพันธ์กับการเคลื่อนที่ และตรวจสอบเส้นทางการเปลี่ยนแอนิเมชันได้จากแผนผัง




ภาพที่ 3-22 แผนผัง Animator ของตัวละครผู้เล่นจาก Unity

จากภาพที่ 3-22 PlayerAnimator.controller มีสถานะ Idle และ Walk โดย Idle
เป็นสถานะเริ่มต้น แสดงเป็นกล่องสีส้ม ลูกศรจาก Entry จึงเข้าสู่ Idle ก่อน ส่วน Walk
แสดงภาพการเดิน ลูกศรระหว่างสองสถานะเป็น Transition ที่ใช้เงื่อนไข isWalking ชนิด Bool
เพื่อเลือกแอนิเมชัน

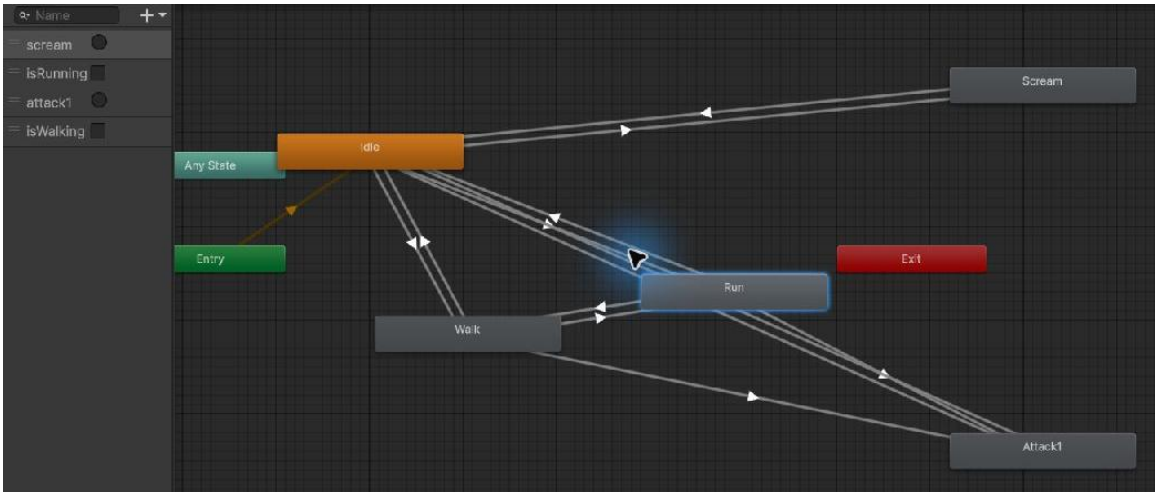
เมื่อ isWalking เป็น true ระบบเปลี่ยนจาก Idle ไป Walk และเมื่อเป็น false จะกลับจาก
Walk ไป Idle ทั้งสอง Transition ปิด Has Exit Time และมี Transition Duration เท่ากับ 0
จึงเปลี่ยนสถานะได้ทันทีเมื่อเงื่อนไขเป็นจริง โดยไม่ต้องรอให้คลิปเดิมเล่นถึงจุดสิ้นสุด

PlayerController ส่งค่าด้วย animator.SetBool(IsWalkingHash, IsMoving)
จึงใช้สถานะการเคลื่อนที่ของผู้เล่นเป็นตัวควบคุมภาพจริง ใน Controller ปัจจุบันยังไม่มีสถานะ Run
แยกต่างหาก การวิ่งจึงไม่ได้หมายความว่ามีคลิป Run ของผู้เล่นอยู่ในแผนผังนี้

Any State เป็นจุดสำหรับกำหนดการเปลี่ยนจากสถานะใดก็ได้ แต่ของผู้เล่นยังไม่มี Transition
ออกจากจุดนี้ ส่วน Entry เป็นจุดเริ่มต้น ไม่ใช่คลิปแอนิเมชัน
การแยกความหมายของแต่ละกล่องช่วยให้ตีความแผนผังได้ถูกต้อง

### 3.5.9 การพัฒนาระบบแอนิเมชันของ Killer

Killer ใช้ KillerAnimator.controller เชื่อมกับ KillerAI
เพื่อแสดงท่าทางตามพฤติกรรมของศัตรู ประกอบด้วย Idle สำหรับหยุดนิ่ง Walk สำหรับการเดิน Run สำหรับไล่ล่า Attack1 สำหรับโจมตี และ Scream สำหรับกรีดร้อง ภาพที่ 3-23 จับจากหน้าต่าง Animator ของ Unity โดยจัดตำแหน่งกล่องให้เห็นชื่อสถานะและลูกศรชัดเจน




ภาพที่ 3-23 แผนผัง Animator และ Parameters ของ Killer จาก Unity

Entry เริ่มที่ Idle ส่วนการเดินใช้ isWalking ชนิด Bool และการวิ่งใช้ isRunning ชนิด Bool สคริปต์กำหนด isWalking เป็น true เมื่ออยู่ใน Patrol, Investigate, InvestigateSound, Search หรือ ReturnToPatrol และมีค่าสัมบูรณ์ของความเร็วแนวนอนมากกว่า 0.01 ส่วน isRunning เป็น true เมื่ออยู่ใน Chase หรือ Attack

การโจมตีใช้ attack1 ชนิด Trigger โดย KillerAI เรียก SetTrigger เมื่อโจมตีตามรอบคูลดาวน์ หรือเริ่มตรวจค้นจุดซ่อนที่สงสัย จึงเรียกท่า Attack1 เป็นเหตุการณ์แต่ละครั้ง ขณะที่ scream เป็น Trigger ซึ่งเรียกเมื่อระบบสั่งกรีดร้องหรือใช้ความสามารถหยุดผู้เล่นตามเงื่อนไขของศัตรู

Any State เชื่อมไป Scream เพื่อให้การกรีดร้องเริ่มได้จากสถานะใดก็ได้ ส่วน Attack1 รับ Transition จาก Idle, Walk และ Run จึงไม่ได้ใช้ Any State สำหรับการโจมตี เมื่อ Attack1 หรือ Scream เล่นครบตาม Exit Time เท่ากับ 1 จะกลับ Idle
แล้วตรวจพารามิเตอร์การเคลื่อนที่เพื่อเลือกสถานะถัดไป

### 3.5.10 เงื่อนไขการเปลี่ยนแอนิเมชันและการเชื่อมกับ FSM

ลูกศรในภาพที่ 3-23 แสดงทิศทางที่ Animator เปลี่ยนสถานะได้ เงื่อนไขของ Transition ต้องสอดคล้องกับค่าที่ KillerAI ส่งเข้ามา โดย Controller ปัจจุบันกำหนดเส้นทางดังต่อไปนี้

Idle ไป Walk เมื่อ isWalking เป็น true และ Walk กลับ Idle เมื่อ isWalking เป็น false

Idle หรือ Walk ไป Run เมื่อ isRunning เป็น true และ Run กลับ Idle เมื่อ isRunning เป็น false

Run ไป Walk เมื่อ isRunning เป็น false และ isWalking เป็น true
โดยเงื่อนไขทั้งสองต้องเป็นจริงร่วมกัน

Idle, Walk หรือ Run ไป Attack1 เมื่อได้รับ Trigger attack1 และ Attack1 กลับ Idle ด้วย Has Exit Time ที่ค่า 1

Any State ไป Scream เมื่อได้รับ Trigger scream และ Scream กลับ Idle ด้วย Has Exit Time ที่ค่า 1

Transition ระหว่างการเคลื่อนที่ของ Killer มี Duration เท่ากับ 0.1 วินาที และปิด Has Exit Time เพื่อให้ตอบสนองต่อพฤติกรรมได้โดยไม่รอจบรอบคลิป การเข้าสู่ Attack1 ใช้ Duration 0.05 วินาที ส่วน Any State ไป Scream ใช้ Duration 0 เพื่อเริ่มกรีดร้องทันที การกลับจาก Attack1 และ Scream ใช้ Duration 0.1 วินาที

FSM ใน KillerAI รับหน้าที่ตัดสินใจ เช่น ลาดตระเวน ค้นหา ไล่ล่า หรือโจมตี ส่วน Animator รับหน้าที่แสดงภาพของการตัดสินใจนั้น จึงไม่จำเป็นต้องมีจำนวนสถานะเท่ากัน ตัวอย่างเช่น Patrol และ Search ใช้ภาพ Walk ร่วมกันได้ แต่ยังมีตรรกะเลือกเป้าหมายต่างกันใน FSM

สำหรับผู้เล่น หลักการเดียวกันใช้กับ Player FSM ซึ่งควบคุมการรับคำสั่งและข้อจำกัดในสถานะ Normal, Hidden, Frozen, Hurt, UsingItem, Transitioning และ Dead ส่วน PlayerAnimator ปัจจุบันเลือกภาพระหว่าง Idle กับ Walk ตาม IsMoving การเพิ่มสถานะใน FSM
จึงไม่ทำให้เกิดคลิปแอนิเมชันใหม่โดยอัตโนมัติ

การคำนวณความเสียหายอยู่ใน KillerAI ซึ่งเรียก playerHealth.TakeDamage ก่อนส่ง Trigger attack1 แอนิเมชันโจมตีจึงเป็นภาพประกอบเหตุการณ์ ไม่ใช่ตัวกำหนดความเสียหายด้วย Animation Event ในส่วนโค้ดนี้ หากต้องการให้ความเสียหายตรงกับเฟรมที่อาวุธสัมผัสผู้เล่น จึงค่อยออกแบบ Animation Event เพิ่มเติม

เมื่อ isRunning เป็น false และ isWalking เป็น true เส้นจาก Run ไป Idle กับเส้นไป Walk อาจผ่านเงื่อนไขพร้อมกัน จึงควรตรวจลำดับ Transition
เพื่อให้ผลการเปลี่ยนภาพสอดคล้องกับพฤติกรรมที่ต้องการ

### 3.5.11 เหตุผลในการเลือกชนิดพารามิเตอร์ของ Animator

Unity มีพารามิเตอร์ Float, Int, Bool และ Trigger เพื่อแทนข้อมูลต่างลักษณะ การเลือกชนิดขึ้นอยู่กับว่าเป็นค่าต่อเนื่อง รหัสสถานะ เงื่อนไขที่คงอยู่ หรือเหตุการณ์เฉพาะครั้ง ไม่จำเป็นต้องใช้ครบทั้งสี่ชนิดใน Controller เดียว โครงการนี้ใช้ Bool และ Trigger จริง ส่วน Float และ Int อธิบายเป็นทางเลือกสำหรับการขยายระบบ

**Float สำหรับค่าต่อเนื่อง**

Float เก็บตัวเลขที่มีทศนิยม เหมาะกับค่าความเร็ว เช่น 0, 1.5 และ 3.2 หากเพิ่มพารามิเตอร์ Speed จะส่งค่าด้วย SetFloat และใช้เปรียบเทียบกับเกณฑ์หรือควบคุม Blend Tree เพื่อผสมท่ายืน เดิน และวิ่งตามความเร็วได้ เหตุผลที่เลือก Float คือรักษารายละเอียดระหว่างค่าความเร็วซึ่ง Int จะเก็บไม่ได้ ปัจจุบันยังไม่มีพารามิเตอร์ Float ใน Animator ทั้งสองตัว

**Int สำหรับรหัสหรือทางเลือกหลายค่า**

Int เก็บจำนวนเต็ม เหมาะกับการแทนโหมดที่แยกกัน เช่น MovementMode เท่ากับ 0 สำหรับยืน 1 สำหรับเดิน และ 2 สำหรับวิ่ง แล้วส่งค่าด้วย SetInteger เหตุผลที่เลือก Int คือใช้ค่าเดียวระบุทางเลือกหลายแบบ ลดการใช้ Bool หลายตัวที่อาจเป็น true พร้อมกัน อย่างไรก็ตาม ตัวแปร int ที่เก็บผล Animator.StringToHash เป็นเพียงรหัสอ้างอิงชื่อพารามิเตอร์ ไม่ได้ทำให้พารามิเตอร์นั้นเป็นชนิด Int และ Controller ปัจจุบันยังไม่ได้ใช้ Int

**Bool สำหรับเงื่อนไขที่ต้องคงสถานะ**

Bool มีค่า true หรือ false เหมาะกับคำถามว่ากำลังเดินหรือกำลังวิ่งหรือไม่ จึงใช้ isWalking ในผู้เล่น และ isWalking กับ isRunning ใน Killer โดยส่งค่าด้วย SetBool เหตุผลที่เลือก Bool คือระบบต้องรักษาค่าขณะพฤติกรรมยังดำเนินอยู่ และเปลี่ยนกลับเมื่อพฤติกรรมหยุด ทำให้การเข้าและออกจาก Walk หรือ Run ตรวจสอบได้ตรงไปตรงมา

**Trigger สำหรับเหตุการณ์เฉพาะครั้ง**

Trigger เหมาะกับคำสั่งเริ่มการกระทำ เช่น โจมตีหรือกรีดร้อง จึงใช้ attack1 และ scream ผ่าน SetTrigger เมื่อ Transition ใช้ Trigger แล้ว Controller จะรีเซ็ตค่าโดยอัตโนมัติ หากยังไม่มี Transition ใช้ค่านั้น อาจต้องเรียก ResetTrigger เพื่อยกเลิกคำสั่งที่ค้างอยู่ เหตุผลที่เลือก Trigger คือไม่ต้องเปิด Bool ค้างไว้และคอยสั่งปิดเองทุกครั้ง ส่วนการออกจากคลิปยังต้องกำหนด Transition และ Exit Time ให้เหมาะสม

แหล่งตรวจสอบระบบคือ PlayerAnimator.controller, KillerAnimator.controller, PlayerController.cs และ KillerAI.cs ในโปรเจกต์ SideScrollingEscape ส่วนชนิดพารามิเตอร์อ้างอิง Unity Manual เรื่อง Animation Parameters ที่ https://docs.unity3d.com/6000.4/Documentation/Manual/AnimationParameters.html ตรวจสอบวันที่ 4 ตุลาคม 2569

## 3.6 การทดสอบระบบ

ตารางต่อไปบันทึกผลการทดสอบใน Unity 6000.4.9f1 เมื่อวันที่ 4 ตุลาคม 2569 โดยใช้ผลชุดเดียวกับหัวข้อ 4.10 ทดสอบใน Play Mode ด้วยสคริปต์ชั่วคราวควบคุมอินพุต ตำแหน่ง เวลา และเงื่อนไขขององค์ประกอบ สถานะผ่านหมายถึงผ่านกรณีควบคุมที่ระบุ รวม 30 กรณีของระบบ ไม่ใช่การรับรองการเล่นครบทุกเส้นทางหรือผลประเมินจากผู้ใช้งานจริง

### 3.6.1 การทดสอบระบบควบคุมพื้นฐาน

| **รหัส** | **รายการทดสอบ** | **ขั้นตอน** | **ผลที่คาดหวัง** | ผลที่บันทึกและสถานะ |
|---|---|---|---|---|
| TC-01 | การเดิน | กด A D<br>หรือปุ่มลูกศร | ผู้เล่นเคลื่อนที่และหันถูกทิศ | กด D ได้ความเร็ว 4.5 และหันขวา กด A ได้ −4.5 และหันซ้าย<br>สถานะ: ผ่าน |
| TC-02 | การวิ่ง | กด Shift<br>พร้อมเคลื่อนที่ | ความเร็วเพิ่ม Stamina ลด และเกิดเสียง Sprint | ความเร็วเดิน 4.5 เพิ่มเป็น 5.7 ขณะวิ่ง ค่า Stamina ลด และส่งเสียงประเภท Sprint<br>สถานะ: ผ่าน |
| TC-03 | การกระโดด | กด Space<br>บนพื้นและกลางอากาศ | กระโดดได้ครั้งเดียวและไม่กระโดดซ้ำกลางอากาศ | กระโดดได้แรงยกครั้งแรก การสั่งกระโดดซ้ำทันทีขณะลอยไม่เพิ่มแรงยก<br>สถานะ: ผ่าน |
| TC-04 | การไต่บันได | กดขึ้นหรือลงเมื่ออยู่บนบันได | ผู้เล่นไต่บันไดและกระโดดออกได้ | ขึ้นบันไดด้วยความเร็ว 4 และลงด้วย −4 กระโดดออกได้และคืนค่าแรงโน้มถ่วง<br>สถานะ: ผ่าน |
| TC-05 | ไฟฉาย | กด F<br>และเปิดทิ้งไว้ | ไฟฉายสลับสถานะและแบตเตอรี่ลดลง | เปิดปิดได้ แบตเตอรี่ลดขณะเปิด เมื่อเหลือศูนย์ไฟดับและเปิดซ้ำไม่ได้<br>สถานะ: ผ่าน |

### 3.6.2 การทดสอบ Player FSM

| รหัส | การเปลี่ยนสถานะ | เงื่อนไขทดสอบ | ผลที่คาดหวัง | ผลที่บันทึกและสถานะ |
|---|---|---|---|---|
| TC-FSM-P01 | Normal > Hidden | เข้าใกล้จุดซ่อนและกด H | ผู้เล่นซ่อนและ HUD แสดง HIDDEN | ToggleHide เปลี่ยนเป็น Hidden และล็อกการเคลื่อนไหว สถานะ: ผ่าน |
| TC-FSM-P02 | Hidden > Normal | กด H เพื่อออกจากจุดซ่อน | กลับมาเคลื่อนที่ได้ตามปกติ | ToggleHide ออกจากที่ซ่อน กลับ Normal และคืนการควบคุม สถานะ: ผ่าน |
| TC-FSM-P03 | Normal > Hurt > Normal | ให้ศัตรูโจมตีโดย HP ยังเหลือ | HP ลดและกลับสู่ Normal หลังช่วง Hurt | รับความเสียหาย 10 ทำให้ HP 100 เป็น 90 เข้า Hurt และกลับ Normal หลังรอ 2.2 วินาที สถานะ: ผ่าน |
| TC-FSM-P04 | Normal > Frozen > Normal | รับผล Frozen และทำเงื่อนไขหลุด | หยุดเคลื่อนที่และกลับมาควบคุมได้ | Freeze เปลี่ยนเป็น Frozen กด X ครบ 2 ครั้งตามค่าทดสอบแล้วกลับ Normal สถานะ: ผ่าน |
| TC-FSM-P05 | Normal > UsingItem > Normal | เปิดและปิด Inventory | คำสั่งการเล่นถูกล็อกและคืนกลับถูกต้อง | เปิดสถานะคลังเป็น UsingItem และล็อกการเคลื่อนไหว ปิดแล้วกลับ Normal สถานะ: ผ่าน |
| TC-FSM-P06 | Normal > Transitioning | ปลดล็อกประตูสำเร็จ | ล็อกการควบคุมระหว่างเปลี่ยนด่าน | BeginTransition ล็อกคำสั่ง และเมื่อประตูสำเร็จผู้เล่นเข้าสู่ Transitioning สถานะ: ผ่าน |
| TC-FSM-P07 | Hurt > Dead | รับความเสียหายจน HP เป็นศูนย์ | เข้าสู่ Dead และแสดง Game Over | รับความเสียหายจน HP หมด ผู้เล่นเป็น Dead และสถานะเกมเป็น Dead สถานะ: ผ่าน |

### 3.6.3 การทดสอบ Enemy FSM

| รหัส | การเปลี่ยนสถานะ | เงื่อนไขทดสอบ | ผลที่คาดหวัง | ผลที่บันทึกและสถานะ |
|---|---|---|---|---|
| TC-FSM-E01 | Patrol > Listening | สร้างเสียงในระยะได้ยิน | ศัตรูหยุดลาดตระเวนและฟังเสียง | ส่งเสียง Rock ในระยะได้ยิน ทำให้ Patrol เปลี่ยนเป็น Listening<br>สถานะ: ผ่าน |
| TC-FSM-E02 | Listening > InvestigateSound | รอจนศัตรูเลือกเสียง | ศัตรูเคลื่อนไปยังต้นกำเนิดเสียง | หมดเวลาฟังแล้วเข้า InvestigateSound และมีความเร็วเคลื่อนที่ไปยังเสียง<br>สถานะ: ผ่าน |
| TC-FSM-E03 | InvestigateSound > ConfirmSound | ศัตรูถึงตำแหน่งเสียง | ศัตรูตรวจสอบบริเวณดังกล่าว | เมื่อตำแหน่งถึงต้นกำเนิดเสียง เปลี่ยนเป็น ConfirmSound<br>สถานะ: ผ่าน |
| TC-FSM-E04 | Patrol/Investigate > Chase | ให้ศัตรูตรวจพบผู้เล่น | ศัตรูไล่ล่าและ HUD แสดง DANGER | ผู้เล่นอยู่ 2 หน่วยด้านที่ Killer หันและไม่มีสิ่งกีดขวาง ตรวจพบจริงแล้วเข้า Chase<br>สถานะ: ผ่าน |
| TC-FSM-E05 | Chase > Attack | ผู้เล่นอยู่ในระยะโจมตี | ศัตรูโจมตีและลด HP | ผู้เล่นอยู่ในระยะโจมตี Chase เข้า Attack จากนั้น HP 100 ลดเป็น 75 และเข้า AttackRecover<br>สถานะ: ผ่าน |
| TC-FSM-E06 | Attack > AttackRecover | รอหลังการโจมตี | ศัตรูพักตามช่วง cooldown | คง AttackRecover ก่อนเวลาสิ้นสุด และกลับ Chase<br>เมื่อครบเวลาขณะที่ยังตรวจพบเป้าหมาย<br>สถานะ: ผ่าน |
| TC-FSM-E07 | Chase > Investigate/Search | หลบออกจากการตรวจจับ | ศัตรูตรวจตำแหน่งล่าสุดและค้นหา | สูญเสียเป้าหมายหลังช่วงความจำแล้วเข้า Investigate<br>เมื่อหมดเวลาตรวจสอบเข้า Search<br>สถานะ: ผ่าน |

| รหัส | การเปลี่ยนสถานะ | เงื่อนไขทดสอบ | ผลที่คาดหวัง | ผลที่บันทึกและสถานะ |
|---|---|---|---|---|
| TC-FSM-E08 | Search > ReturnToPatrol | รอจนหมดเวลาค้นหา | ศัตรูกลับเส้นทางลาดตระเวน | หมดเวลาค้นหาแล้วเปลี่ยนจาก Search เป็น ReturnToPatrol<br>สถานะ: ผ่าน |
| TC-FSM-E09 | Listening > Confused | ขว้างก้อนหินซ้ำบริเวณเดิม | ศัตรูเข้าสู่ Confused ตามเงื่อนไข | ส่ง Rock ซ้ำจุดเดิม 3 ครั้ง เมื่อยืนยันเสียงแล้วเข้า Confused<br>สถานะ: ผ่าน |
| TC-FSM-E10 | สถานะทั่วไป > Scream | ทำให้ศัตรูที่ตั้งค่า Scream เข้าเงื่อนไข | ผู้เล่นถูก Stun หรือ Frozen | เรียกเหตุการณ์กรีดร้องแล้ว Killer เข้า Scream และผู้เล่นถูก Stun จนควบคุมไม่ได้ชั่วคราว<br>สถานะ: ผ่าน |

### 3.6.4 การทดสอบระบบเกมและ Level 3

| รหัส | รายการทดสอบ | ขั้นตอน | ผลที่คาดหวัง | ผลที่บันทึกและสถานะ |
|---|---|---|---|---|
| TC-06 | เก็บกุญแจ | เก็บกุญแจหนึ่งชิ้น | จำนวนกุญแจและ Inventory อัปเดต | Collect เพิ่มกุญแจจาก 0 เป็น 1 และเพิ่มรายการใน Inventory<br>สถานะ: ผ่าน |
| TC-07 | ประตูยังล็อก | ใช้ประตูก่อนกุญแจครบ | แสดงจำนวนกุญแจที่ยังขาด | ขณะมีกุญแจ 1 จาก 5 ประตูยัง Locked และแจ้งว่าขาดอีก 4 ดอก<br>สถานะ: ผ่าน |
| TC-08 | ปลดล็อกประตู | เก็บครบและกด E ค้าง | ความคืบหน้าครบและผ่านด่าน | กด E ค้างตามระยะเวลาปลดล็อก ประตูเป็น Completed และเกมเป็น LevelComplete<br>สถานะ: ผ่าน |

| รหัส | รายการทดสอบ | ขั้นตอน | ผลที่คาดหวัง | ผลที่บันทึกและสถานะ |
|---|---|---|---|---|
| TC-09 | ยกเลิกการปลดล็อก | ปล่อย E เดินออก หรือถูกโจมตี | ความคืบหน้าถูกยกเลิก | ยกเลิกและคืนความคืบหน้าเป็นศูนย์เมื่อปล่อย E ออกจากระยะ รับความเสียหาย หรือ Killer อยู่ใกล้ สถานะ: ผ่าน |
| TC-10 | กับดักกระจก | เหยียบกับดักใน Level 3 | เกิดเสียง Glass และศัตรูตรวจสอบ | กับดักทำงาน ปิด Collider ส่ง Glass จน Killer เข้า Listening และ AudioSource เล่นเสียงที่มีข้อมูลเสียงจริง สถานะ: ผ่าน |
| TC-11 | การสุ่มวัตถุ | เข้า Level 3 หลายรอบ | กุญแจและกับดักกระจายตามจุดที่เดินได้ | สุ่มตำแหน่ง 5 รอบ ตำแหน่งกุญแจเปลี่ยน และ Raycast พบพื้น Ground ใต้กุญแจ สถานะ: ผ่าน |
| TC-12 | เสียงหัวใจ | เข้าใกล้ศัตรู | ความถี่เสียงและขอบจอแดงเพิ่มตามอันตราย | ความเข้มเตือนเพิ่มจาก 0 เป็นประมาณ 0.929 เมื่อเข้าใกล้ ขอบจอมีค่า Alpha และคลิปหัวใจมีข้อมูลเสียง สถานะ: ผ่าน |
| TC-13 | จบเกม | ผ่าน Level 3 | ระบบแสดง Game Won | แจ้งผ่านด่านสุดท้ายและสั่งดำเนินต่อแล้วสถานะเกมเป็น GameWon สถานะ: ผ่าน |

### 3.6.5 การทดสอบโดยผู้ใช้งาน

การทดสอบโดยผู้ใช้งานมีวัตถุประสงค์เพื่อประเมินความถูกต้อง ความเสถียร ความเข้าใจง่ายของการควบคุม ความชัดเจนของ UI ความเหมาะสมของระดับความยาก และบรรยากาศของเกม ผู้ทดสอบให้คะแนนด้วยมาตราส่วน 5 ระดับ ได้แก่ 5 ดีมาก 4 ดี 3 ปานกลาง 2 น้อย และ 1 น้อยมาก

ผลการประเมินจะวิเคราะห์ด้วยค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐาน ทั้งนี้ต้องระบุจำนวนผู้ทดสอบ คุณสมบัติของกลุ่มตัวอย่าง วันที่หรือช่วงเวลาทดสอบ และผลคะแนนจริงก่อนนำเสนอข้อสรุป

## 3.7 การบำรุงรักษาระบบ

### 3.7.1 การบำรุงรักษาเพื่อแก้ไขข้อผิดพลาด

ตรวจสอบและแก้ไขปัญหาการเปลี่ยนสถานะของผู้เล่นและศัตรู การติดขอบพื้น การเดินบนสะพาน การตรวจจับจุดซ่อน การปลดล็อกประตู และการติดตามกล้องในพื้นที่ต่างระดับ

### 3.7.2 การบำรุงรักษาเพื่อปรับตัว

ปรับ Input System ความละเอียด อัตราส่วนภาพ ขอบเขตกล้อง และค่าประสิทธิภาพ เพื่อให้รองรับอุปกรณ์หรือสภาพแวดล้อมที่แตกต่างกัน

### 3.7.3 การบำรุงรักษาเพื่อพัฒนาเพิ่มเติม

เพิ่มชนิดศัตรู ไอเท็ม กับดัก ด่าน และสถานะ FSM ใหม่ได้ผ่านโครงสร้างโมดูล พร้อมปรับความยากของด่านผ่าน LevelConfig โดยไม่เปลี่ยนตรรกะหลัก

### 3.7.4 การบำรุงรักษาเชิงป้องกัน

ทดสอบทุกด่านหลังแก้ไขระบบส่วนกลาง ตรวจสอบ Unity Console ไม่ให้มีข้อผิดพลาด สำรอง Scene และ Prefab ก่อนแก้ไข และอัปเดตแผนภาพ ตาราง รวมถึงภาพหน้าจอในรายงานให้ตรงกับเกมเวอร์ชันล่าสุด

# บทที่ 4 ผลการดำเนินงาน

บทนี้นำเสนอผลการตรวจโครงสร้างและการสาธิตต้นแบบใน Unity Play Mode เมื่อวันที่ 4 ตุลาคม 2569 ภาพเกมจับจากการทำงานจริง
ส่วนภาพเฟรมแอนิเมชันและแผนภาพระบุแหล่งที่มาตามคำบรรยาย
การสาธิตใช้สคริปต์ชั่วคราวควบคุมตำแหน่ง เรียกเมธอดและบางสถานะ หยุดการอัปเดตศัตรูเพื่อจัดภาพ และปิดโอกาสสุ่ม Jumpscare ในช่วงเก็บภาพ จึงไม่ใช่การทดลองเล่นครบวงจรและไม่ยืนยันผลผ่านทุก Test Case ยังไม่มีข้อมูลแบบสอบถามจากผู้ทดลองเล่น

## 4.1 ผลการพัฒนาระบบควบคุมตัวละครผู้เล่น

ผู้พัฒนาได้สร้างระบบควบคุมตัวละครผู้เล่นให้รองรับการเคลื่อนที่ภายในเกมแบบ Side-scrolling ผู้เล่นสามารถเดิน วิ่ง กระโดด ไต่บันได ใช้ไฟฉาย เก็บไอเท็ม ซ่อนตัว เปิดกระเป๋าไอเท็ม และโต้ตอบกับประตูทางออกได้ตามปุ่มควบคุมที่กำหนด การเคลื่อนที่แบบเดินเป็นรูปแบบการเคลื่อนที่พื้นฐาน ส่วนการวิ่งช่วยให้ผู้เล่นเคลื่อนที่เร็วขึ้น แต่ส่งผลให้ค่า Stamina ลดลงและสร้างเสียงที่ศัตรูสามารถตรวจจับได้จากระยะไกลขึ้น การกระโดดถูกควบคุมไม่ให้เกิดการกระโดดซ้ำกลางอากาศ ขณะที่ระบบไต่บันไดช่วยให้ผู้เล่นเดินทางระหว่างพื้นที่ต่างระดับภายในด่านได้ ระบบไฟฉายสามารถเปิดและปิดได้ระหว่างเล่น พร้อมแสดงปริมาณแบตเตอรี่คงเหลือบนหน้าจอ เมื่อเปิดไฟฉายอย่างต่อเนื่อง ปริมาณแบตเตอรี่จะลดลงตามเวลาที่ใช้งาน ทำให้ผู้เล่นต้องพิจารณาการใช้ไฟฉายให้เหมาะสมกับสถานการณ์




ภาพที่ 4-1 ผลการทำงานของระบบควบคุมตัวละครผู้เล่น (ภาพสาธิตใน Play Mode)

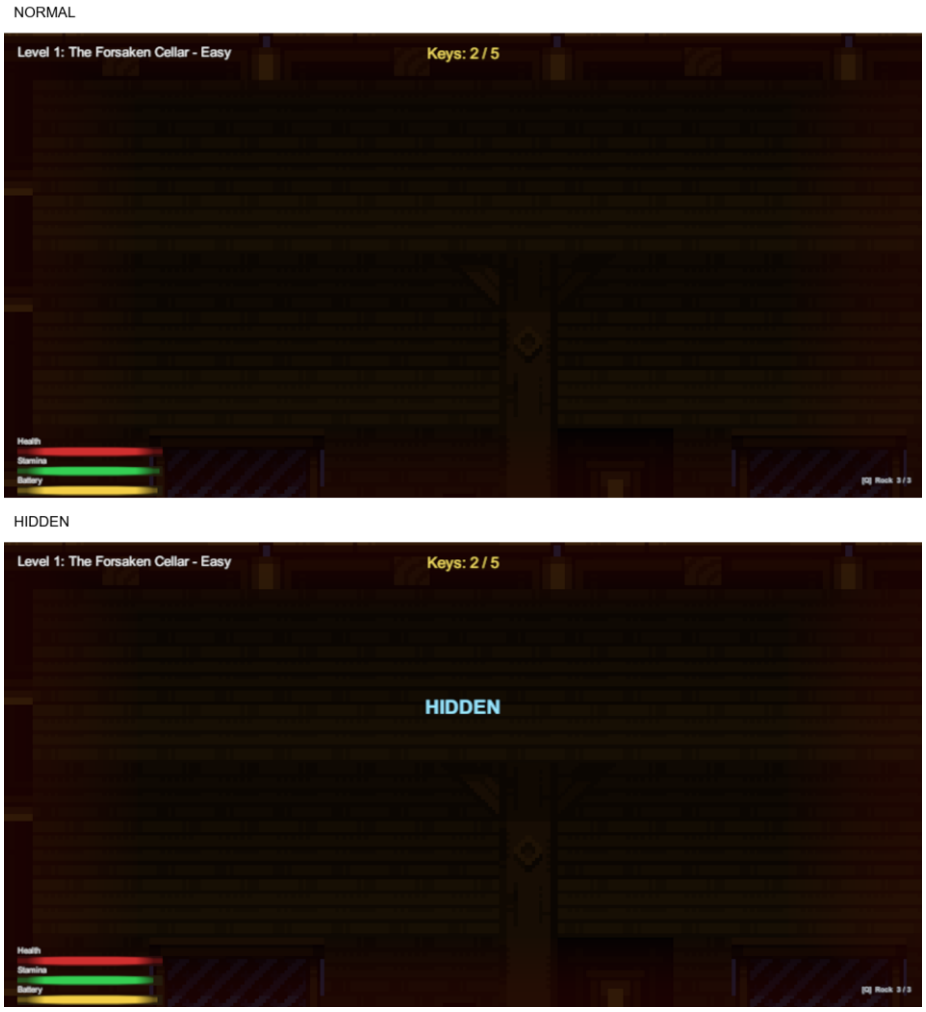



ภาพที่ 4-2 ผลการทำงานของระบบไฟฉาย (ภาพสาธิตใน Play Mode)

## 4.2 ผลการพัฒนา Finite State Machine ของผู้เล่น

ระบบควบคุมผู้เล่นใช้เทคนิค Finite State Machine เพื่อแบ่งพฤติกรรมออกเป็นสถานะที่ชัดเจน ประกอบด้วย Normal, Hidden, Frozen, Hurt, UsingItem, Transitioning และ Dead สถานะ Normal เป็นสถานะหลักที่ผู้เล่นสามารถเดิน วิ่ง กระโดด และรับคำสั่งควบคุมทั่วไปได้ เมื่อผู้เล่นเข้าใช้จุดซ่อน ระบบจะเปลี่ยนเป็นสถานะ Hidden และจำกัดการเคลื่อนไหวบางส่วน หากผู้เล่นได้รับผลที่ทำให้หยุดการเคลื่อนไหว ระบบจะเข้าสู่สถานะ Frozen ชั่วคราว และกลับสู่ Normal เมื่อพ้นระยะเวลาที่กำหนดเมื่อได้รับความเสียหาย ผู้เล่นจะเปลี่ยนเข้าสู่สถานะ Hurt เพื่อป้องกันการรับความเสียหายซ้ำในช่วงเวลาสั้น ๆ หากพลังชีวิตลดลงจนหมด ระบบจะเปลี่ยนเป็นสถานะ Dead และแสดงหน้าจอ Game Over ส่วนสถานะ UsingItem ใช้ระหว่างเปิดกระเป๋าหรือใช้ไอเท็ม และสถานะ Transitioning ใช้ขณะระบบกำลังเปลี่ยนฉาก

บันทึกการสาธิตพบสถานะ Normal, UsingItem, Hidden, Frozen, Hurt และ Dead ตามเมธอดที่เรียก ภาพแสดงผล HIDDEN และ Game Over สอดคล้องกับสถานะที่บันทึก
การสาธิตนี้ยืนยันการแสดงผลภายใต้เงื่อนไขที่จัดไว้
ยังไม่ยืนยันว่าทุกการเปลี่ยนสถานะจะถูกต้องเมื่อหลายเหตุการณ์เกิดพร้อมกัน




ภาพที่ 4-3 สาธิตหน้าจอ Normal และ Hidden ของผู้เล่น

## 4.3 ผลการพัฒนาระบบปัญญาประดิษฐ์ของศัตรู

ระบบปัญญาประดิษฐ์ของศัตรูได้รับการพัฒนาให้ตอบสนองต่อผู้เล่นผ่านการมองเห็นและการได้ยิน โดยมี KillerAI เป็นส่วนควบคุมพฤติกรรมหลัก ศัตรูสามารถลาดตระเวน รับรู้สิ่งผิดปกติ ตรวจสอบตำแหน่งเสียง ค้นหาผู้เล่น ไล่ล่า โจมตี และกลับไปยังเส้นทางลาดตระเวนได้ Enemy FSM ประกอบด้วยสถานะ Patrol, Alert, Listening, InvestigateSound, ConfirmSound, Confused, Investigate, Search, Chase, Attack, AttackRecover, ReturnToPatrol และ Scream สถานะเหล่านี้ทำให้ศัตรูสามารถตอบสนองต่อสถานการณ์ได้ละเอียดกว่าการแบ่งพฤติกรรมเพียงการลาดตระเวนและไล่ล่า ในสถานะ Patrol ศัตรูจะเดินตามเส้นทางลาดตระเวน เมื่อได้ยินเสียงจะเข้าสู่กระบวนการ Listening และ InvestigateSound เพื่อเลือกและเดินทางไปยังตำแหน่งกำเนิดเสียง หากตรวจสอบแล้วยังไม่พบผู้เล่น

ศัตรูอาจเข้าสู่สถานะ Confused, Investigate หรือ Search ก่อนกลับไปลาดตระเวน เมื่อศัตรูตรวจพบผู้เล่น ระบบจะเปลี่ยนเข้าสู่สถานะ Chase และเคลื่อนที่ไล่ตามตำแหน่งของผู้เล่น หากผู้เล่นอยู่ในระยะโจมตี ศัตรูจะเข้าสู่สถานะ Attack หลังจากโจมตีแล้วจะพักตามระยะเวลาที่กำหนดในสถานะ AttackRecover ก่อนตัดสินใจไล่ตามหรือตรวจสอบพื้นที่ต่อไป




ภาพที่ 4-4 Killer ในสถานะ Patrol ที่จัดไว้เพื่อสาธิต




ภาพที่ 4-5 สาธิตการรับเสียง Rock และสถานะ Listening



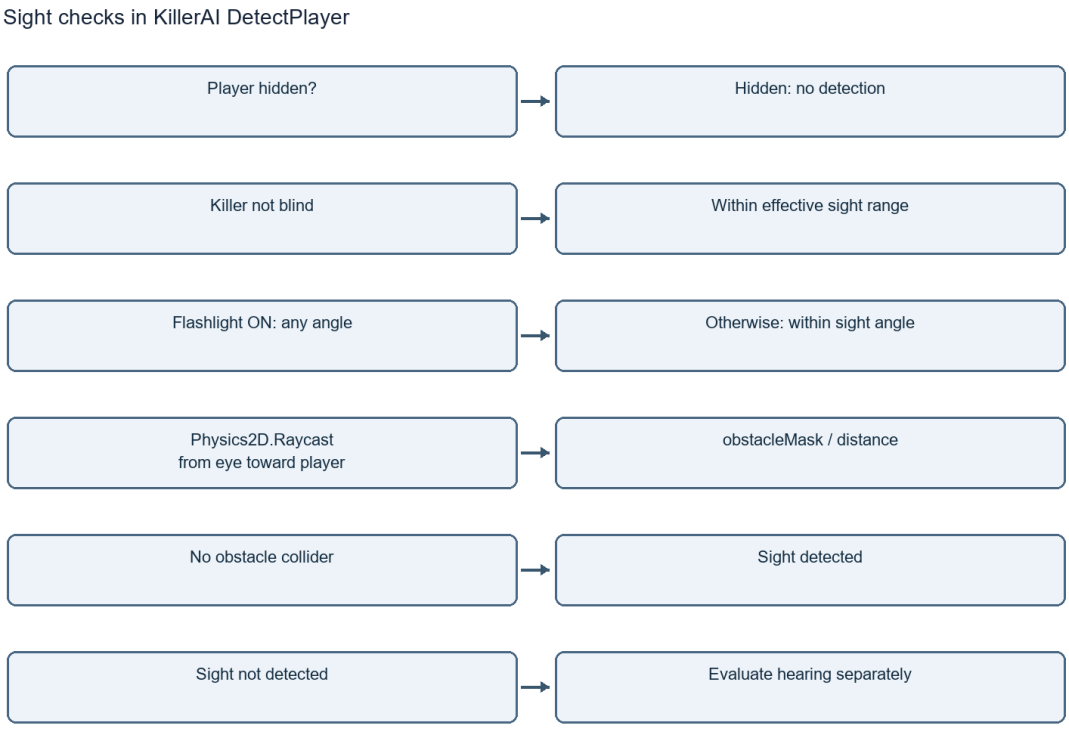

ภาพที่ 4-6 สาธิตสถานะ Chase และข้อความ DANGER

## 4.4 ผลการพัฒนาระบบตรวจจับผู้เล่น

ระบบตรวจจับแบ่งออกเป็นการตรวจจับด้วยการมองเห็นและการตรวจจับด้วยเสียง การมองเห็นใช้ระยะตรวจจับ ทิศทาง และสิ่งกีดขวางภายในฉากเป็นเงื่อนไข ศัตรูจึงไม่สามารถตรวจพบผู้เล่นผ่านวัตถุที่บดบังการมองเห็นได้

ระบบเสียงรับเหตุการณ์จาก FootstepSystem และกิจกรรมที่สร้างเสียงภายในด่าน เช่น การวิ่งและการเหยียบกับดักกระจก เสียงแต่ละชนิดมีความสำคัญแตกต่างกัน โดยเสียง Sprint และ Glass มีผลต่อการรับรู้ของศัตรูมากกว่าเสียง Footstep ทั่วไป

เมื่อศัตรูได้รับเหตุการณ์เสียง ระบบจะบันทึกตำแหน่งกำเนิดเสียงและเปลี่ยนไปยังสถานะที่เกี่ยวข้องเพื่อทำการตรวจสอบ ส่งผลให้ผู้เล่นต้องตัดสินใจว่าจะเคลื่อนที่อย่างรวดเร็วหรือหลีกเลี่ยงการสร้างเสียงเพื่อไม่ให้ศัตรูทราบตำแหน่ง



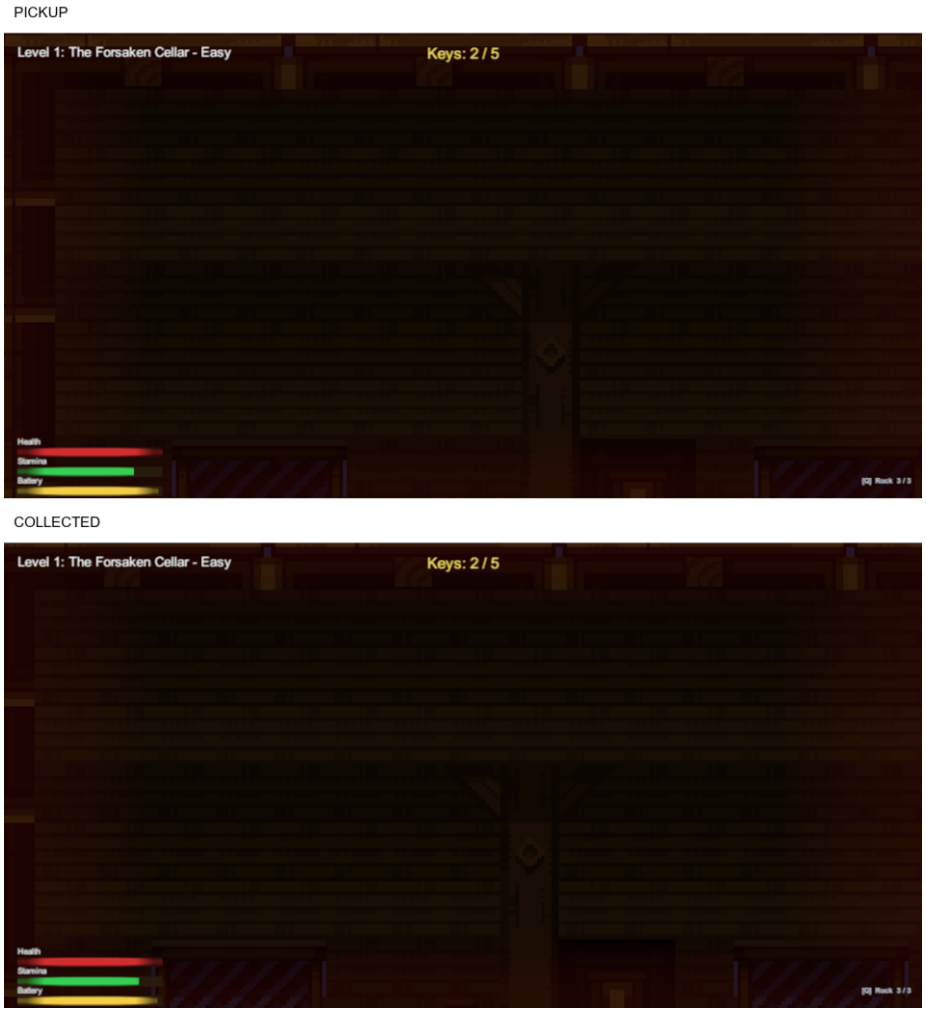

ภาพที่ 4-7 แผนภาพเงื่อนไขการตรวจแนวสายตาจาก KillerAI




ภาพที่ 4-8 สาธิตพื้นที่กับดักเสียงของ Level 3

## 4.5 ผลการพัฒนาระบบไอเท็มและประตูทางออก

ระบบไอเท็มใช้ InventoryManager สำหรับจัดเก็บและนับจำนวนกุญแจที่ผู้เล่นเก็บได้ เมื่อผู้เล่นเก็บกุญแจ จำนวนกุญแจบน HUD และข้อมูลในกระเป๋าไอเท็มจะเปลี่ยนแปลงทันที ประตูทางออกใช้ FSM ควบคุมสถานะ ได้แก่ Locked, Ready, Unlocking, Open, Transitioning และ Completed หากผู้เล่นยังเก็บกุญแจไม่ครบ ประตูจะแสดงจำนวนกุญแจที่ยังขาด เมื่อผู้เล่นเก็บกุญแจครบและกดปุ่ม E ค้าง ระบบจะแสดงความคืบหน้าการปลดล็อก เมื่อดำเนินการสำเร็จ ประตูจะเปิดและโหลดด่านถัดไป หากเป็นด่านสุดท้าย ระบบจะแสดงผลการจบเกม การแยกสถานะของประตูช่วยป้องกันการปลดล็อกซ้ำ และรองรับการยกเลิกเมื่อผู้เล่นปล่อยปุ่ม เดินออกจากพื้นที่ หรือถูกศัตรูโจมตีระหว่างปลดล็อก




ภาพที่ 4-9 ผลการทำงานของระบบเก็บกุญแจ (ภาพสาธิตใน Play Mode)




ภาพที่ 4-10 ผลการทำงานของระบบประตูทางออก (ภาพสาธิตใน Play Mode)

## 4.6 ผลการพัฒนาระบบส่วนติดต่อผู้ใช้งาน

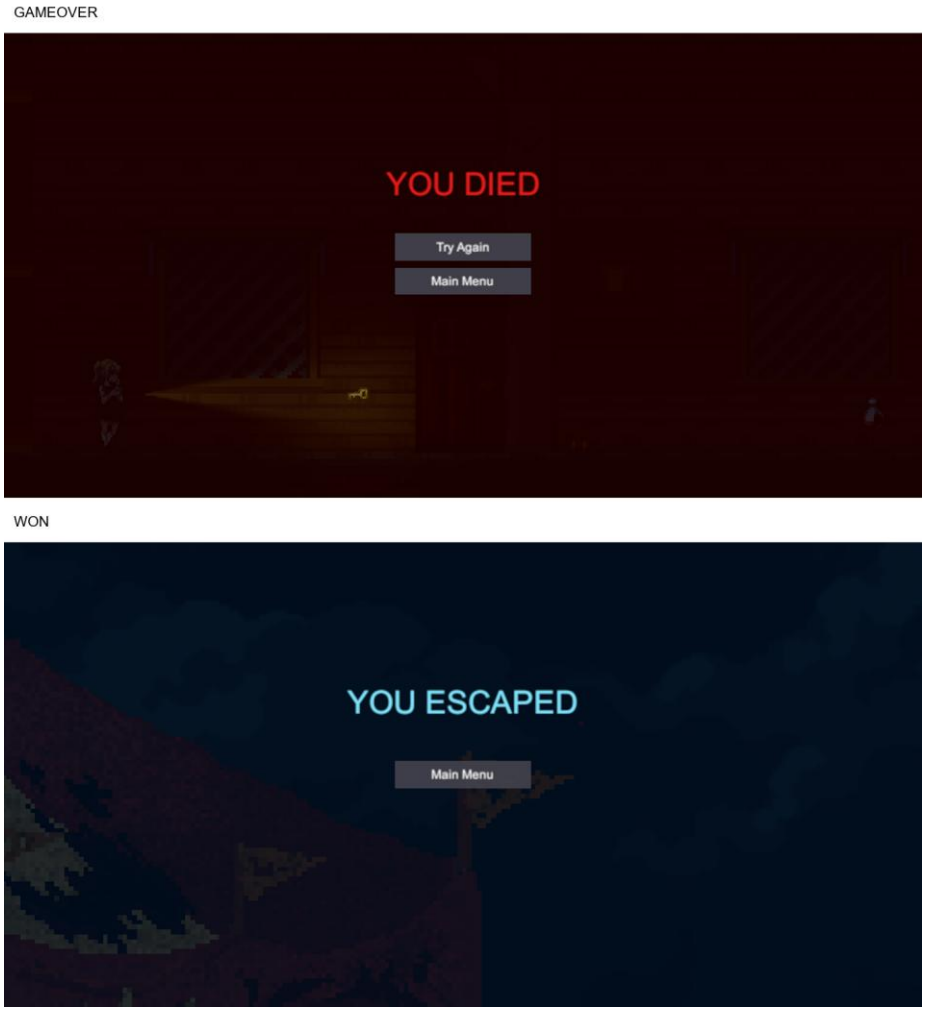
ระบบส่วนติดต่อผู้ใช้งานได้รับการออกแบบให้แสดงข้อมูลที่จำเป็นระหว่างเล่น เช่น พลังชีวิต จำนวนกุญแจ แบตเตอรี่ไฟฉาย Stamina ข้อความโต้ตอบ สถานะการซ่อน และคำเตือนเมื่อศัตรูอยู่ใกล้

เมื่อผู้เล่นเข้าใกล้วัตถุที่สามารถโต้ตอบได้ ระบบจะแสดงข้อความแนะนำปุ่มควบคุม เช่น การเก็บไอเท็ม การเข้าจุดซ่อน และการปลดล็อกประตู ข้อความจะเปลี่ยนไปตามสถานการณ์และหายไปเมื่อผู้เล่นออกจากพื้นที่โต้ตอบ

เมื่อศัตรูตรวจพบหรือไล่ล่าผู้เล่น HUD จะแสดงคำเตือนและเอฟเฟกต์ภาพตามระดับอันตราย หากผู้เล่นเสียชีวิต ระบบจะแสดงหน้า Game Over แต่หากผ่านด่านสำเร็จจะแสดง Level Complete หรือ Game Won ตามสถานะของเกม




ภาพที่ 4-11 ผลการแสดงข้อมูลบนหน้าจอระหว่างเล่น (ภาพสาธิตใน Play Mode)

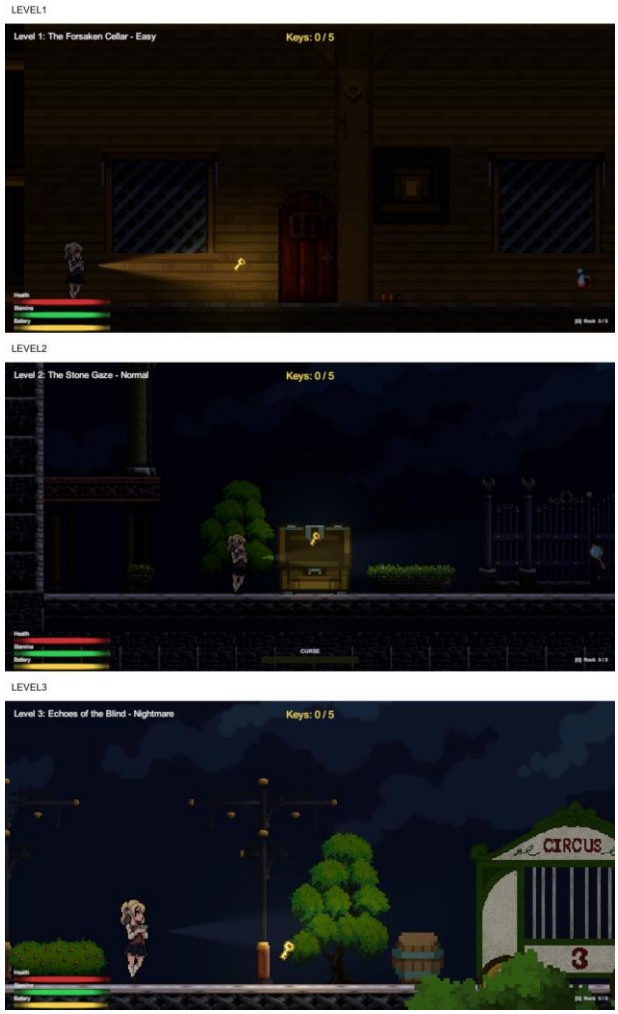



ภาพที่ 4-12 ผลการแสดงหน้าจอสรุปสถานะเกม (ภาพสาธิตใน Play Mode)

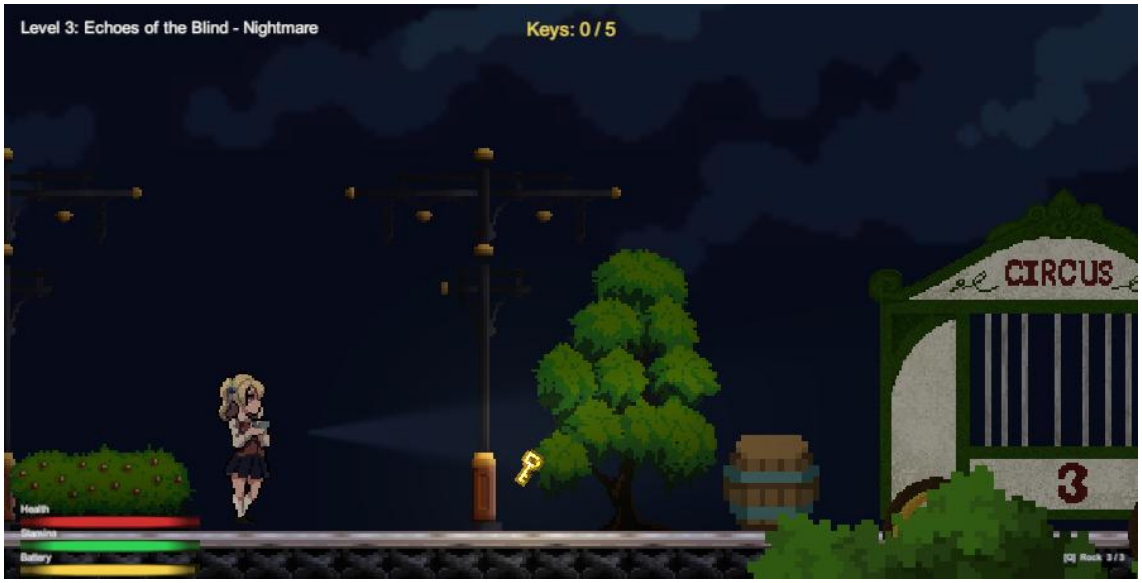
## 4.7 ผลการพัฒนาด่านภายในเกม

เกมประกอบด้วยด่านที่มีการกำหนดระดับความยากแตกต่างกันผ่าน LevelConfig โดยแต่ละด่านสามารถปรับค่าการทำงาน เช่น ระยะมองเห็น ระยะได้ยิน ความเร็วของศัตรู ความเสียหาย และปริมาณแบตเตอรี่เริ่มต้น Level 1 เป็นด่านเริ่มต้นที่ช่วยให้ผู้เล่นเรียนรู้ระบบพื้นฐาน ศัตรูมีความเร็วและความสามารถในการตรวจจับต่ำกว่าด่านอื่น Level 2 เพิ่มความกดดันจากระบบคำสาปและลดพื้นที่การตรวจจับของศัตรูบางส่วนเพื่อให้เหมาะกับโครงสร้างของด่านLevel 3 ได้รับการออกแบบเป็นสวนสนุกเพื่อสร้างบรรยากาศน่ากลัว ภายในด่านมีเส้นทางต่างระดับ กุญแจ กับดักกระจก และจุดซ่อนที่กระจายตามตำแหน่งที่สามารถเดินได้ ระบบ Level3RandomEvents ทำให้ตำแหน่งวัตถุบางส่วนเปลี่ยนแปลงเมื่อเริ่มเล่นใหม่ จึงช่วยลดการจดจำตำแหน่งจากการเล่นรอบก่อนLevel 3 กำหนดให้ศัตรูเน้นการ

ตรวจจับด้วยเสียง โดยมีระยะได้ยินและความเร็วไล่ล่าสูง เมื่อผู้เล่นสร้างเสียงหรือเข้าใกล้ศัตรู ระบบจะแสดงขอบจอสีแดงและเล่นเสียงหัวใจตามระดับอันตราย ส่งผลให้ด่านมีบรรยากาศกดดันและแตกต่างจากด่านก่อนหน้า




ภาพที่ 4-13 ผลการพัฒนาด่านภายในเกม (ภาพสาธิตใน Play Mode)




ภาพที่ 4-14 สวนสนุกและพื้นที่ป่าใน Level 3 (ภาพสาธิตใน Play Mode)

## 4.8 ผลการพัฒนาระบบความตึงเครียด

TensionDirector รวมสถานะของศัตรูเป็น Calm, Suspicious, Hunted, Escaped และ Cooldown แล้วส่งเหตุการณ์ให้ LevelAudioController ปรับดนตรี ส่วน UIManager แสดง DANGER และ Level3HeartbeatWarning ควบคุมคำเตือนตามระยะห่าง
ภาพเปรียบเทียบแสดงหน้าจอปกติกับหน้าจอที่เรียก Chase เพื่อสาธิต ไม่ได้จับหน้าต่าง Debug ของ TensionDirector จึงยังไม่ใช้ภาพยืนยันลำดับของทั้งห้าสถานะครบถ้วน

สิ่งที่เห็นจากภาพคือข้อความ DANGER และขอบจอสีแดงตามเงื่อนไขที่จัดไว้ ส่วนความชัดเจนของเสียงและความเหมาะสมของระดับความกดดันต้องตรวจด้วยการฟังและทดลองเล่นเพิ่มเติม




ภาพที่ 4-15 เปรียบเทียบหน้าจอปกติกับคำเตือนเมื่อเรียก Chase (ภาพสาธิตใน Play Mode)

## 4.9 ผลการพัฒนาระบบแอนิเมชันของตัวละคร

ผลการพัฒนาในส่วนนี้ต่อเนื่องจากการออกแบบ Animator ในหัวข้อ 3.5.8–3.5.11 โดยตรวจสอบไฟล์ Animation Clip, Animator Controller และสคริปต์ที่เชื่อมต่อกัน ภาพประกอบเป็นเฟรมจาก Sprite Sheet ที่ใช้ในโครงการ มิใช่ภาพหน้าจอขณะทดลองเล่น จึงใช้แสดงลักษณะของท่าทางและผลการจัดเตรียมคลิป ส่วนผลการตอบสนองระหว่างเล่นต้องยืนยันด้วยการทดสอบเพิ่มเติม

### 4.9.1 ผลการจัดทำแอนิเมชันของผู้เล่น



ภาพที่ 4-16 ตัวอย่างสี่เฟรมแรกของแอนิเมชัน Idle ของผู้เล่น



ภาพที่ 4-17 ตัวอย่างสี่เฟรมแรกของแอนิเมชัน Walk ของผู้เล่น

ภาพที่ 4-16 และ 4-17 แสดงท่ายืนและท่าเดินที่นำไปใช้กับ PlayerIdle.anim และ PlayerWalk.anim ตามลำดับ ทั้งสองคลิปกำหนด Sample Rate เท่ากับ 12 และเปิด Loop Time เพื่อเล่นวนขณะตัวละครคงอยู่ในสถานะเดิม โดยคลิป Idle มีช่วงเวลา 0.75 วินาที ส่วน Walk มีช่วงเวลา 1 วินาที

จากการตรวจสอบโครงสร้าง PlayerAnimator พบสถานะ Idle และ Walk เชื่อมด้วย isWalking ชนิด Bool และ PlayerController มีคำสั่ง SetBool(IsWalkingHash, IsMoving) ตรงกับชื่อพารามิเตอร์ ผลที่ยืนยันได้คือมีคลิป สถานะ และจุดส่งค่าจากสคริปต์ครบสำหรับการเลือกท่ายืนและเดิน ส่วนความถูกต้องของภาพที่แสดงเมื่อกดหรือปล่อยปุ่มยังต้องตรวจใน Play Mode

Controller ของผู้เล่นในรุ่นที่ตรวจสอบยังไม่มีสถานะ Run แยกต่างหาก จึงรายงานเฉพาะผลการจัดทำ Idle และ Walk การเพิ่มความเร็ววิ่งในระบบควบคุมไม่ได้เป็นหลักฐานว่ามีแอนิเมชัน Run เพิ่มขึ้นด้วย

### 4.9.2 ผลการจัดทำแอนิเมชันของ Killer

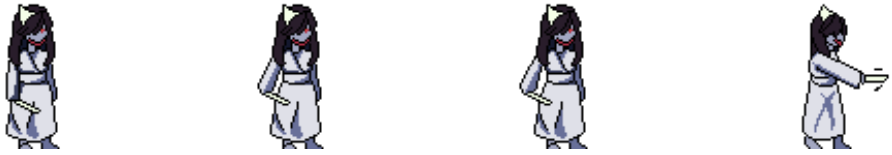
KillerAnimator มีสถานะ Idle, Walk, Run, Attack1 และ Scream โดยเชื่อมกับไฟล์ KillerIdle.anim, KillerWalk.anim, KillerRun.anim, KillerAttack1.anim และ KillerScream.anim ภาพต่อไปนี้แสดงเฟรมต้นคลิปจากไฟล์ภาพที่ใช้กับศัตรู เพื่อให้เห็นความแตกต่างระหว่างการเคลื่อนที่ การโจมตี และการกรีดร้อง



ภาพที่ 4-18 ตัวอย่างสี่เฟรมแรกของแอนิเมชัน Walk ของ Killer



ภาพที่ 4-19 ตัวอย่างสี่เฟรมแรกของแอนิเมชัน Run ของ Killer



ภาพที่ 4-20 ตัวอย่างสี่เฟรมแรกของแอนิเมชัน Attack1 ของ Killer



ภาพที่ 4-21 ตัวอย่างสี่เฟรมแรกของแอนิเมชัน Scream ของ Killer

ผลการตรวจไฟล์คลิปพบว่า Idle, Walk และ Run เปิด Loop Time เพื่อแสดงท่าทางต่อเนื่อง ส่วน Attack1 และ Scream ปิด Loop Time เพื่อให้เล่นเป็นครั้งตามเหตุการณ์ การกลับสู่ Idle หลังสองท่านี้กำหนดด้วย Transition ที่เปิด Has Exit Time และตั้ง Exit Time เท่ากับ 1 ตามแผนผังในภาพที่ 3-23

### 4.9.3 ผลการตรวจสอบการเชื่อมต่อและกรณีทดสอบแอนิเมชัน

การตรวจสอบไฟล์ Controller และสคริปต์พบว่า isWalking และ isRunning ของ Killer เป็น Bool และมีคำสั่ง SetBool ใน KillerAI ส่วน attack1 และ scream เป็น Trigger และมีคำสั่ง SetTrigger ที่ใช้เรียกเหตุการณ์โจมตีและกรีดร้อง ชื่อพารามิเตอร์ที่สคริปต์เรียกตรงกับรายการใน Controller จึงยืนยันการเชื่อมต่อได้ในระดับโครงสร้างไฟล์

Bool ทำหน้าที่คงสถานะการเคลื่อนที่ ขณะที่ Trigger ใช้เริ่มการกระทำเฉพาะครั้ง ผลการตรวจสอบนี้สอดคล้องกับเหตุผลการเลือกชนิดพารามิเตอร์ในหัวข้อ 3.5.11 ไม่พบ Float หรือ Int ใน Animator ทั้งสองตัว จึงไม่มีผลทดสอบการทำงานของพารามิเตอร์สองชนิดนี้ในรุ่นปัจจุบัน

ได้ทดสอบ Animator ใน Play Mode ตามกรณีต่อไปนี้ โดยปรับพารามิเตอร์ของ Controller เดิมและตรวจชื่อสถานะที่ทำงานจริง ผลทั้ง 5 กรณีผ่านตามตารางที่ 4-8

AN-P01 การเริ่มต้นและหยุดเคลื่อนที่ของผู้เล่น

เริ่มฉากโดยไม่กดปุ่มเคลื่อนที่ แล้วตรวจว่า Idle ทำงานและ isWalking เป็น false จากนั้นเดินและปล่อยปุ่ม ผลที่คาดหวังคือเปลี่ยนไป Walk เมื่อเป็น true และกลับ Idle เมื่อเป็น false โดยภาพไม่ค้าง

AN-K01 การเดินลาดตระเวนและการไล่ล่า

ให้ Killer เดินลาดตระเวนแล้วตรวจ isWalking จากนั้นกระตุ้นให้เข้าสู่ Chase ผลที่คาดหวังคือใช้ Walk ขณะเดิน และเปลี่ยนเป็น Run เมื่อ isRunning เป็น true

AN-K02 การโจมตีและกลับสู่สถานะเคลื่อนที่

ให้ผู้เล่นอยู่ในระยะโจมตีจน KillerAI ส่ง attack1 ผลที่คาดหวังคือเล่น Attack1 ตามเหตุการณ์ แล้วกลับ Idle ตาม Exit Time ก่อนเลือกสถานะเคลื่อนที่ตามค่าพารามิเตอร์ โดยบันทึกทั้งภาพและความเสียหายที่เกิดขึ้น

AN-K03 การกรีดร้องจากสถานะอื่น

ใช้ศัตรูหรือเงื่อนไขที่เปิดความสามารถกรีดร้อง แล้วกระตุ้นการส่ง scream ผลที่คาดหวังคือเปลี่ยนจากสถานะปัจจุบันผ่าน Any State ไป Scream และกลับ Idle หลังคลิปถึง Exit Time

AN-K04 การหยุดไล่ล่าและกลับไปเดิน

ทำให้ Killer ออกจาก Chase และกลับไปเดิน ผลที่คาดหวังคือหยุด Run และแสดงท่าที่สัมพันธ์กับพฤติกรรม โดยตรวจลำดับ Transition เมื่อ isRunning เป็น false และ isWalking เป็น true พร้อมกัน

ตารางที่ 4-8 ผลตรวจโครงสร้างและสถานะทดสอบแอนิเมชัน

| รหัส | ผลตรวจโครงสร้าง | สถานะทดสอบจริง |
|---|---|---|
| AN-P01 | พบ Idle, Walk และ Bool isWalking เชื่อมกับสคริปต์ | ผ่าน: isWalking false ⟶ true ⟶ false ทำให้ Idle ⟶ Walk ⟶ Idle |
| AN-K01 | พบ Walk, Run และ Bool isWalking กับ isRunning | ผ่าน: isWalking true แสดง Walk และ isRunning true เปลี่ยนเป็น Run |
| AN-K02 | พบ Trigger attack1 และ Transition ของ Attack1 | ผ่าน: Trigger attack1 เข้า Attack1 และกลับ Idle หลังคลิป |
| AN-K03 | พบ Trigger scream และ Any State ไป Scream | ผ่าน: Trigger scream จาก Idle, Walk, Run และ Attack1 เข้า Scream และกลับ Idle ได้ทุกกรณี |
| AN-K04 | พบ Transition ออกจาก Run ตาม Bool | ผ่าน: ขณะ Run และ Bool ทั้งสองเป็น true เมื่อ isRunning เป็น false ภาพกลับ Walk |

## 4.10 ผลการทดสอบการทำงานของระบบ

ทดสอบวันที่ 4 ตุลาคม 2569 ด้วย Unity 6000.4.9f1 ใน Play Mode
โดยใช้สคริปต์ชั่วคราวควบคุมตำแหน่ง อินพุต เวลา และเงื่อนไขขององค์ประกอบ แล้วตรวจสถานะ ค่าความเร็ว HP ทรัพยากร และ Animator จากการทำงานจริง สถานะ “ผ่าน”
หมายถึงผ่านกรณีควบคุมที่บันทึกไว้
ไม่ใช่การรับรองว่าการเล่นครบทุกเส้นทางหรือการทดสอบโดยผู้ใช้งานผ่านทั้งหมด

ตารางที่ 4-1 ผลการทดสอบระบบควบคุมพื้นฐาน

| รหัส | รายการทดสอบ | ผลที่คาดหวัง | ผลจริง | สถานะ |
|---|---|---|---|---|
| TC-01 | การเดิน | ผู้เล่นเคลื่อนที่และหันถูกทิศ | กด D ได้ความเร็ว 4.5 และหันขวา กด A ได้ −4.5 และหันซ้าย | ผ่าน |
| TC-02 | การวิ่ง | ความเร็วเพิ่ม Stamina ลด และเกิดเสียง Sprint | ความเร็วเดิน 4.5 เพิ่มเป็น 5.7 ขณะวิ่ง ค่า Stamina ลด และส่งเสียงประเภท Sprint | ผ่าน |
| TC-03 | การกระโดด | กระโดดได้ครั้งเดียวและไม่กระโดดซ้ำกลางอากาศ | กระโดดได้แรงยกครั้งแรก การสั่งกระโดดซ้ำทันทีขณะลอยไม่เพิ่มแรงยก | ผ่าน |
| TC-04 | การไต่บันได | ผู้เล่นขึ้นลงบันไดและกระโดดออกได้ | ขึ้นบันไดด้วยความเร็ว 4 และลงด้วย −4 กระโดดออกได้และคืนค่าแรงโน้มถ่วง | ผ่าน |
| TC-05 | ไฟฉาย | ไฟฉายเปิดปิดได้และแบตเตอรี่ลดลง | เปิดปิดได้ แบตเตอรี่ลดขณะเปิด เมื่อเหลือศูนย์ไฟดับและเปิดซ้ำไม่ได้ | ผ่าน |

ตารางที่ 4-2 ผลการทดสอบ Player FSM

| รหัส | การเปลี่ยนสถานะ | ผลที่คาดหวัง | ผลจริง | สถานะ |
|---|---|---|---|---|
| TC-FSM-P01 | Normal ⟶ Hidden | ผู้เล่นซ่อนและ HUD แสดง HIDDEN | ToggleHide เปลี่ยนเป็น Hidden และล็อกการเคลื่อนไหว | ผ่าน |
| TC-FSM-P02 | Hidden ⟶ Normal | ผู้เล่นออกจากจุดซ่อนและเคลื่อนที่ได้ | ToggleHide ออกจากที่ซ่อน กลับ Normal และคืนการควบคุม | ผ่าน |
| TC-FSM-P03 | Normal ⟶ Hurt ⟶ Normal | HP ลดและกลับสู่ Normal หลังช่วง Hurt | รับความเสียหาย 10 ทำให้ HP 100 เป็น 90 เข้า Hurt และกลับ Normal หลังรอ 2.2 วินาที | ผ่าน |
| TC-FSM-P04 | Normal ⟶ Frozen ⟶ Normal | ผู้เล่นหยุดเคลื่อนไหวชั่วคราวและกลับมาควบคุมได้ | Freeze เปลี่ยนเป็น Frozen กด X ครบ 2 ครั้งตามค่าทดสอบแล้วกลับ Normal | ผ่าน |
| TC-FSM-P05 | Normal ⟶ UsingItem ⟶ Normal | การควบคุมถูกล็อกระหว่างใช้ไอเท็มและคืนค่าถูกต้อง | เปิดสถานะคลังเป็น UsingItem และล็อกการเคลื่อนไหว ปิดแล้วกลับ Normal | ผ่าน |

| รหัส | การเปลี่ยนสถานะ | ผลที่คาดหวัง | ผลจริง | สถานะ |
|---|---|---|---|---|
| TC-FSM-P06 | Normal ⟶ Transitioning | ล็อกการควบคุมระหว่างเปลี่ยนฉาก | BeginTransition ล็อกคำสั่ง และเมื่อประตูสำเร็จผู้เล่นเข้าสู่ Transitioning | ผ่าน |
| TC-FSM-P07 | Hurt ⟶ Dead | แสดงสถานะ Dead และหน้า Game Over | รับความเสียหายจน HP หมด ผู้เล่นเป็น Dead และสถานะเกมเป็น Dead | ผ่าน |

ตารางที่ 4-3 ผลการทดสอบ Enemy FSM

| รหัส | การเปลี่ยนสถานะ | ผลที่คาดหวัง | ผลจริง | สถานะ |
|---|---|---|---|---|
| TC-FSM-E01 | Patrol ⟶ Listening | ศัตรูหยุดลาดตระเวนและรับฟังเสียง | ส่งเสียง Rock ในระยะได้ยิน ทำให้ Patrol เปลี่ยนเป็น Listening | ผ่าน |
| TC-FSM-E02 | Listening ⟶ InvestigateSound | ศัตรูเคลื่อนที่ไปยังต้นกำเนิดเสียง | หมดเวลาฟังแล้วเข้า InvestigateSound และมีความเร็วเคลื่อนที่ไปยังเสียง | ผ่าน |
| TC-FSM-E03 | InvestigateSound ⟶ ConfirmSound | ศัตรูตรวจสอบบริเวณที่เกิดเสียง | เมื่อตำแหน่งถึงต้นกำเนิดเสียง เปลี่ยนเป็น ConfirmSound | ผ่าน |
| TC-FSM-E04 | Patrol/Investigate ⟶ Chase | ศัตรูเริ่มไล่ล่าและ HUD แสดง DANGER | ผู้เล่นอยู่ 2 หน่วยด้านที่ Killer หันและไม่มีสิ่งกีดขวาง ตรวจพบจริงแล้วเข้า Chase | ผ่าน |
| TC-FSM-E05 | Chase ⟶ Attack | ศัตรูโจมตีและลด HP ของผู้เล่น | ผู้เล่นอยู่ในระยะโจมตี Chase เข้า Attack จากนั้น HP 100 ลดเป็น 75 และเข้า AttackRecover | ผ่าน |
| TC-FSM-E06 | Attack ⟶ AttackRecover | ศัตรูหยุดพักตามเวลาหลังการโจมตี | คง AttackRecover ก่อนเวลาสิ้นสุด และกลับ Chase เมื่อครบเวลาขณะที่ยังตรวจพบเป้าหมาย | ผ่าน |
| TC-FSM-E07 | Chase ⟶ Investigate/Search | ศัตรูค้นหาตำแหน่งล่าสุดของผู้เล่น | สูญเสียเป้าหมายหลังช่วงความจำแล้วเข้า Investigate เมื่อหมดเวลาตรวจสอบเข้า Search | ผ่าน |
| TC-FSM-E08 | Search ⟶ ReturnToPatrol | ศัตรูกลับสู่เส้นทางลาดตระเวน | หมดเวลาค้นหาแล้วเปลี่ยนจาก Search เป็น ReturnToPatrol | ผ่าน |

| รหัส | การเปลี่ยนสถานะ | ผลที่คาดหวัง | ผลจริง | สถานะ |
|---|---|---|---|---|
| TC-FSM-E09 | Listening ⟶ Confused | ศัตรูเข้าสู่ Confused ตามเงื่อนไข | ส่ง Rock ซ้ำจุดเดิม 3 ครั้ง เมื่อยืนยันเสียงแล้วเข้า Confused | ผ่าน |
| TC-FSM-E10 | สถานะทั่วไป ⟶ Scream | ผู้เล่นได้รับผล Stun หรือ Frozen | เรียกเหตุการณ์กรีดร้องแล้ว Killer เข้า Scream และผู้เล่นถูก Stun จนควบคุมไม่ได้ชั่วคราว | ผ่าน |

ตารางที่ 4-4 ผลการทดสอบระบบเกมและ Level 3

| รหัส | รายการทดสอบ | ผลที่คาดหวัง | ผลจริง | สถานะ |
|---|---|---|---|---|
| TC-06 | เก็บกุญแจ | จำนวนกุญแจและ Inventory เปลี่ยนแปลง | Collect เพิ่มกุญแจจาก 0 เป็น 1 และเพิ่มรายการใน Inventory | ผ่าน |
| TC-07 | ใช้ประตูก่อนกุญแจครบ | แสดงจำนวนกุญแจที่ยังขาด | ขณะมีกุญแจ 1 จาก 5 ประตูยัง Locked และแจ้งว่าขาดอีก 4 ดอก | ผ่าน |
| TC-08 | ปลดล็อกประตู | ความคืบหน้าครบและผ่านด่าน | กด E ค้างตามระยะเวลาปลดล็อก ประตูเป็น Completed และเกมเป็น LevelComplete | ผ่าน |
| TC-09 | ยกเลิกการปลดล็อก | ความคืบหน้าถูกยกเลิก | ยกเลิกและคืนความคืบหน้าเป็นศูนย์เมื่อปล่อย E ออกจากระยะรับความเสียหาย หรือ Killer อยู่ใกล้ | ผ่าน |
| TC-10 | กับดักกระจก | เกิดเสียง Glass และศัตรูเข้าตรวจสอบ | กับดักทำงาน ปิด Collider ส่ง Glass จน Killer เข้า Listening และ AudioSource เล่นเสียงที่มีข้อมูลเสียงจริง | ผ่าน |
| TC-11 | การสุ่มวัตถุ | กุญแจและกับดักกระจายตามจุดที่เดินได้ | สุ่มตำแหน่ง 5 รอบ ตำแหน่งกุญแจเปลี่ยน และ Raycast พบพื้น Ground ใต้กุญแจ | ผ่าน |
| TC-12 | เสียงหัวใจ | ความถี่เสียงและขอบจอเพิ่มตามระดับอันตราย | ความเข้มเตือนเพิ่มจาก 0 เป็นประมาณ 0.929 เมื่อเข้าใกล้ ขอบจอมีค่า Alpha และคลิปหัวใจมีข้อมูลเสียง | ผ่าน |

| รหัส | รายการทดสอบ | ผลที่คาดหวัง | ผลจริง | สถานะ |
|---|---|---|---|---|
| TC-13 | ผ่าน Level 3 | ระบบแสดง Game Won | แจ้งผ่านด่านสุดท้ายและสั่งดำเนินต่อแล้วสถานะเกมเป็น GameWon | ผ่าน |

ตารางที่ 4-9 สรุปผลทดสอบและสถานะการประเมิน

| กลุ่มรายการ | ผลที่ตรวจได้ | สถานะ |
|---|---|---|
| การควบคุมและ Player FSM | TC-01–TC-05 และ TC-FSM-P01–P07 รวม 12 กรณี | ผ่านกรณีควบคุม 12/12 |
| Enemy FSM | TC-FSM-E01–E10 รวม 10 กรณี | ผ่านกรณีควบคุม 10/10 |
| ระบบเกมและด่าน | TC-06–TC-13 รวม 8 กรณี | ผ่านกรณีควบคุม 8/8 |
| แอนิเมชัน | AN-P01 และ AN-K01–AN-K04 รวม 5 กรณี | ผ่านกรณีควบคุม 5/5 |
| ผู้ทดลองเล่น | ยังไม่ได้เก็บแบบสอบถามหรือข้อเสนอแนะจริง | ยังไม่ประเมิน; คะแนนเป็นตัวอย่างสมมติ |

## 4.11 ผลการประเมินโดยผู้ใช้งาน

กำหนดการประเมิน 6 ด้านด้วยคะแนน 1–5 โดย 5 หมายถึงดีมาก 4 ดี 3 ปานกลาง 2 น้อย และ 1 น้อยมาก ณ วันที่จัดทำรายงานยังไม่ได้เก็บแบบสอบถาม จึงไม่มีจำนวนผู้ตอบจริง ค่าเฉลี่ยส่วนเบี่ยงเบนมาตรฐาน หรือข้อเสนอแนะจากผู้ใช้ ตารางนี้แสดงค่าเฉลี่ยตัวอย่างจากข้อมูลสมมติ 5 แถวควบคู่กับสถานะของข้อมูลจริง ตัวเลขใช้สาธิตการคำนวณเท่านั้น รายละเอียดแสดงในหัวข้อ 4.11.2

ตารางที่ 4-5 สถานะข้อมูลการประเมินจากผู้ใช้งานจริง

| รายการประเมิน | ค่าเฉลี่ยสมมติ | ระดับสมมติและสถานะจริง |
|---|---|---|
| ความสะดวกในการควบคุมตัวละคร | 4.40 (ตัวอย่าง) | ดีมาก (สมมติ)<br>ผลจริง: ยังไม่เก็บแบบสอบถาม |
| ความชัดเจนของ UI และข้อความแนะนำ | 4.20 (ตัวอย่าง) | ดี (สมมติ)<br>ผลจริง: ยังไม่เก็บแบบสอบถาม |
| ความเหมาะสมของพฤติกรรมศัตรู | 4.00 (ตัวอย่าง) | ดี (สมมติ)<br>ผลจริง: ยังไม่เก็บแบบสอบถาม |
| ความสมดุลของระดับความยาก | 3.60 (ตัวอย่าง) | ดี (สมมติ)<br>ผลจริง: ยังไม่เก็บแบบสอบถาม |
| ความน่ากลัวและบรรยากาศของเกม | 4.40 (ตัวอย่าง) | ดีมาก (สมมติ)<br>ผลจริง: ยังไม่เก็บแบบสอบถาม |
| ความพึงพอใจโดยรวม | 4.40 (ตัวอย่าง) | ดีมาก (สมมติ)<br>ผลจริง: ยังไม่เก็บแบบสอบถาม |

ยังไม่สามารถจัดอันดับด้านที่ได้รับคะแนนสูงสุดหรือสรุประดับความพึงพอใจจริงได้ หลังเก็บข้อมูลควรคำนวณค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐานรายด้าน ระบุจำนวนผู้ตอบและช่วงเวลาทดสอบ แล้วปรับบทอภิปรายให้สอดคล้องกับข้อมูลที่ได้ ตัวเลขในหัวข้อ 4.11.2 เป็นตัวอย่างสมมติและไม่ใช้แทนผลส่วนนี้

### 4.11.2 ตัวอย่างคะแนนสมมติสำหรับแสดงวิธีคำนวณ

ข้อมูลทั้งหมดในหัวข้อนี้สร้างขึ้นเพื่อเป็นตัวอย่าง ไม่ได้มาจากผู้ทดลองเล่นจริง รหัส S1–S5 หมายถึงแถวข้อมูลสมมติ 5 แถว ไม่ใช่จำนวนกลุ่มตัวอย่างจริง จึงห้ามนำค่าเฉลี่ยหรือส่วนเบี่ยงเบนมาตรฐานนี้ไปอ้างว่าเป็นผลความพึงพอใจของเกม

ตารางที่ 4-6 คะแนนสมมติสำหรับตัวอย่างการคำนวณเท่านั้น

| รหัสสมมติ | ควบคุม | UI | ศัตรู | ความยาก | บรรยากาศ | โดยรวม |
|---|---|---|---|---|---|---|
| S1 | 4 | 4 | 4 | 3 | 5 | 4 |
| S2 | 5 | 4 | 4 | 4 | 4 | 4 |
| S3 | 4 | 5 | 3 | 4 | 5 | 5 |
| S4 | 4 | 4 | 4 | 3 | 4 | 4 |
| S5 | 5 | 4 | 5 | 4 | 4 | 5 |

ค่าเฉลี่ย $\bar{x} = \Sigma x_i / n$ และส่วนเบี่ยงเบนมาตรฐานของตัวอย่าง $s = \sqrt{[\Sigma(x_i - \bar{x})^2 / (n - 1)]}$ โดย n ในตัวอย่างนี้คือจำนวนแถวสมมติ สำหรับด้านควบคุม ผลรวมเท่ากับ 22 จึงได้ค่าเฉลี่ย 22/5 = 4.40 และ s = 0.55 เมื่อปัดสองตำแหน่ง

ตารางที่ 4-7 ผลคำนวณจากคะแนนสมมติเท่านั้น

| ด้านสมมติ | ค่าเฉลี่ย | s | ระดับตัวอย่าง |
|---|---|---|---|
| การควบคุม | 4.40 | 0.55 | ดีมาก |
| UI | 4.20 | 0.45 | ดี |
| พฤติกรรมศัตรู | 4.00 | 0.71 | ดี |
| ความยาก | 3.60 | 0.55 | ดี |
| บรรยากาศ | 4.40 | 0.55 | ดีมาก |
| ความพึงพอใจโดยรวม | 4.40 | 0.55 | ดีมาก |

เกณฑ์ตัวอย่างแบ่งค่าเฉลี่ยเป็น 4.21–5.00 ดีมาก, 3.41–4.20 ดี, 2.61–3.40 ปานกลาง, 1.81–2.60 น้อย และ 1.00–1.80 น้อยมาก ในข้อมูลสมมตินี้บรรยากาศกับความพึงพอใจโดยรวมมีค่าเฉลี่ย 4.40 และด้านความยาก 3.60 เป็นเพียงการฝึกแปลผล ยังไม่ใช่ข้อค้นพบจากเกมหรือผู้ใช้งานจริง

## 4.12 สรุปผลการดำเนินงาน

มีต้นแบบเกมสามด่าน พร้อมสคริปต์ควบคุมผู้เล่น Killer ไอเท็ม ประตู และส่วนติดต่อผู้ใช้ การสาธิตแสดงหน้าจอเมนู การเคลื่อนที่ คลังไอเท็ม สถานะซ่อน คำเตือน และหน้าจอผลลัพธ์ภายใต้การจัดเงื่อนไข การตรวจไฟล์ยืนยันโครงสร้าง FSM และการเชื่อม Animator ผลทดสอบองค์ประกอบ 35 กรณีผ่านตามขอบเขตในหัวข้อ 4.10 ส่วนการเล่นครบทุกเส้นทางและความพึงพอใจของผู้ใช้ยังต้องประเมินเพิ่มเติม

ผลการตรวจโครงสร้างแอนิเมชันพบว่ามีคลิป Idle และ Walk สำหรับผู้เล่น และ Idle, Walk, Run, Attack1 และ Scream สำหรับ Killer พร้อมการส่งพารามิเตอร์ Bool และ Trigger จากสคริปต์ตามหัวข้อ 4.9 ผลทดสอบ Controller ใน Play Mode ผ่าน AN-P01 และ AN-K01–AN-K04 ตามตารางที่ 4-8

การทดสอบเพิ่มเติมยืนยันผลในกรณีควบคุมรวม 35 กรณีหลัก ขอบเขตนี้ครอบคลุมองค์ประกอบและการเปลี่ยนสถานะที่กำหนด ยังไม่ใช่การเล่นครบทุกเส้นทาง การประเมินโดยผู้ใช้ หรือการรับรองประสิทธิภาพทุกอุปกรณ์ คะแนนความพึงพอใจยังเป็นตัวอย่างสมมติ

# บทที่ 5

# สรุปผลโครงงานและข้อเสนอแนะ

บทนี้นำเสนอสรุปผลการพัฒนาโครงงานเกมคอมพิวเตอร์ 2 มิติแนว Side-scrolling Hide-and-Seek Horror Game โดยสรุปตามวัตถุประสงค์และขอบเขตของโครงงาน อธิบายวิธีดำเนินงานโดยย่อ ผลที่ได้จากการพัฒนาระบบ ปัญหาและอุปสรรคที่พบ ตลอดจนข้อเสนอแนะสำหรับการนำผลงานไปใช้และการพัฒนาต่อยอดในอนาคต

## 5.1 สรุปผลโครงงาน

โครงงานนี้มีวัตถุประสงค์เพื่อพัฒนาเกมคอมพิวเตอร์ 2 มิติแนว Side-scrolling Hide-and-Seek Horror Game ด้วย Unity และภาษา C# โดยนำเทคนิค Finite State Machine หรือ FSM มาใช้ควบคุมสถานะของตัวละครผู้เล่นและพฤติกรรมของศัตรู เพื่อให้แต่ละระบบเปลี่ยนพฤติกรรมตามเงื่อนไขที่กำหนดไว้อย่างเป็นลำดับ นอกจากนี้ยังมีวัตถุประสงค์เพื่อพัฒนาระบบตรวจจับผู้เล่น ระบบไอเท็ม ระบบประตูทางออก ส่วนติดต่อผู้ใช้งาน และด่านที่มีระดับความยากแตกต่างกัน

การดำเนินโครงงานเริ่มจากการศึกษาหลักการพัฒนาเกม 2 มิติ การใช้งาน Unity การเขียนโปรแกรมด้วยภาษา C# และแนวคิด Finite State Machine จากนั้นจึงวิเคราะห์ความต้องการ ออกแบบโครงสร้างระบบ พัฒนาส่วนประกอบต่าง ๆ และทดสอบการทำงานภายในเกม โดยแบ่งระบบออกเป็นส่วนย่อย เพื่อให้สามารถพัฒนาและแก้ไขแต่ละระบบได้อย่างเป็นอิสระ

ผลการดำเนินโครงงานสามารถสรุปตามวัตถุประสงค์ได้ดังนี้

ผลทดสอบทางเทคนิคใน Unity 6000.4.9f1 เมื่อวันที่ 4 ตุลาคม 2569 ผ่านครบ 35 กรณีหลัก แบ่งเป็นระบบควบคุมพื้นฐาน 5 กรณี Player FSM 7 กรณี Enemy FSM 10 กรณี ระบบเกมและด่าน 8 กรณี และแอนิเมชัน 5 กรณี รายละเอียดผลระบบ 30 กรณีอยู่ในหัวข้อ 3.6 และ 4.10 ส่วนผลแอนิเมชันอยู่ในตารางที่ 4-8 การทดสอบใช้ Play Mode แบบควบคุมอินพุต ตำแหน่ง เวลา และเงื่อนไขขององค์ประกอบ จึงยืนยันเฉพาะกรณีที่กำหนดไว้ ยังไม่ใช่ผลการเล่นครบทุกเส้นทางหรือการประเมินโดยผู้ใช้งาน

### 5.1.1 ผลการพัฒนาระบบควบคุมตัวละครผู้เล่น

สามารถพัฒนาระบบควบคุมตัวละครผู้เล่นให้รองรับการเดิน วิ่ง กระโดด ใช้ไฟฉาย เก็บไอเท็ม ซ่อนตัว เปิดกระเป๋าไอเท็ม และโต้ตอบกับประตูทางออกได้ การวิ่งส่งผลให้ผู้เล่นเคลื่อนที่เร็วขึ้น แต่ทำให้ Stamina ลดลงและสร้างเสียงที่ศัตรูสามารถตรวจจับได้ ส่วนไฟฉายมีปริมาณแบตเตอรี่จำกัด ทำให้ผู้เล่นต้องวางแผนการใช้งานระหว่างสำรวจด่าน

ระบบควบคุมผู้เล่นใช้ FSM แบ่งสถานะออกเป็น Normal, Hidden, Frozen, Hurt, UsingItem, Transitioning และ Dead การแบ่งสถานะดังกล่าวช่วยควบคุมเงื่อนไขการรับคำสั่งและป้องกันพฤติกรรมที่ขัดแย้งกัน เช่น การควบคุมตัวละครระหว่างเสียชีวิต การเคลื่อนที่ขณะถูกหยุด หรือการรับคำสั่งระหว่างเปลี่ยนฉาก

**5.1.2 ผลการพัฒนาระบบปัญญาประดิษฐ์ของศัตรู**

สามารถพัฒนาระบบปัญญาประดิษฐ์ของศัตรูให้ตอบสนองต่อผู้เล่นผ่านการมองเห็นและการได้ยิน ศัตรูสามารถลาดตระเวน รับฟังเสียง ตรวจสอบตำแหน่งที่เกิดเสียง ค้นหาผู้เล่น ไล่ล่า โจมตี และกลับไปยังเส้นทางลาดตระเวนได้ Enemy FSM ประกอบด้วยสถานะ Patrol, Alert, Listening, InvestigateSound, ConfirmSound, Confused, Investigate, Search, Chase, Attack, AttackRecover, ReturnToPatrol และ Scream สถานะเหล่านี้ช่วยให้ศัตรูมีพฤติกรรมหลายขั้นตอนและตอบสนองต่อสถานการณ์ได้เหมาะสมกว่าการกำหนดเงื่อนไขทั้งหมดไว้ในกระบวนการเดียว

เมื่อศัตรูได้ยินเสียงจากการเดิน วิ่ง หรือกับดัก ศัตรูจะจดจำตำแหน่งกำเนิดเสียงไว้ชั่วคราวและเคลื่อนที่ไปตรวจสอบ หากมองเห็นผู้เล่นจะเปลี่ยนเข้าสู่สถานะ Chase และโจมตีเมื่อผู้เล่นอยู่ในระยะที่กำหนด หากสูญเสียการตรวจจับ ศัตรูจะตรวจสอบตำแหน่งล่าสุดหรือค้นหาในบริเวณใกล้เคียงก่อนกลับไปลาดตระเวน

**5.1.3 ผลการพัฒนาระบบตรวจจับ**

สามารถพัฒนาระบบตรวจจับด้วยการมองเห็นและการได้ยินให้ทำงานร่วมกับ Enemy FSM ได้ ระบบการมองเห็นพิจารณาระยะตรวจจับ ทิศทาง และสิ่งกีดขวางภายในฉาก ส่วนระบบการได้ยินรับเหตุการณ์เสียงจากการกระทำของผู้เล่นและวัตถุภายในด่าน

เสียงแต่ละประเภทมีความสำคัญแตกต่างกัน โดยเสียงจากการวิ่งและเสียงจากกับดักกระจกสามารถดึงดูดความสนใจของศัตรูได้มากกว่าเสียงเดินทั่วไป ระบบดังกล่าวทำให้เสียงเป็นส่วนหนึ่งของการเล่น ผู้เล่นจึงต้องเลือกระหว่างการเคลื่อนที่อย่างรวดเร็วกับการลดโอกาสที่ศัตรูจะตรวจพบตำแหน่ง

**5.1.4 ผลการพัฒนาระบบไอเท็มและประตูทางออก**

สามารถพัฒนาระบบเก็บกุญแจและกระเป๋าไอเท็มโดยใช้ InventoryManager สำหรับจัดเก็บและตรวจสอบจำนวนไอเท็ม เมื่อผู้เล่นเก็บกุญแจ ข้อมูลบน HUD และในกระเป๋าไอเท็มจะได้รับการปรับปรุงตามจำนวนที่เก็บได้

ระบบประตูทางออกใช้ FSM แบ่งสถานะเป็น Locked, Ready, Unlocking, Open, Transitioning และ Completed ประตูจะยังไม่สามารถเปิดได้หากผู้เล่นเก็บกุญแจไม่ครบ เมื่อผู้เล่นมีจำนวนกุญแจตามที่กำหนดและกดปุ่มโต้ตอบค้าง ระบบจะแสดงความคืบหน้าการปลดล็อก ก่อนเปิดประตูและเปลี่ยนไปยังด่านถัดไปหรือแสดงผลการจบเกม

**5.1.5 ผลการพัฒนาด่านและบรรยากาศของเกม**

สามารถพัฒนาด่านภายในเกมให้มีระดับความยากแตกต่างกันผ่าน LevelConfig ซึ่งกำหนดค่าต่าง ๆ เช่น ความเร็วของศัตรู ระยะการมองเห็น ระยะการได้ยิน ความเสียหาย และปริมาณแบตเตอรี่เริ่มต้น

Level 3 ได้รับการออกแบบให้เป็นสวนสนุก โดยต้นไม้ พุ่มไม้ พื้นหญ้า และเส้นทางต่างระดับเพื่อสร้างบรรยากาศน่ากลัว ภายในด่านมีการสุ่มตำแหน่งกุญแจและกับดักในพื้นที่ที่ผู้เล่นสามารถเดินถึง ทำให้การเล่นแต่ละรอบมีความแตกต่างกัน

ศัตรูใน Level 3 เน้นการตรวจจับด้วยเสียง มีความเร็วในการไล่ล่าและระยะการได้ยินสูงกว่าด่านอื่น เมื่อผู้เล่นเข้าใกล้ศัตรูหรือถูกไล่ล่า ระบบจะเล่นเสียงหัวใจ แสดงขอบจอเตือน และปรับระดับดนตรีตามสถานการณ์ ส่งผลให้ผู้เล่นรับรู้ถึงอันตรายและความกดดันได้อย่างชัดเจน

**5.1.6 ผลการพัฒนาส่วนติดต่อผู้ใช้งาน**

สามารถพัฒนาส่วนติดต่อผู้ใช้งานให้แสดงข้อมูลสำคัญระหว่างเล่น ได้แก่ พลังชีวิต Stamina ปริมาณแบตเตอรี่ จำนวนกุญแจ ข้อความโต้ตอบ สถานะการซ่อน และคำเตือนเมื่อศัตรูอยู่ใกล้

ระบบแสดงผลจะเปลี่ยนตามสถานการณ์ เช่น แสดงข้อความแนะนำเมื่อผู้เล่นเข้าใกล้วัตถุ แสดง DANGER เมื่อศัตรูไล่ล่า แสดง Game Over เมื่อผู้เล่นเสียชีวิต และแสดง Level Complete หรือ Game Won เมื่อผู้เล่นผ่านเงื่อนไขของด่าน

**5.1.7 ผลการพัฒนาระบบแอนิเมชัน**

ผู้เล่นใช้คลิป Idle และ Walk โดยพารามิเตอร์ Bool isWalking ควบคุมการเปลี่ยนภาพตามการเคลื่อนที่ ส่วน Killer ใช้ Idle, Walk, Run, Attack1 และ Scream โดยใช้ Bool isWalking และ isRunning รักษาสถานะการเดินและวิ่ง และใช้ Trigger attack1 กับ scream เริ่มการโจมตีและกรีดร้องเฉพาะครั้ง FSM ตัดสินใจพฤติกรรม ขณะที่ Animator แสดงภาพของพฤติกรรมนั้น

ผลทดสอบ Controller เดิมใน Play Mode ผ่าน AN-P01 และ AN-K01–AN-K04 รวม 5 กรณี ได้แก่ Idle ⟶ Walk ⟶ Idle ของผู้เล่น การเดินและวิ่งของ Killer การเล่น Attack1 แล้วกลับ Idle การเข้าสู่ Scream จาก Idle, Walk, Run และ Attack1 แล้วกลับ Idle และการออกจาก Run ไป Walk เมื่อ isRunning เปลี่ยนเป็น false ทั้งนี้ Float และ Int ยังไม่ได้ใช้ใน Controller ปัจจุบัน จึงเป็นแนวทางขยายระบบตามหัวข้อ 3.5.11 และไม่มีผลทดสอบของพารามิเตอร์สองชนิดนี้

**5.1.8 อภิปรายผลการพัฒนา**

ผลการพัฒนาแสดงให้เห็นว่า Finite State Machine เหมาะสำหรับระบบที่ประกอบด้วยพฤติกรรมหลายสถานะและมีเงื่อนไขการเปลี่ยนสถานะที่ชัดเจน เนื่องจากช่วยแยกหน้าที่ของแต่ละสถานะ ลดความซับซ้อนของการตรวจสอบเงื่อนไข และช่วยให้สามารถติดตามพฤติกรรมของระบบได้ง่ายขึ้น

ผลดังกล่าวสอดคล้องกับหลักการของ FSM ที่กำหนดให้ระบบอยู่ในสถานะหนึ่ง ณ ช่วงเวลาหนึ่ง และเปลี่ยนไปยังสถานะใหม่เมื่อเกิดเหตุการณ์หรือเป็นไปตามเงื่อนไขที่กำหนด การนำแนวคิดนี้มาใช้กับผู้เล่นช่วยป้องกันคำสั่งที่ขัดแย้งกัน ส่วนการใช้กับศัตรูช่วยให้กระบวนการลาดตระเวน ตรวจจับ ค้นหา ไล่ล่า และโจมตีมีลำดับที่ชัดเจน

การใช้ระบบตรวจจับด้วยเสียงร่วมกับ FSM ช่วยเพิ่มความหลากหลายของพฤติกรรมศัตรู เนื่องจากศัตรูไม่จำเป็นต้องมองเห็นผู้เล่นโดยตรงจึงจะตอบสนองได้ อย่างไรก็ตาม ความสมจริงของศัตรูยังขึ้นอยู่กับการกำหนดค่าความเร็ว ระยะตรวจจับ ระยะเวลาของแต่ละสถานะ และรูปแบบพื้นที่ภายในด่าน หากกำหนดค่าไม่เหมาะสม เกมอาจง่ายหรือยากเกินไป

การแบ่งระบบออกเป็นส่วนย่อย เช่น PlayerController, KillerAI, InventoryManager, ExitDoor, GameManager, UIManager และ TensionDirector ช่วยให้สามารถปรับปรุงแต่ละระบบได้โดยไม่กระทบส่วนอื่นมากเกินไป และช่วยให้ตรวจสอบข้อผิดพลาดได้สะดวกกว่าการรวมการทำงานทั้งหมดไว้ในสคริปต์เดียว

ผลที่ผ่านทั้ง 35 กรณีสนับสนุนว่าการเปลี่ยนสถานะและการเชื่อม Animator ทำงานได้ตามเงื่อนไขควบคุมที่กำหนด อย่างไรก็ตาม การทดสอบใช้การเรียกองค์ประกอบและจัดสภาพแวดล้อมผ่านสคริปต์ชั่วคราว จึงยังต้องตรวจการทำงานร่วมกันระหว่างเล่นครบเส้นทาง ความเสถียรเมื่อเล่นต่อเนื่องและการตอบสนองบนอุปกรณ์ต่างกันก่อนสรุปความพร้อมสำหรับเผยแพร่

ยังไม่ได้เก็บแบบสอบถามจากผู้ทดลองเล่น จึงไม่สามารถสรุประดับความพึงพอใจ ความสมดุลของความยาก หรือความน่ากลัวที่ผู้เล่นรับรู้ได้ คะแนน ค่าเฉลี่ย และส่วนเบี่ยงเบนมาตรฐานตัวอย่างในบท 4 เป็นข้อมูลสมมติสำหรับแสดงวิธีคำนวณเท่านั้น ไม่ใช่หลักฐานผลประเมินจริงและไม่ใช้สนับสนุนข้อสรุปด้านประสบการณ์ผู้เล่น

## 5.2 ปัญหาที่พบในโครงงาน

### 5.2.1 ปัญหาการเปลี่ยนสถานะของ FSM

ในระหว่างการพัฒนา พบว่าบางสถานการณ์ทำให้ระบบพยายามเปลี่ยนสถานะมากกว่าหนึ่งครั้งในเวลาใกล้เคียงกัน เช่น ผู้เล่นได้รับความเสียหายขณะเปิดกระเป๋า ถูกศัตรูโจมตีระหว่างปลดล็อกประตู หรือศัตรูได้ยินเสียงพร้อมกับมองเห็นผู้เล่น ปัญหานี้อาจทำให้คำสั่งบางอย่างทำงานซ้อนกันหรือระบบเปลี่ยนไปยังสถานะที่ไม่เหมาะสม

แนวทางแก้ไขคือกำหนดลำดับความสำคัญของสถานะ ตรวจสอบสถานะปัจจุบันก่อนรับคำสั่ง และจำกัดให้แต่ละระบบเปลี่ยนสถานะผ่านฟังก์ชันส่วนกลาง เช่น SetPlayerState() หรือ SetState() เพื่อให้สามารถควบคุมการเปลี่ยนสถานะได้จากตำแหน่งเดียว

### 5.2.2 ปัญหาการตรวจจับของศัตรู

ระบบการมองเห็นของศัตรูอาจตรวจจับผิดพลาดเมื่อผู้เล่นอยู่ใกล้ขอบสิ่งกีดขวาง หรือเมื่อ Collider ของวัตถุไม่ตรงกับภาพที่แสดงในฉาก ส่วนระบบการได้ยินอาจทำให้ศัตรูตอบสนองต่อเสียงหลายเหตุการณ์พร้อมกัน จนเกิดการเปลี่ยนเป้าหมายบ่อยเกินไป

แนวทางแก้ไขคือปรับ LayerMask และ Collider ให้ตรงกับวัตถุ ตรวจสอบแนว Raycast ที่ใช้ในการมองเห็น และกำหนดระดับความสำคัญของเสียง ศัตรูจะเลือกตรวจสอบเสียงที่มีความสำคัญมากกว่าหรือเสียงที่เกิดขึ้นล่าสุด พร้อมเก็บตำแหน่งเสียงไว้เพียงช่วงเวลาที่กำหนด

### 5.2.3 ปัญหาการเคลื่อนที่ของศัตรูในฉากสองมิติ

ศัตรูอาจติดขอบพื้น สิ่งกีดขวาง หรือพื้นที่ต่างระดับ โดยเฉพาะฉากที่ประกอบจาก Tilemap หลายชั้น หากจุดลาดตระเวนอยู่ในตำแหน่งที่ศัตรูเดินไม่ถึง ระบบอาจพยายามเคลื่อนที่ไปยังเป้าหมายเดิมอย่างต่อเนื่อง

แนวทางแก้ไขคือกำหนดเส้นทางลาดตระเวนให้สอดคล้องกับพื้นที่ที่เดินได้ ตรวจสอบพื้นและขอบทางเดินก่อนเปลี่ยนทิศทาง รวมถึงกำหนดเงื่อนไขให้ศัตรูกลับไปยังจุดลาดตระเวนที่เหมาะสมเมื่อไม่สามารถเดินทางไปยังเป้าหมายเดิมได้

**5.2.4 ปัญหาการสุ่มตำแหน่งวัตถุใน Level 3**

การสุ่มตำแหน่งกุญแจและกับดักอาจทำให้วัตถุปรากฏซ้อนกัน อยู่ใกล้กันเกินไป หรืออยู่ในตำแหน่งที่ผู้เล่นไม่สามารถเข้าถึงได้ หากสุ่มโดยใช้พิกัดทั่วไปโดยไม่ตรวจสอบพื้นภายในฉาก

แนวทางแก้ไขคือให้ Level3RandomEvents เลือกตำแหน่งจากพื้นที่ที่ตรวจสอบแล้วว่าสามารถเดินได้ กำหนดระยะห่างขั้นต่ำระหว่างวัตถุ และแยกจุดสุ่มของกุญแจกับกับดัก เพื่อลดโอกาสที่วัตถุหลายชิ้นจะรวมอยู่ในบริเวณเดียวกัน

**5.2.5 ปัญหาการปรับสมดุลระดับความยาก**

ค่าความเร็ว ระยะการมองเห็น ระยะการได้ยิน และความเสียหายของศัตรูมีผลต่อความยากอย่างมาก หากเพิ่มหลายค่าพร้อมกัน ผู้เล่นอาจไม่มีเวลาเพียงพอสำหรับหลบหนีหรือหาจุดซ่อน โดยเฉพาะ Level 3 ซึ่งกำหนดให้ศัตรูมีความเร็วและระยะการได้ยินสูง

แนวทางแก้ไขคือแยกค่าความยากของแต่ละด่านไว้ใน LevelConfig และปรับค่าทีละส่วนจากผลการทดลองเล่น เช่น ปรับความเร็วไล่ล่า ระยะได้ยิน เวลาค้นหา และจำนวนจุดซ่อน เพื่อหาค่าที่สร้างความกดดันโดยยังเปิดโอกาสให้ผู้เล่นสามารถผ่านด่านได้

**5.2.6 ปัญหาการจัดวางกล้องและองค์ประกอบของด่าน**

กล้องอาจอยู่ต่ำหรือสูงเกินไป ทำให้ไม่สามารถมองเห็นเส้นทางและองค์ประกอบสำคัญของด่านได้ครบถ้วน นอกจากนี้ การใช้ขอบเขตกล้องร่วมกันกับด่านที่มีขนาดแตกต่างกันอาจทำให้เกิดพื้นที่ว่างนอกฉากหรือทำให้วัตถุบางส่วนอยู่นอกหน้าจอ

แนวทางแก้ไขคือกำหนดตำแหน่งและขอบเขตกล้องแยกตามแต่ละด่าน ปรับค่าแกนตั้งของกล้องให้เหมาะกับระดับพื้น และทดสอบบริเวณขอบซ้าย ขอบขวา พื้นที่ชั้นบน และพื้นที่ชั้นล่างก่อนสรุปการจัดวางด่าน

**5.2.7 ปัญหาการทำงานร่วมกันของหลายระบบ**

ระบบผู้เล่น ศัตรู UI เสียง ไอเท็ม และการเปลี่ยนฉากมีการส่งข้อมูลถึงกันหลายส่วน หากวัตถุหรือคอมโพเนนต์บางรายการไม่ได้กำหนดใน Inspector อาจเกิดข้อผิดพลาดหรือทำให้ฟังก์ชันบางอย่างไม่ทำงาน

แนวทางแก้ไขคือเพิ่มการตรวจสอบค่าว่างก่อนเรียกใช้งาน แยกความรับผิดชอบของแต่ละสคริปต์ ใช้ Event สำหรับส่งข้อมูลระหว่างระบบที่ไม่จำเป็นต้องอ้างอิงกันโดยตรง และตรวจสอบ Unity Console ทุกครั้งหลังแก้ไข Scene, Prefab หรือสคริปต์ส่วนกลาง

### 5.2.8 สิ่งที่ได้เรียนรู้จากการพัฒนา

ผู้พัฒนาได้เรียนรู้ว่าการออกแบบโครงสร้างระบบก่อนเขียนโปรแกรมช่วยลดการแก้ไขโค้ดในภายหลัง การแบ่งระบบออกเป็นโมดูลและกำหนดสถานะด้วย FSM ทำให้สามารถวิเคราะห์สาเหตุของข้อผิดพลาดได้ชัดเจนขึ้น

นอกจากนี้ การทดสอบจากการเล่นจริงมีความสำคัญต่อการพัฒนาเกม เนื่องจากระบบที่ทำงานถูกต้องตามโค้ดอาจยังไม่ให้ประสบการณ์ที่เหมาะสม เช่น ศัตรูอาจเร็วเกินไป คำเตือนอาจไม่ชัดเจน หรือการจัดวางวัตถุอาจทำให้ผู้เล่นไม่เข้าใจเส้นทาง ดังนั้นผลจากผู้ทดลองเล่นจึงควรถูกนำมาใช้ร่วมกับการทดสอบทางเทคนิค

## 5.3 ข้อเสนอแนะ

### 5.3.1 ข้อเสนอแนะสำหรับการนำระบบไปใช้

ก่อนนำเกมไปใช้งานหรือเผยแพร่ ควรต่อยอดจากผลทดสอบองค์ประกอบ 35 กรณีที่ผ่านแล้วด้วยการเล่นครบเส้นทางตั้งแต่เริ่มจนจบทั้งสามด่าน ทดสอบการทำงานร่วมกันของผู้เล่น ศัตรู ไอเท็ม ประตู UI และเสียง รวมถึงกรณีขอบเขต เช่น การเปลี่ยนสถานะพร้อมกัน การยกเลิกปลดล็อกในหลายจังหวะ และการเล่นซ้ำต่อเนื่อง โดยบันทึกผลและข้อผิดพลาดจริงแยกจากผลทดสอบแบบควบคุม

ควรทดสอบกับอุปกรณ์ที่มีประสิทธิภาพแตกต่างกัน ตรวจสอบอัตราเฟรม ความละเอียดหน้าจอ ตำแหน่ง UI และการตอบสนองของปุ่มควบคุม เนื่องจากผลการทำงานอาจแตกต่างกันตามขนาดหน้าจอและความสามารถของอุปกรณ์

การปรับค่าความยากผ่าน LevelConfig ควรดำเนินการอย่างระมัดระวัง เพราะความเร็ว ระยะตรวจจับ ความเสียหาย และทรัพยากรเริ่มต้นมีผลต่อกัน หากต้องการปรับความยากควรเปลี่ยนค่าทีละส่วนและทดลองเล่นซ้ำก่อนนำไปใช้จริง

### 5.3.2 ข้อเสนอแนะสำหรับการพัฒนาระบบเพิ่มเติม

ควรเพิ่มรูปแบบพฤติกรรมของศัตรูให้หลากหลาย เช่น การตรวจสอบจุดซ่อน การคาดเดาเส้นทางของผู้เล่น การตอบสนองร่วมกันของศัตรูหลายตัว และการจดจำพื้นที่ที่พบผู้เล่นบ่อย เพื่อให้การเล่นแต่ละรอบไม่สามารถคาดเดาได้ง่าย

ควรเพิ่มความหลากหลายของไอเท็ม เช่น ไอเท็มฟื้นฟูพลังชีวิต แบตเตอรี่สำรอง อุปกรณ์สร้างเสียงล่อศัตรู หรือไอเท็มป้องกันการโจมตีชั่วคราว โดยต้องกำหนดข้อจำกัดเพื่อไม่ให้ไอเท็มลดความน่ากลัวของเกมมากเกินไป

ควรพัฒนาระบบตั้งค่าให้รองรับการปรับระดับเสียง ความสว่าง ความละเอียดหน้าจอ และการกำหนดปุ่มควบคุมใหม่ รวมถึงเพิ่มตัวเลือกด้านการเข้าถึง เช่น ขนาดข้อความ คำบรรยายเสียง และการปรับความรุนแรงของเอฟเฟกต์ภาพ

ควรเพิ่มระบบบันทึกความคืบหน้าในอนาคต โดยต้องกำหนดข้อมูลที่จำเป็น เช่น ด่านปัจจุบัน จำนวนไอเท็ม สถานะผู้เล่น และค่าตั้งค่า พร้อมตรวจสอบความถูกต้องของข้อมูลก่อนโหลดกลับเข้าสู่เกม

### 5.3.3 ข้อเสนอแนะสำหรับการศึกษาและวิจัยครั้งต่อไป

การศึกษาครั้งต่อไปควรเปรียบเทียบประสิทธิภาพของ FSM กับเทคนิคควบคุมพฤติกรรมแบบอื่น เช่น Behavior Tree หรือ Utility AI โดยพิจารณาความซับซ้อนของโค้ด เวลาในการประมวลผล ความสะดวกในการเพิ่มพฤติกรรม และความสมจริงที่ผู้เล่นรับรู้

ควรศึกษาผลของเสียงต่อพฤติกรรมของผู้เล่น โดยทดลองปรับระดับความดัง ความถี่ของเสียงหัวใจ และระยะการได้ยินของศัตรู แล้วประเมินว่าค่าใดสามารถสร้างความกดดันได้เหมาะสมโดยไม่ทำให้ผู้เล่นสับสนหรือรู้สึกว่าเกมไม่ยุติธรรม

ควรศึกษาการสุ่มตำแหน่งวัตถุและเหตุการณ์ภายในด่านว่ามีผลต่อความน่ากลัวและความต้องการเล่นซ้ำหรือไม่ โดยเปรียบเทียบระหว่างด่านที่มีตำแหน่งคงที่กับด่านที่มีการสุ่ม พร้อมเก็บข้อมูลระยะเวลาเล่น จำนวนครั้งที่ถูกจับ และความคิดเห็นของผู้เล่น

ควรเก็บข้อมูลจากผู้ทดลองเล่นจริง โดยคัดเลือกผู้เล่นใหม่และผู้ที่คุ้นเคยกับเกมแนวสยองขวัญ กำหนดขั้นตอนเล่นให้เหมือนกัน ระบุจำนวนผู้ตอบ ช่วงเวลาทดสอบ คะแนนทั้ง 6 ด้าน และข้อเสนอแนะ จากนั้นคำนวณค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐานจากข้อมูลจริง เพื่อใช้ปรับระดับความยาก การควบคุม และการนำเสนอข้อมูลบนหน้าจอ